



# *UNIT-6: EXPLORING GRAPH*

## *INTRODUCTION OF GRAPH*

*Mr. Prayag Patel*

*Assistant Professor  
IT-ICT Department, LJIET  
[prayag.patel@ljinstitutes.edu.in](mailto:prayag.patel@ljinstitutes.edu.in)*



# *Outline*

- ❖ *Introduction using Graphs and Games.*
- ❖ *Concept of Graph.*
- ❖ *Representation of Graph.*



# *Introduction using Graphs and Games*

- ❖ *Various problem can be formulated using graph.*
- ❖ *Game playing application can be represented using graphs.*
- ❖ *Consider a game named Nim which its popular in china.*



# *Introduction using Graphs and Games*

## *❖ Rules for Game:*

- 1) Initially there is a heap of objects between two players.*
- 2) Each player can pick up the object alternatively.*
- 3) Each player can pick up at most twice the number of objects than its opponent.*
- 4) An object is removed from one row at a time.*
- 5) The person who pick up lastly will lost game and opponent will win.*



# *Introduction using Graphs and Games*

❖ *Example: Heap of object is arranged in two rows namely **A & B**. There are two players : **I & YOU***

| <b>A</b> | <b>B</b> |
|----------|----------|
| <b>3</b> | <b>2</b> |



# *Introduction using Graphs and Games*

| A | B |
|---|---|
| 3 | 2 |

| A | B | Moves              |
|---|---|--------------------|
| 3 | 2 | I take 1 from B    |
| 3 | 1 | YOU take 2 from A  |
| 1 | 1 | I take 1 from A    |
| 0 | 1 | YOU take 1 from B  |
|   |   | I win & you loose. |

| A | B | Moves              |
|---|---|--------------------|
| 3 | 2 | I take 1 from A    |
| 2 | 2 | YOU take 1 from B  |
| 2 | 1 | I take 1 from A    |
| 1 | 1 | YOU take 1 from B  |
| 1 | 0 | I take 1 from A    |
|   |   | I loose & you win. |



# Introduction using Graphs and Games

| A | B |
|---|---|
| 3 | 2 |

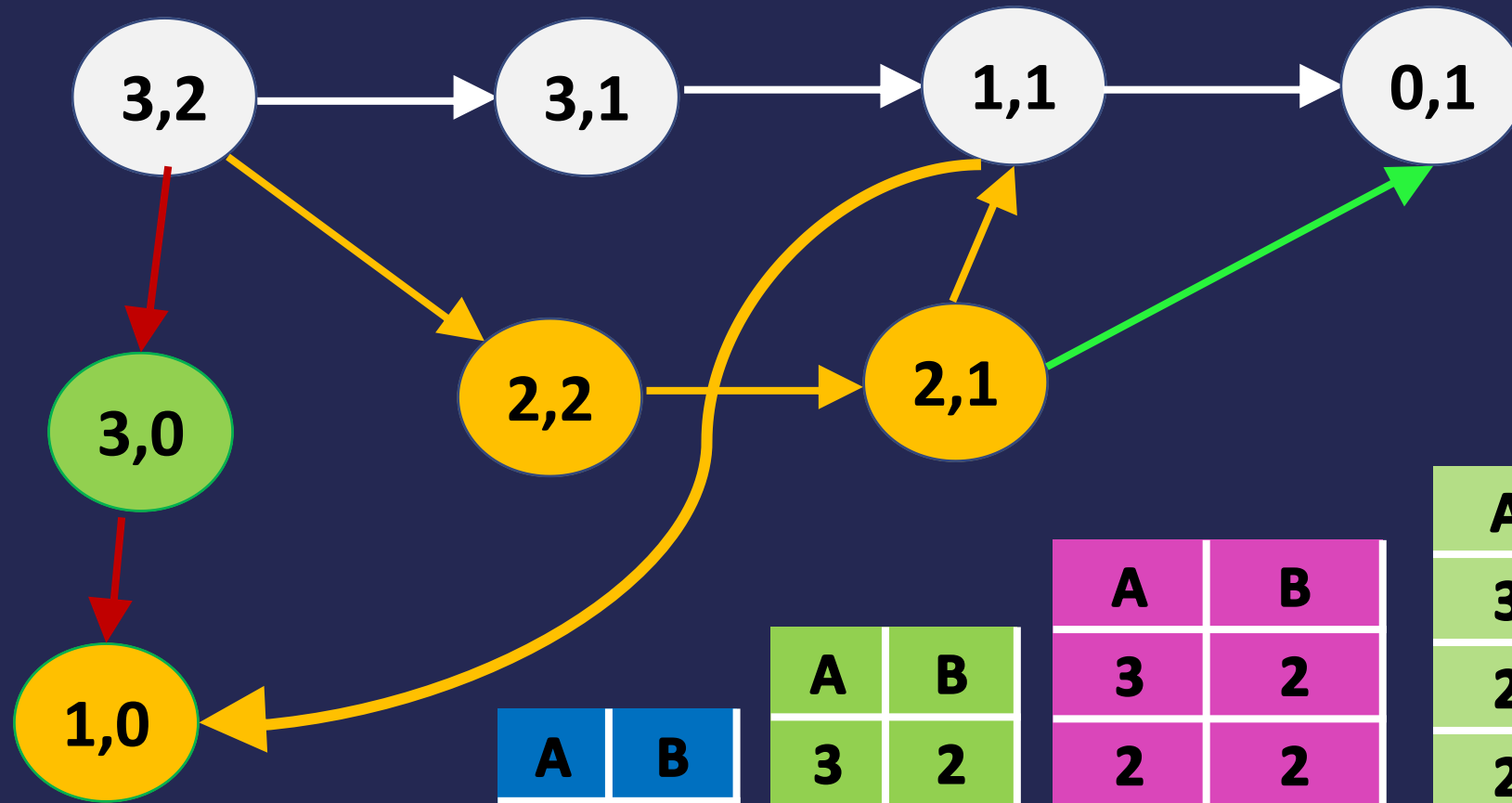
| A | B | Moves              |
|---|---|--------------------|
| 3 | 2 | I take 1 from A    |
| 2 | 2 | YOU take 1 from B  |
| 2 | 1 | I take 2 from A    |
| 0 | 1 | YOU take 1 from B  |
|   |   | I win & you loose. |

| A | B | Moves              |
|---|---|--------------------|
| 3 | 2 | I take 2 from B    |
| 3 | 0 | YOU take 2 from A  |
| 1 | 0 | I take 1 from A    |
|   |   | I loose & you win. |

| A | B | Moves              |
|---|---|--------------------|
| 3 | 2 | I take 2 from B    |
| 3 | 0 | YOU take 3 from A  |
|   |   | I win & you loose. |



# Introduction using Graphs and Games



| A | B |
|---|---|
| 3 | 2 |
| 3 | 0 |

| A | B |
|---|---|
| 3 | 2 |
| 3 | 0 |
| 1 | 0 |

| A | B |
|---|---|
| 3 | 2 |
| 2 | 2 |
| 2 | 1 |
| 0 | 1 |

| A | B |
|---|---|
| 3 | 2 |
| 2 | 2 |
| 2 | 1 |
| 1 | 1 |
| 1 | 0 |

| A | B |
|---|---|
| 3 | 2 |
| 3 | 1 |
| 1 | 1 |
| 0 | 1 |





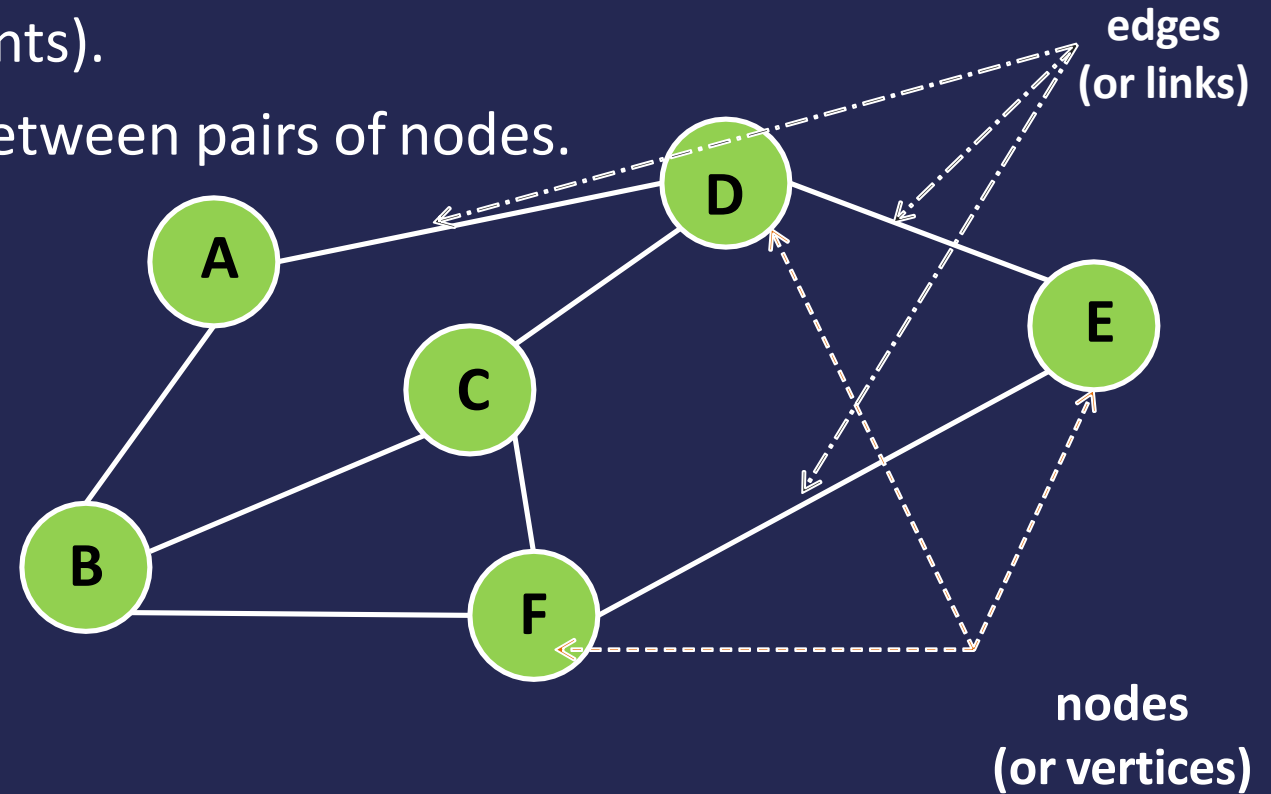
# Concept of Graph

## ❖ Graph:

- A graph is a collection of nodes and edges
- Denoted by  $G = (V, E)$ .

$V$  = **nodes** (vertices, points).

$E$  = **edges** (links, arcs) between pairs of nodes.

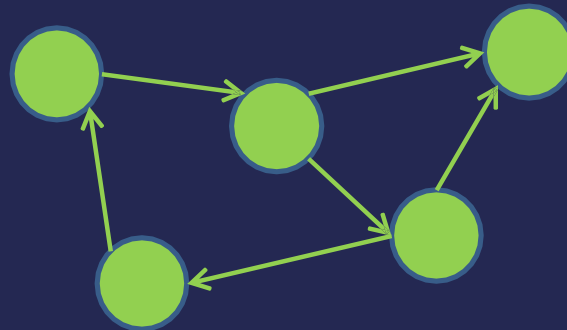




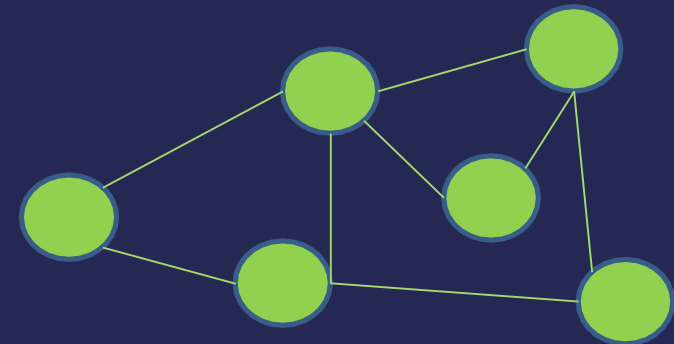
# Concept of Graph

## ❖ *Directed & Undirected Graph:*

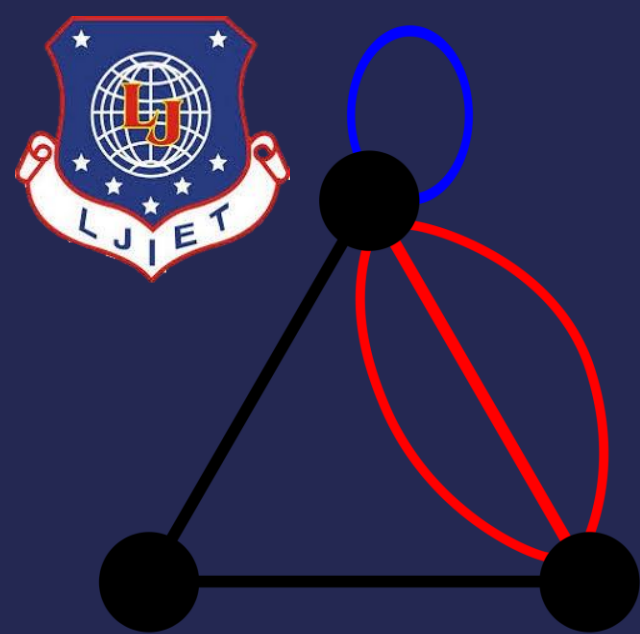
- **Directed Graph:** A graph in which **every edge is directed** from one node to another is called a directed graph or digraph.
- **Undirected Graph:** A graph in which **every edge is undirected and no direction is associated with them** is called an undirected graph.



Directed Graph



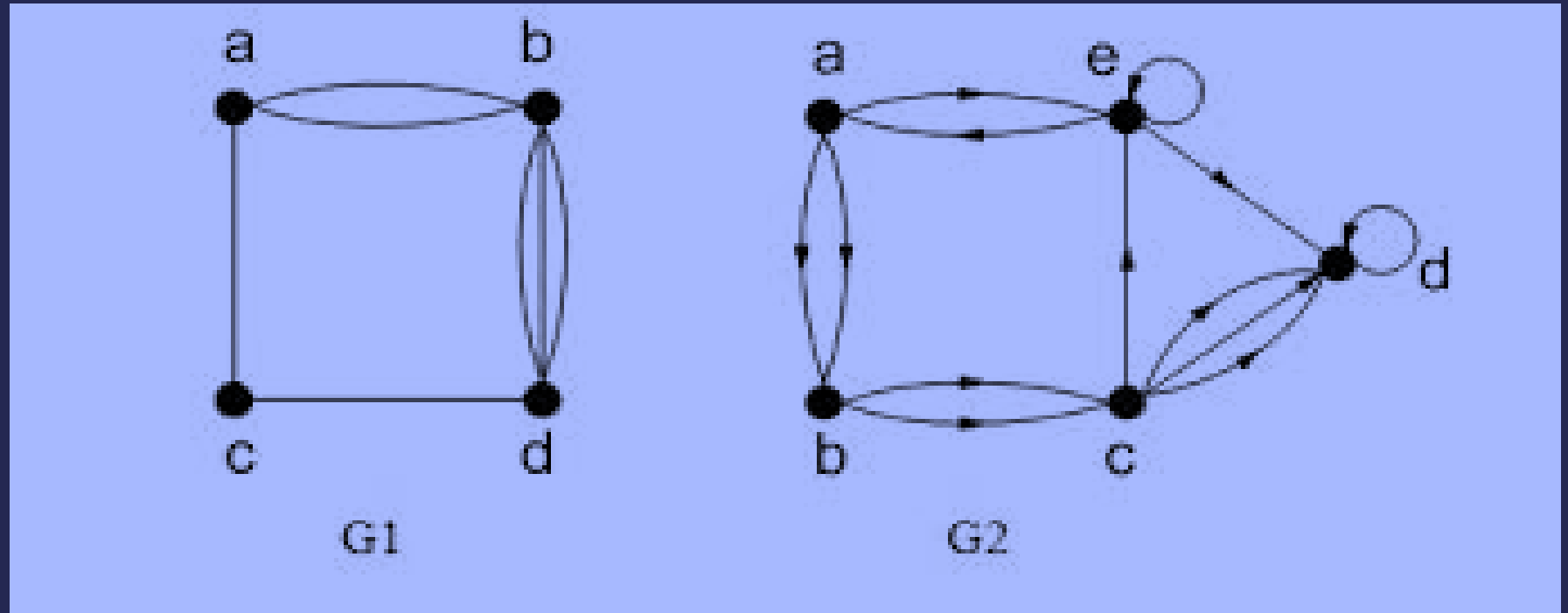
Undirected Graph



# Concept of Graph

## ❖ Multigraph:

- A **multigraph** is a **graph** which is permitted to have **multiple** edges (also called parallel edges), that is, edges that have the same end nodes. Thus two vertices may be connected by more than one edge.

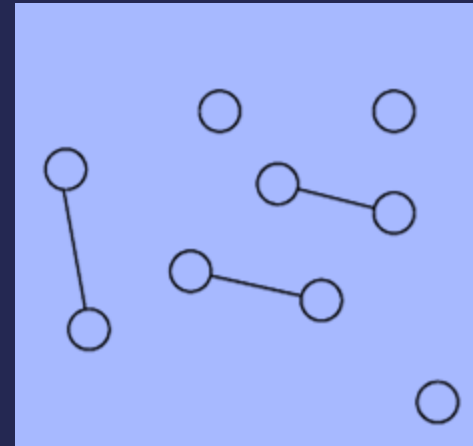




# Concept of Graph

## ❖ *Sparse Graph:*

- Sparse graph is a graph in which the number of edges is close to the minimal number of edges.
- Sparse graph can be a disconnected graph.

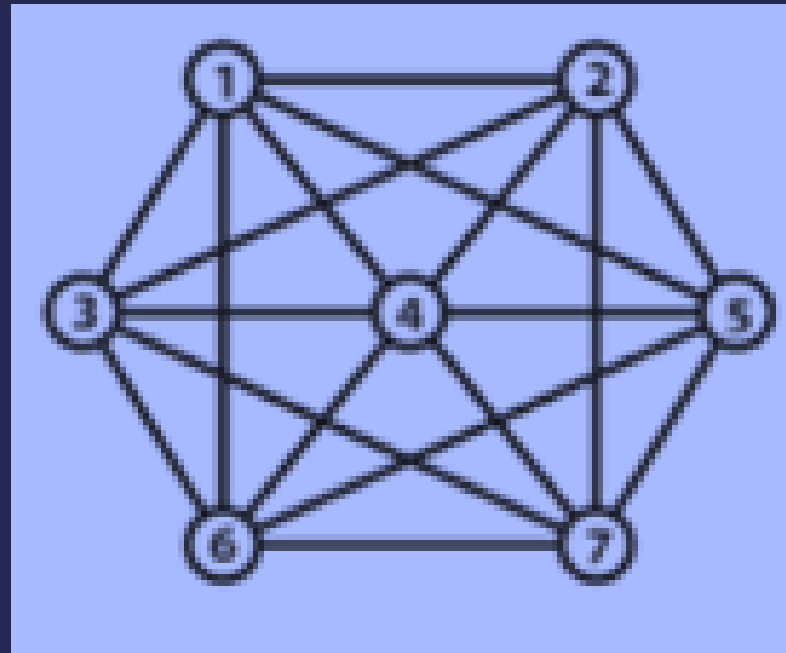




# Concept of Graph

## ❖ Dense Graph:

- A Dense graph is a graph in which the number of edges is close to the maximal number of edges.

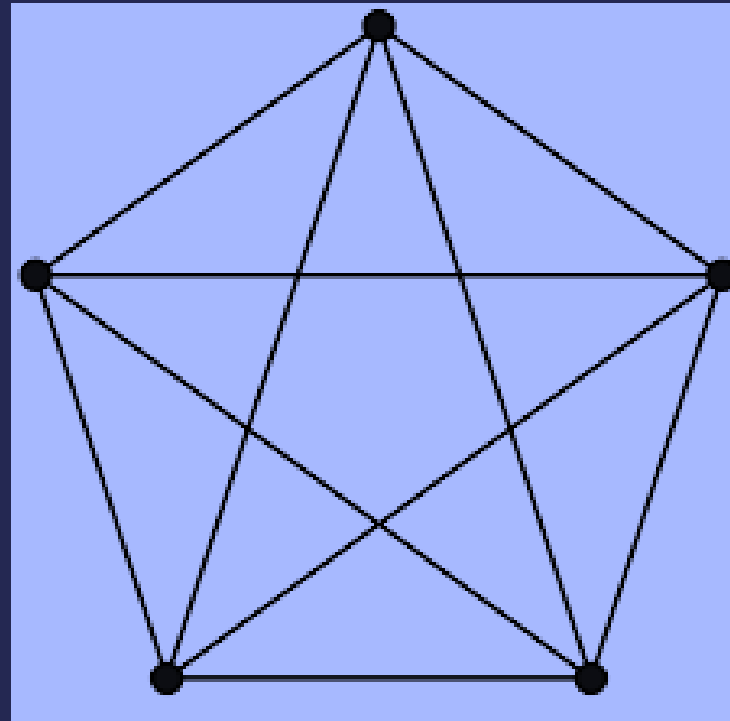




# Concept of Graph

## ❖ Complete Graph:

- An Undirected graph in which every vertex has an edge to every other vertex.

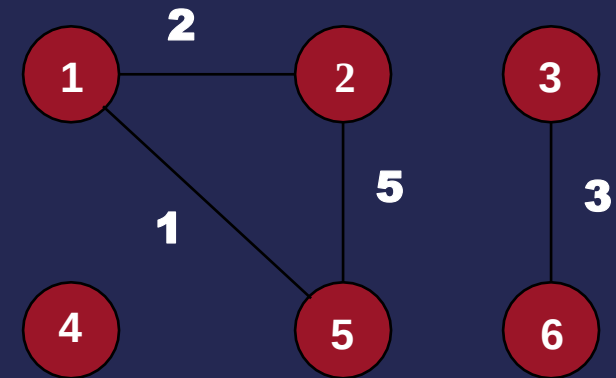
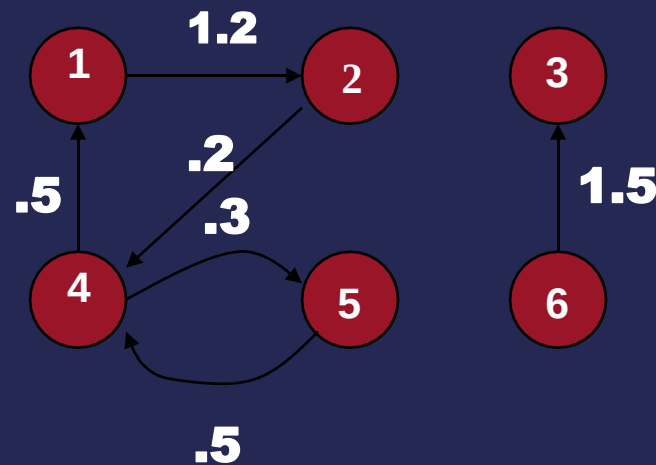




# Concept of Graph

## ❖ *Weighted graph:*

Weighted graph is a graph for which each edge has an associated weight

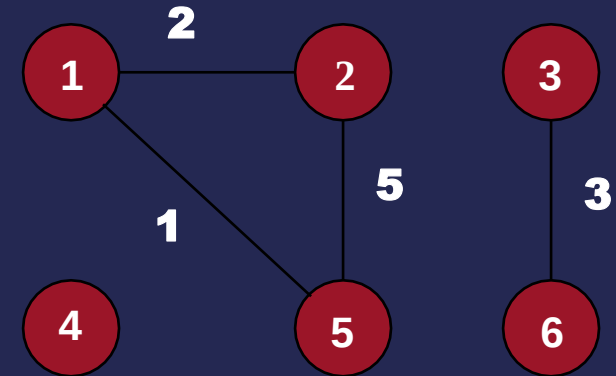
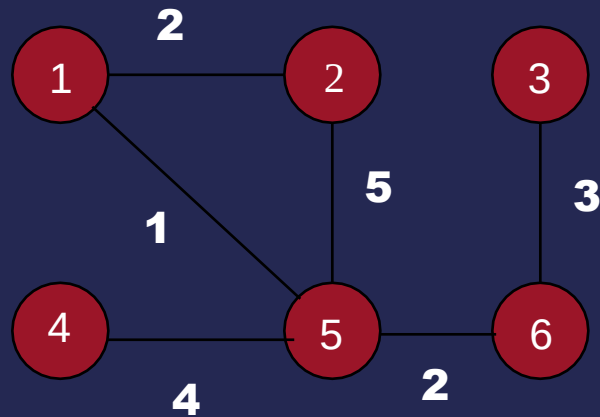




# Concept of Graph

## ❖ *Connected graph:*

An undirected graph is called connected graph if every vertex is reachable from all other vertices.



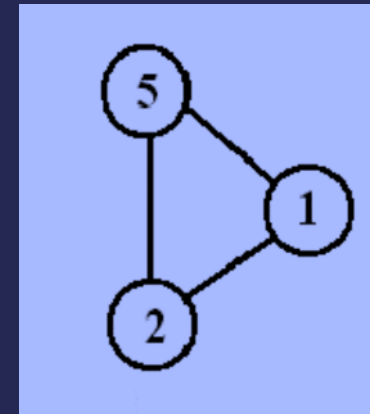
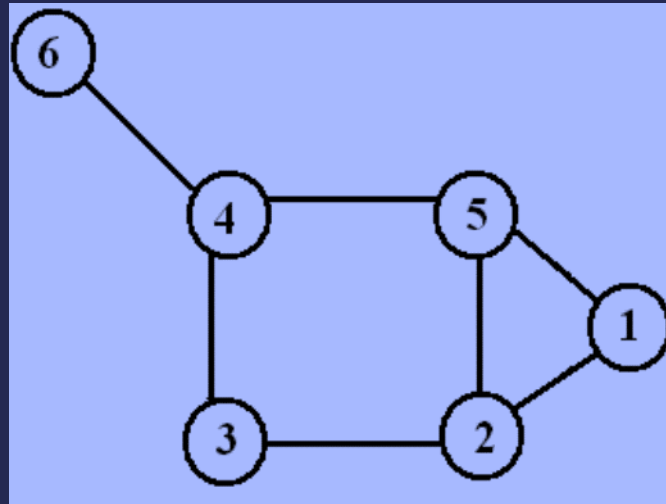




# Concept of Graph

## ❖ Subgraph :

- Vertex and edge sets are subsets of those of  $G$
- A super graph of a graph  $G$  is a graph that contains  $G$  as a subgraph.

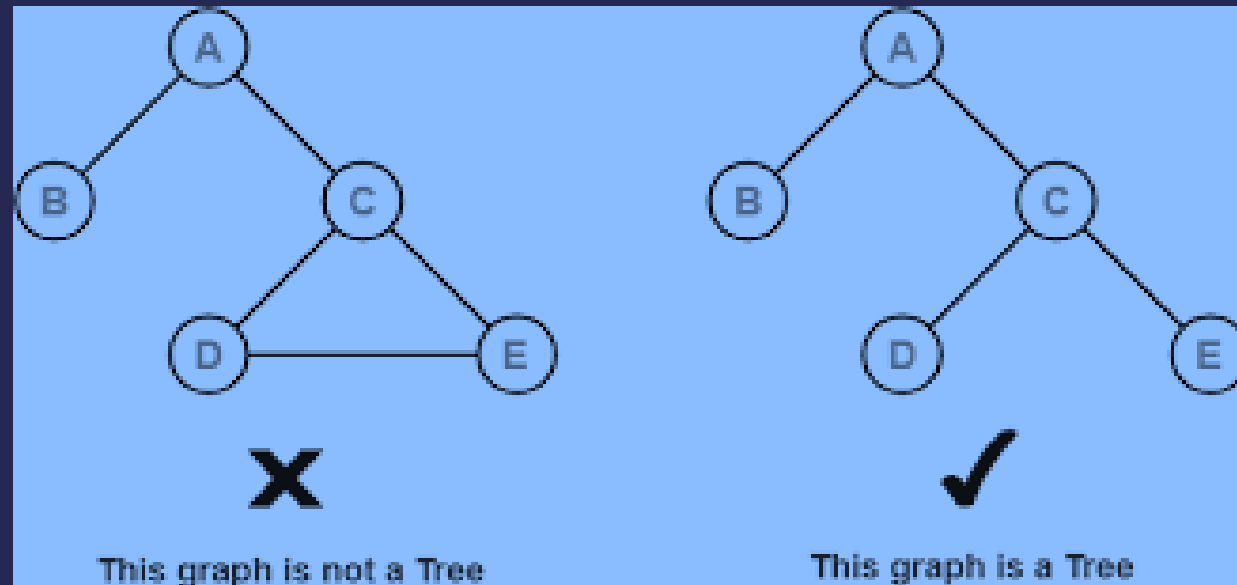




# Concept of Graph

## ❖ Trees :

An undirected graph is a tree if it is connected and does not contain a cycle (Connected Acyclic Graph)





# *Representation of Graph*

❖ *Two ways to Representation of Graph :*

1. Adjacency Matrix
2. Adjacency List



# Representation of Graph

## ❖ Two ways to Representation of Graph :

### 1. Adjacency Matrix.

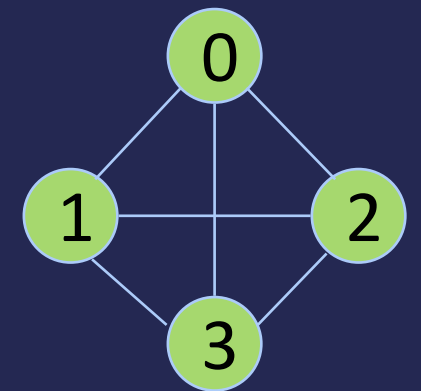
Let  $G=(V,E)$  be a graph with  $n$  vertices.

The **adjacency matrix** of  $G$  is a two-dimensional  $n$  by  $n$  array.

If the edge  $(v_i, v_j)$  is in  $E(G)$ ,  $\text{adj\_mat}[i][j]=1$

If there is no such edge in  $E(G)$ ,  $\text{adj\_mat}[i][j]=0$

|   |   |   |   |
|---|---|---|---|
| 0 | 1 | 1 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 0 |

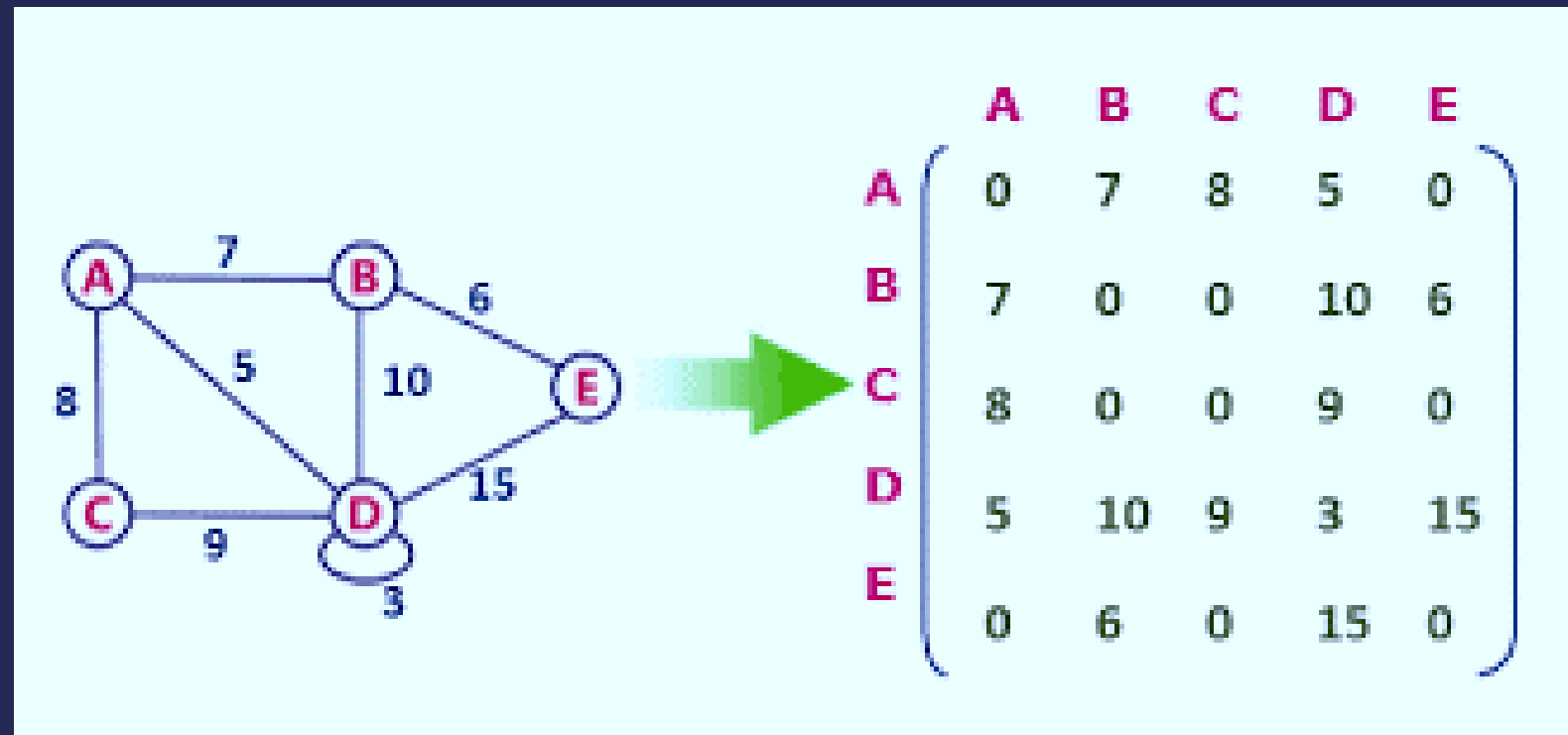




# Representation of Graph

❖ *Two ways to Representation of Graph :*

1. Adjacency Matrix.



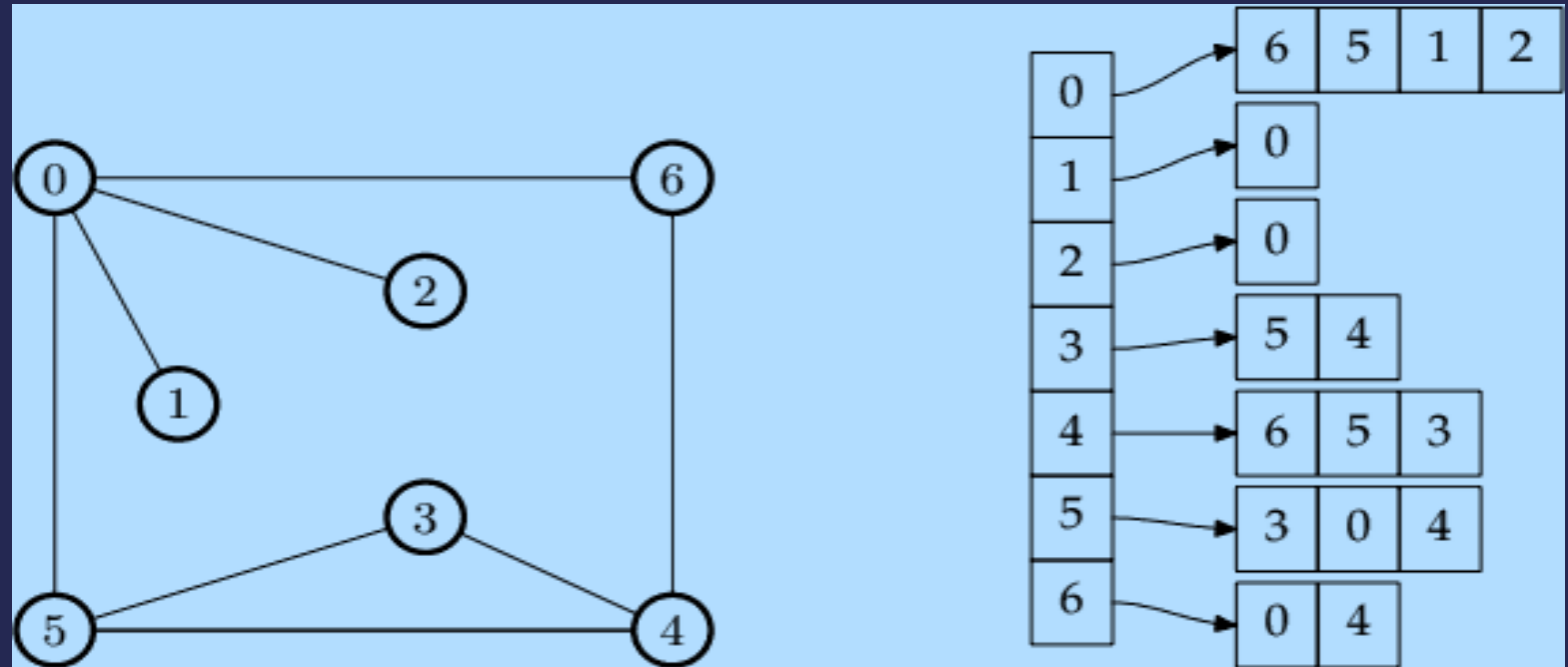


# Representation of Graph

## ❖ Two ways to Representation of Graph :

### 2. Adjacency List.

- Adjacency List representation is one of the alternatives to adjacency matrix. For every vertex adjacency list stores a list of vertices, which are adjacent to current one.





# *GRAPH TRAVERSING*

## *BREADTH FIRST SEARCH*

*Mr. Prayag Patel*

*Assistant Professor  
IT-ICT Department, LJIET  
[prayag.patel@ljinstitutes.edu.in](mailto:prayag.patel@ljinstitutes.edu.in)*



# *Graph Traversing*

## Two Algorithms

**Breadth first Search**

**Depth first Search**





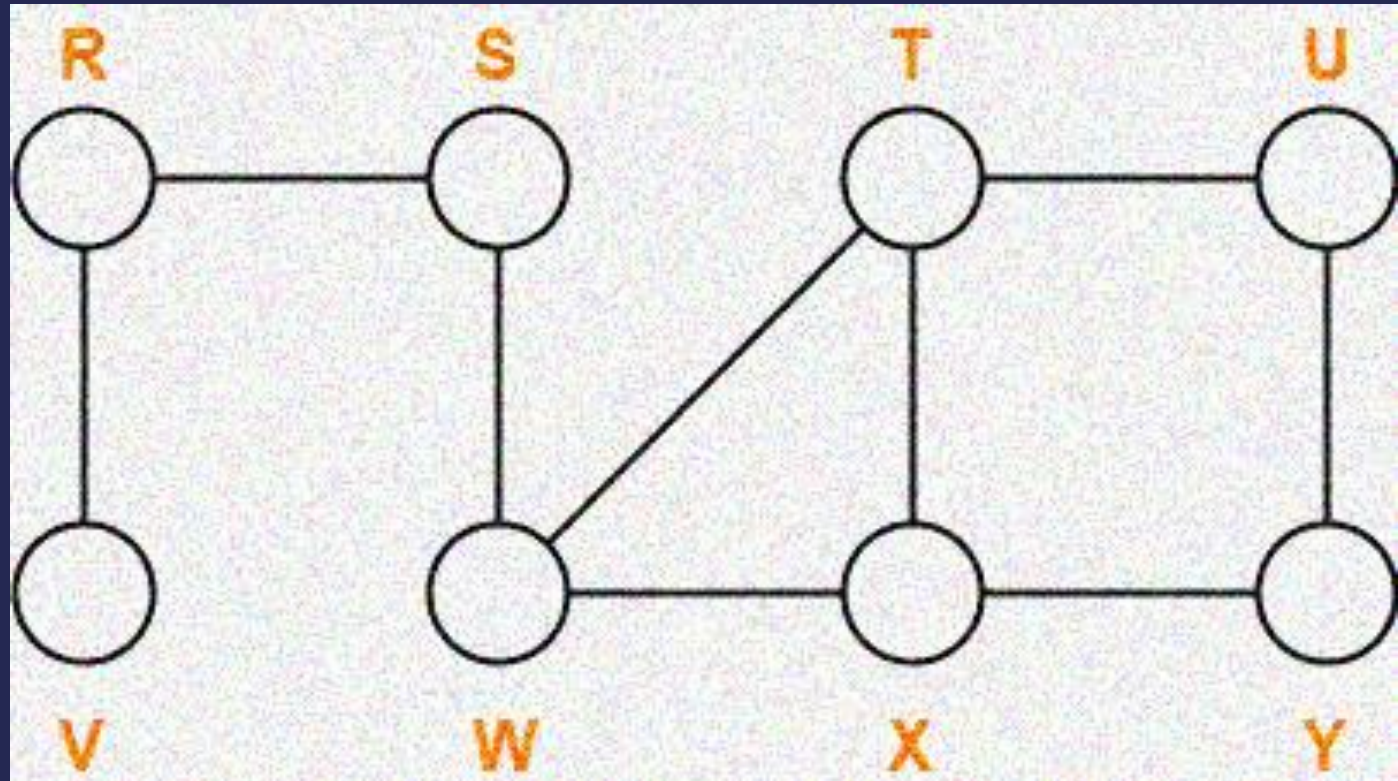
# *Breadth first Search*

- Breadth First Search or BFS is a graph traversal algorithm.
- It is used for traversing or searching a graph in a systematic fashion.
- BFS uses a strategy that searches in the graph in breadth first manner whenever possible.
- Queue data structure is used in the implementation of breadth first search.



# Breadth first Search

*Example: Traverse the following graph using Breadth First Search Technique.*





# Breadth first Search

## Step 1:

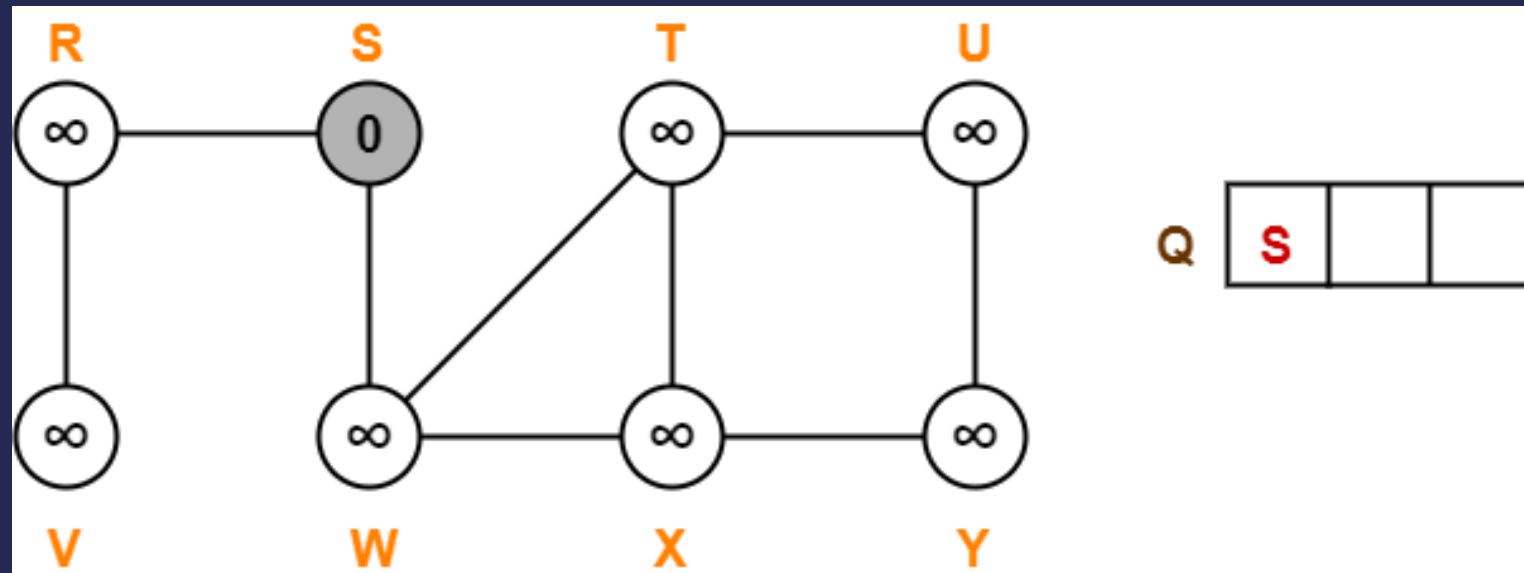
For all the vertices  $u$  except source vertex  $S$  of the graph, we initialize:

- $\text{color}[u] = \text{WHITE}$
- $\pi[u] = \text{NIL}$
- $d[u] = \infty$

We **enqueue** the source vertex  $S$  in the queue  $Q$ .

For source vertex  $S$ , we initialize the variables as-

- $\text{color}[S] = \text{GREY}$
- $\pi[S] = \text{NIL}$
- $d[S] = 0$





# Breadth first Search

## Step 2:

**Dequeue** vertex S from the queue Q

For all adjacent white vertices 'v' (vertices R and W) of vertex S, we do-

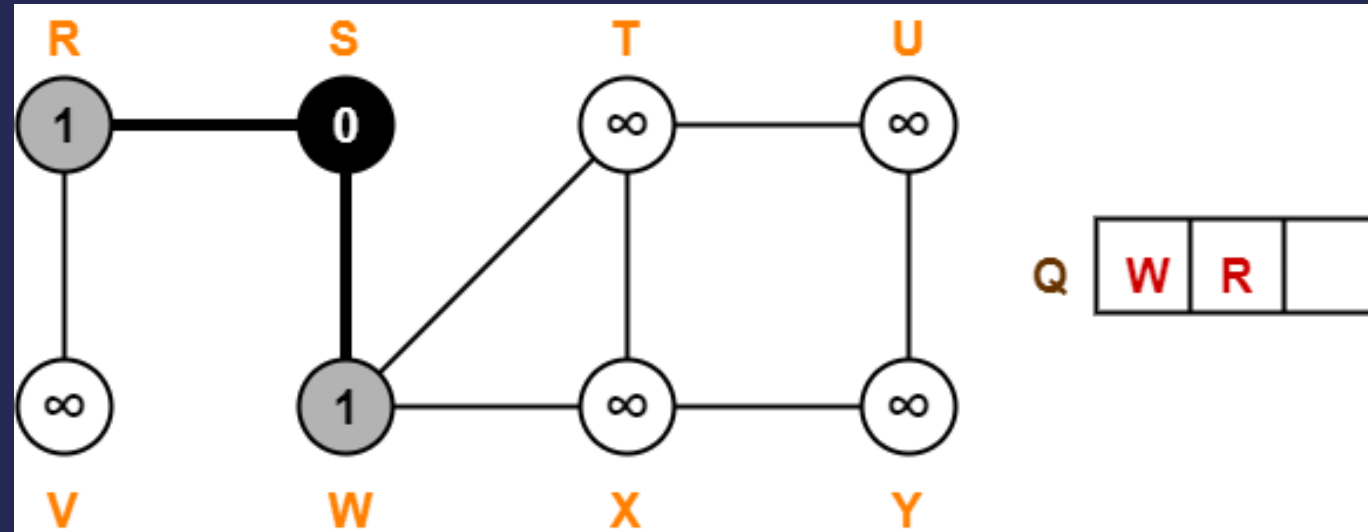
1.  $\text{color}[v] = \text{GREY}$

2.  $d[v] = d[S] + 1 = 0 + 1 = 1$

3.  $\pi[v] = S$

4. **Enqueue** all adjacent white vertices of S in queue Q

$\text{color}[S] = \text{BLACK}$





# Breadth first Search

## Step 3:

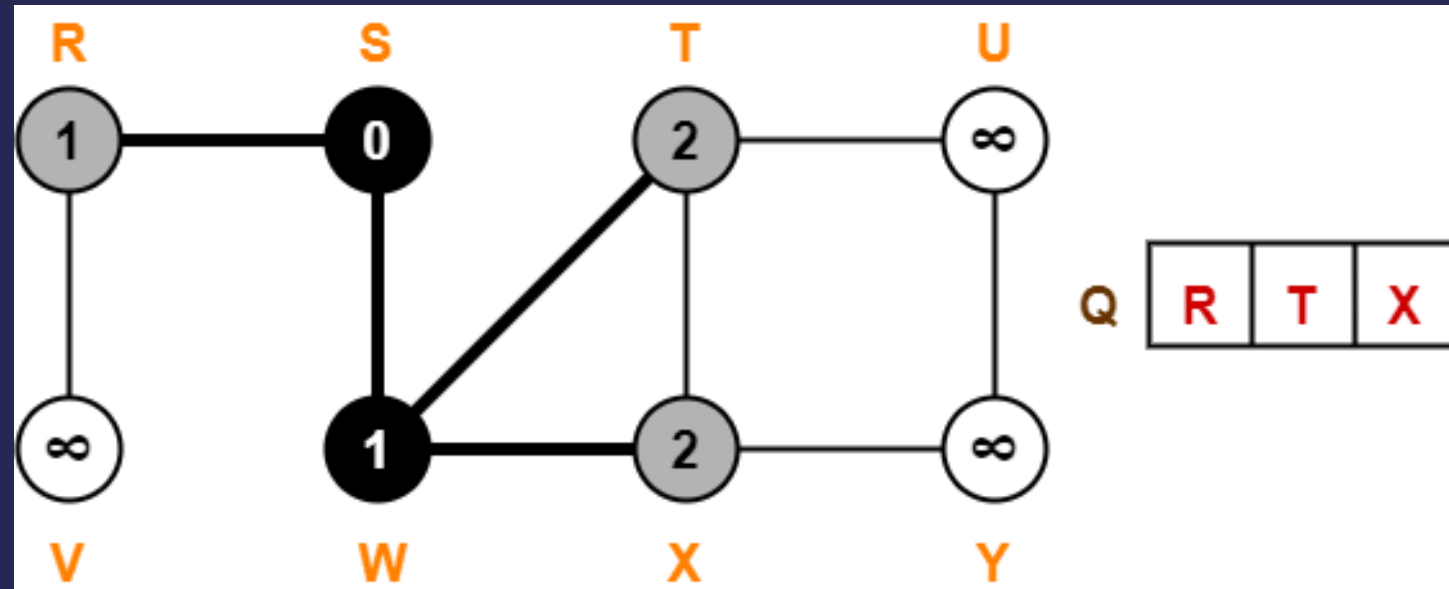
**Dequeue** vertex W from the queue Q

For all adjacent white vertices 'v' (vertices T and X) of vertex W, we do-

1.  $\text{color}[v] = \text{GREY}$
2.  $d[v] = d[W] + 1 = 1 + 1 = 2$
3.  $\pi[v] = W$

4. **Enqueue** all adjacent white vertices of W in queue Q

$\text{color}[W] = \text{BLACK}$





# Breadth first Search

## Step 4:

**Dequeue** vertex R from the queue Q

For all adjacent white vertices 'v' (vertex V) of vertex R, we do-

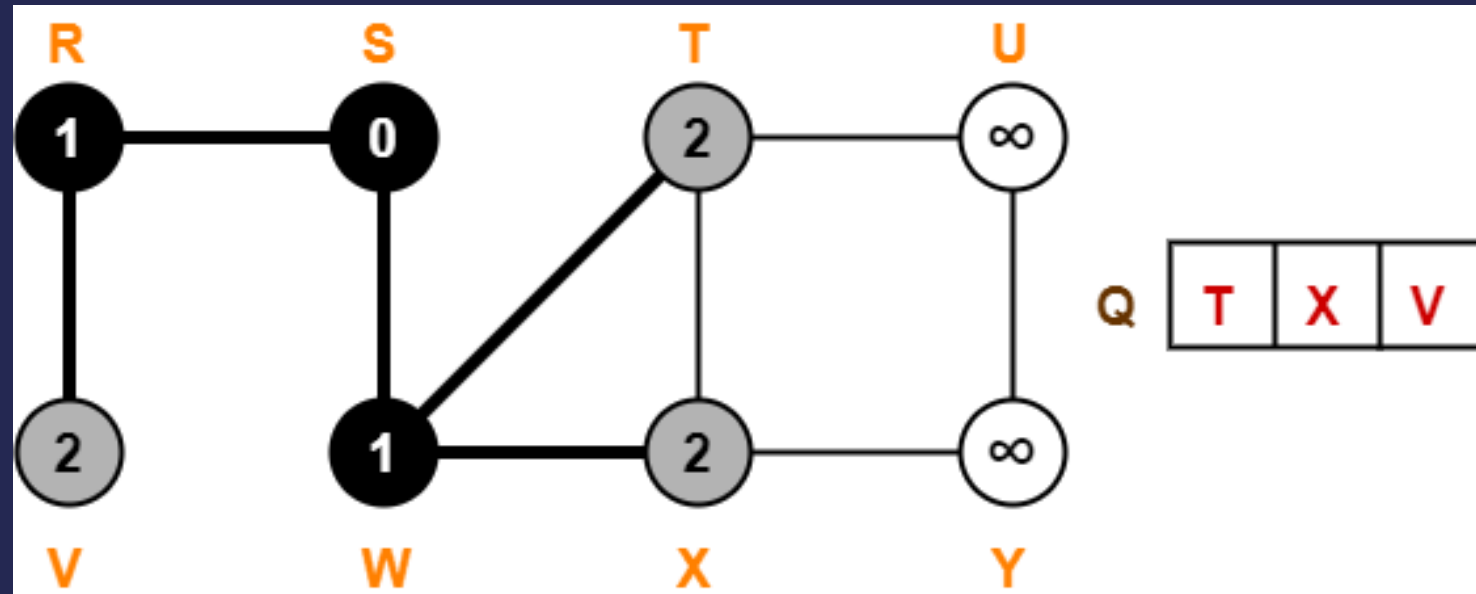
1.  $\text{color}[v] = \text{GREY}$

2.  $d[v] = d[R] + 1 = 1 + 1 = 2$

3.  $\pi[v] = R$

4. **Enqueue** all adjacent white vertices of R in queue Q

$\text{color}[R] = \text{BLACK}$





# Breadth first Search

## Step 5:

**Dequeue** vertex T from the queue Q

For all adjacent white vertices 'v' (vertex U) of vertex T, we do-

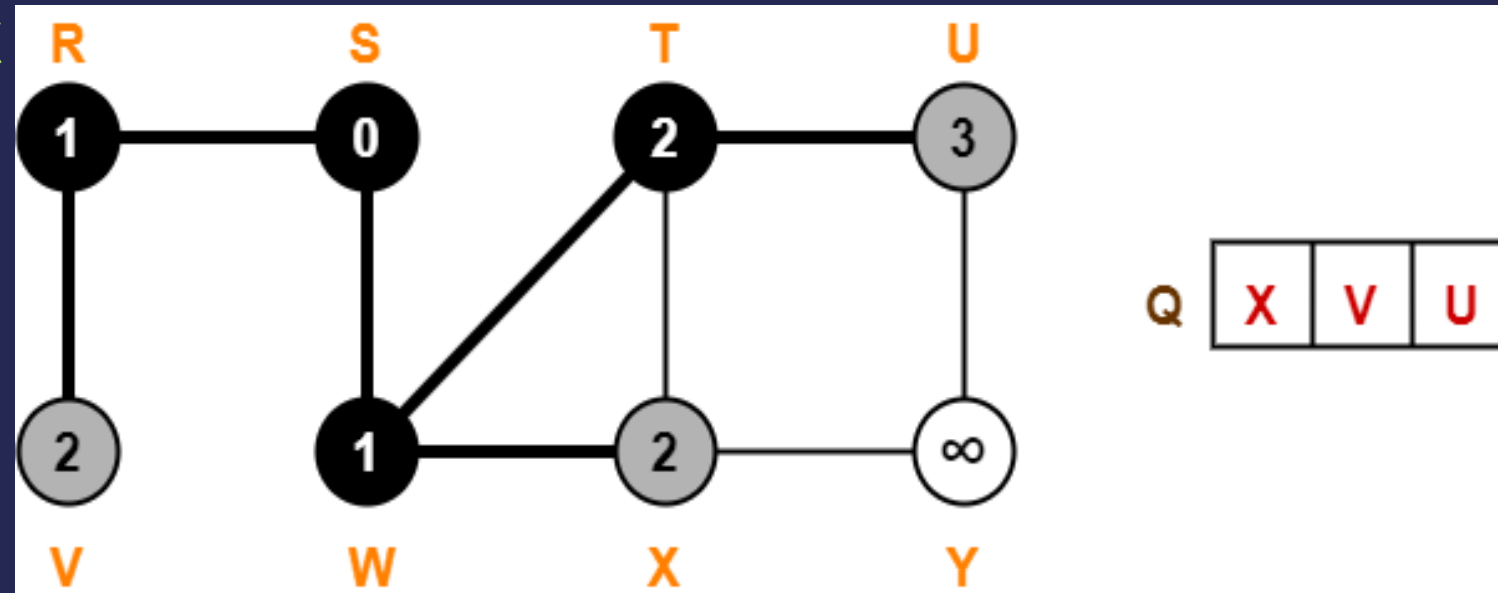
1.  $\text{color}[v] = \text{GREY}$

2.  $d[v] = d[T] + 1 = 2 + 1 = 3$

3.  $\pi[v] = T$

4. **Enqueue** all adjacent white vertices of T in queue Q

$\text{color}[T] = \text{BLACK}$







# Breadth first Search

## Step 6:

**Dequeue** vertex X from the queue Q

For all adjacent white vertices 'v' (vertex Y) of vertex X, we do-

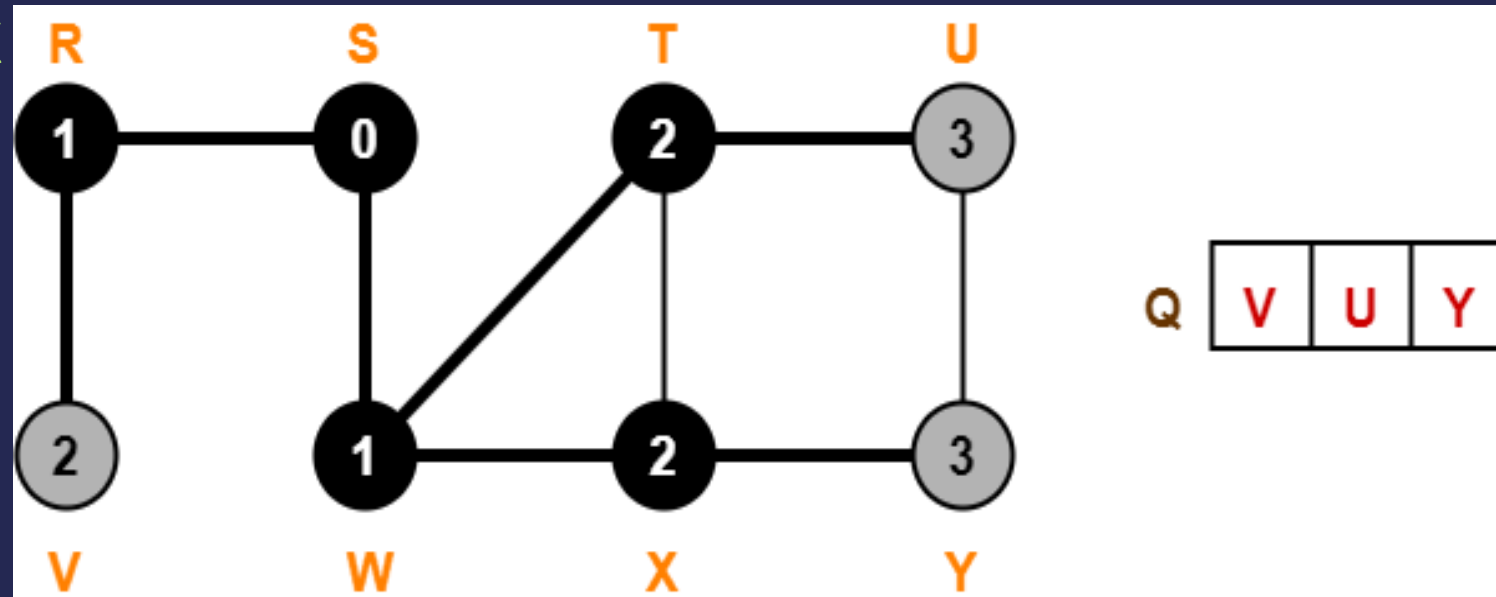
1.  $\text{color}[v] = \text{GREY}$

2.  $d[v] = d[X] + 1 = 2 + 1 = 3$

3.  $\pi[v] = X$

4. **Enqueue** all adjacent white vertices of X in queue Q

$\text{color}[X] = \text{BLACK}$







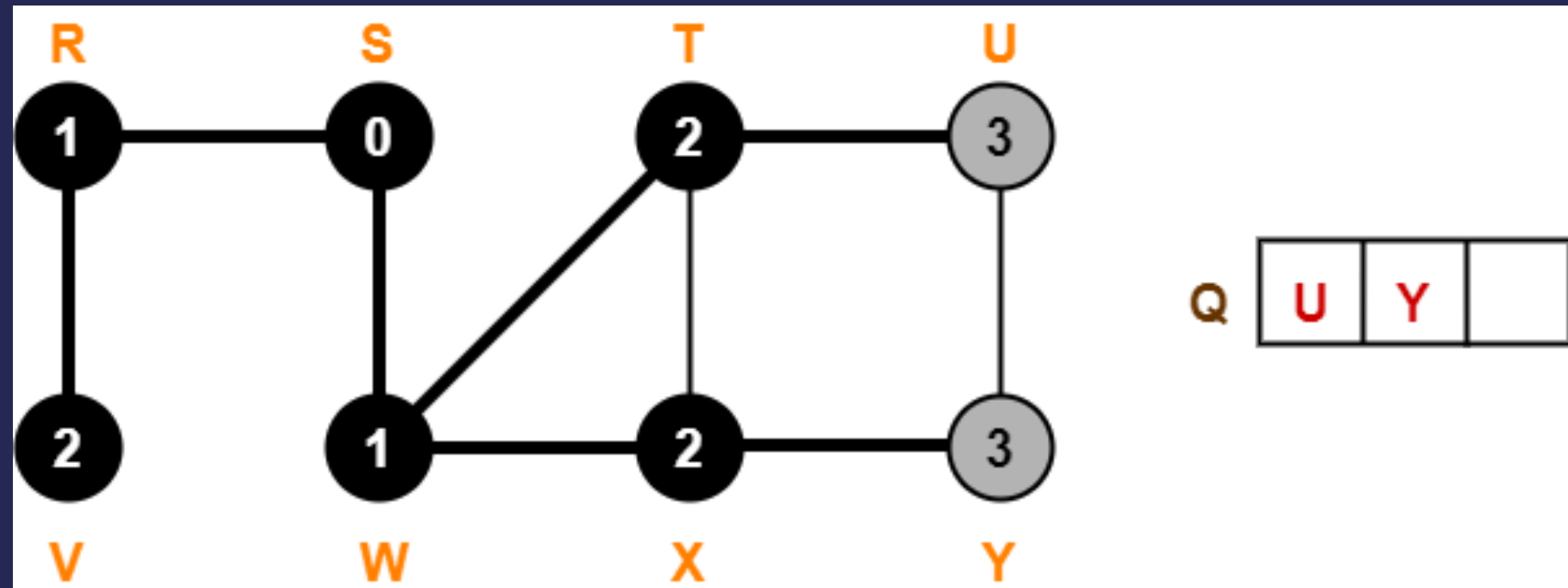
# Breadth first Search

Step 7:

Dequeue vertex **V** from the queue Q

There are no adjacent white vertices to vertex **V**.

color[V] = BLACK





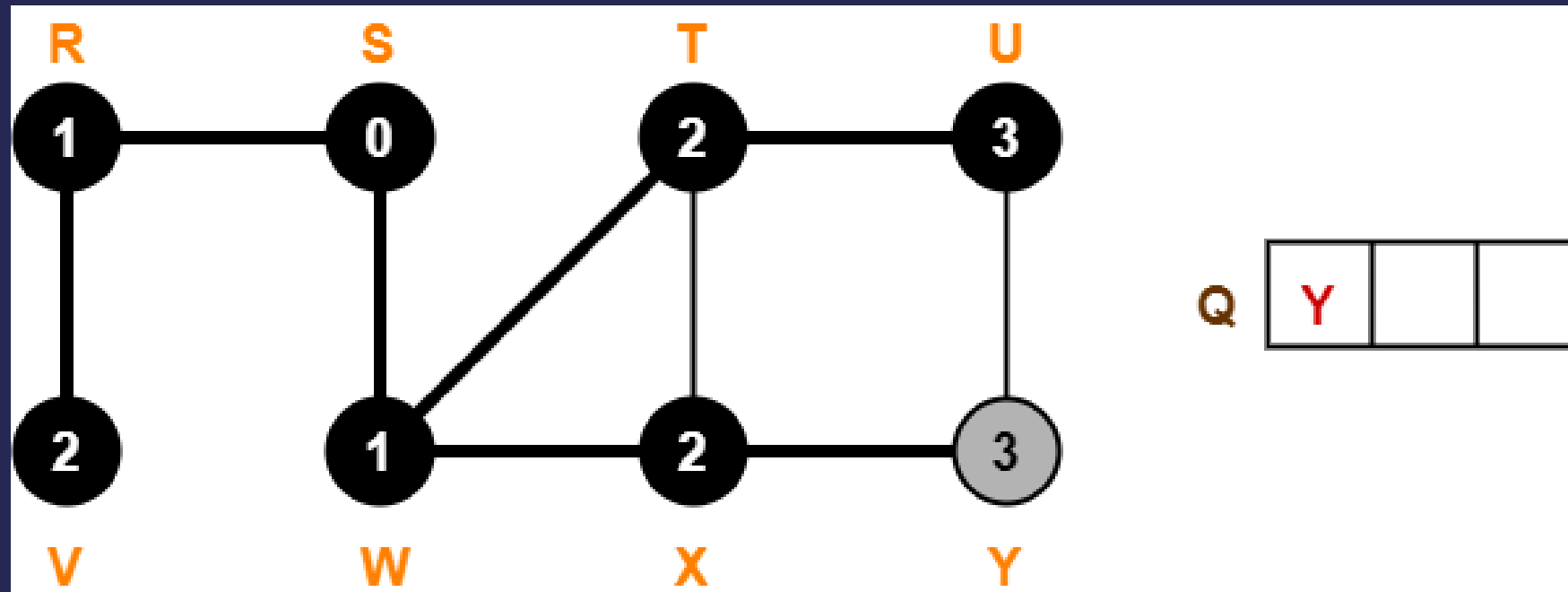
# Breadth first Search

Step 8:

Deque **U** from the queue Q

There are no adjacent white vertices to vertex **U**.

color[U] = BLACK





# Breadth first Search

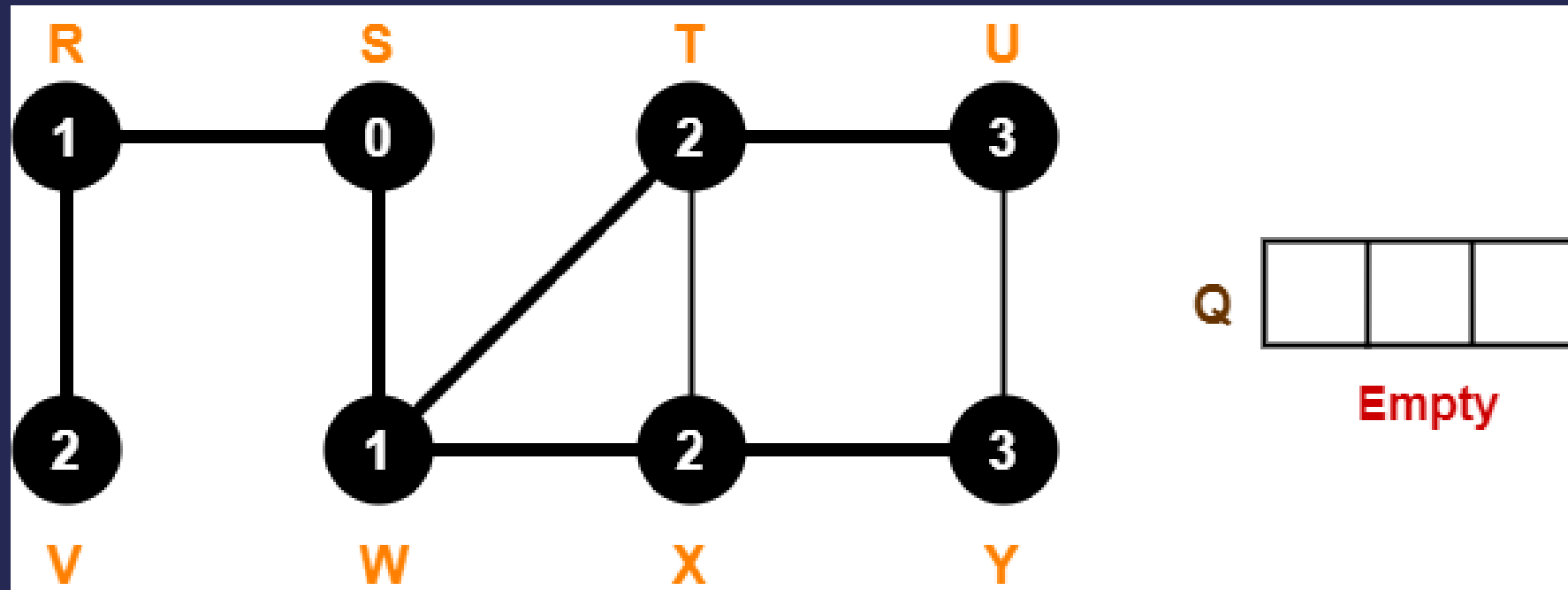
Step 9:

Deque **Y** from the queue Q

There are no adjacent white vertices to vertex **Y**.

$\text{color}[\mathbf{Y}] = \text{BLACK}$

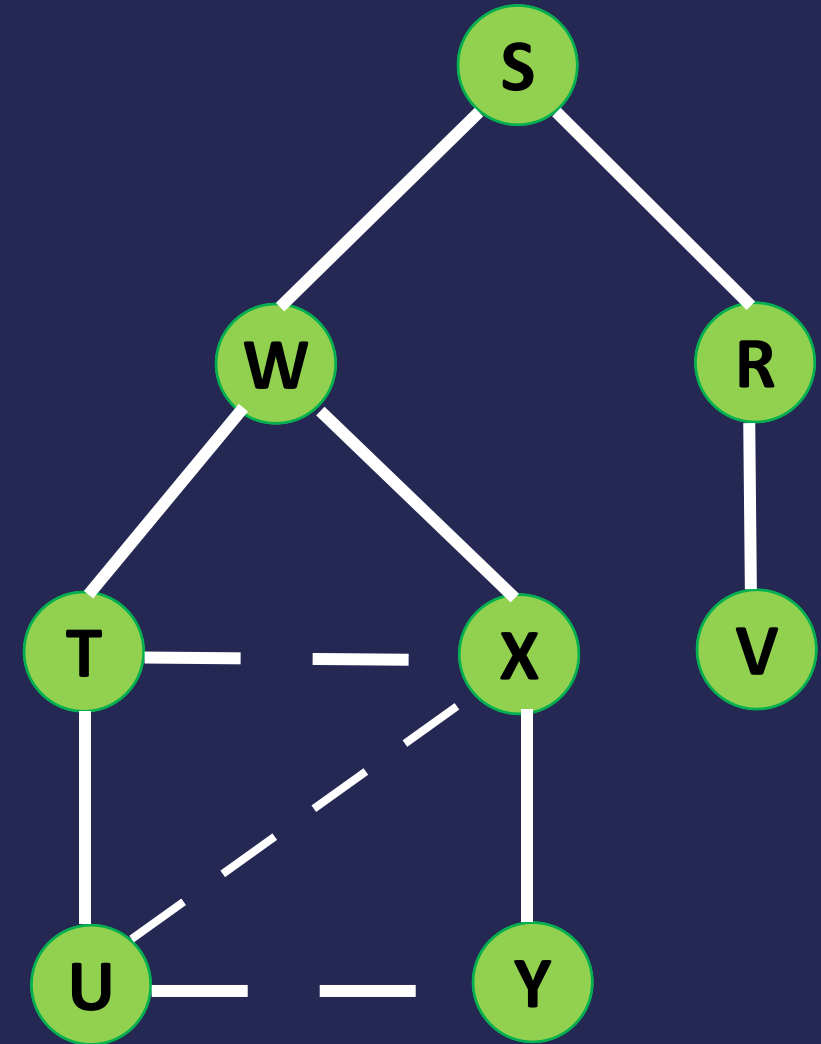
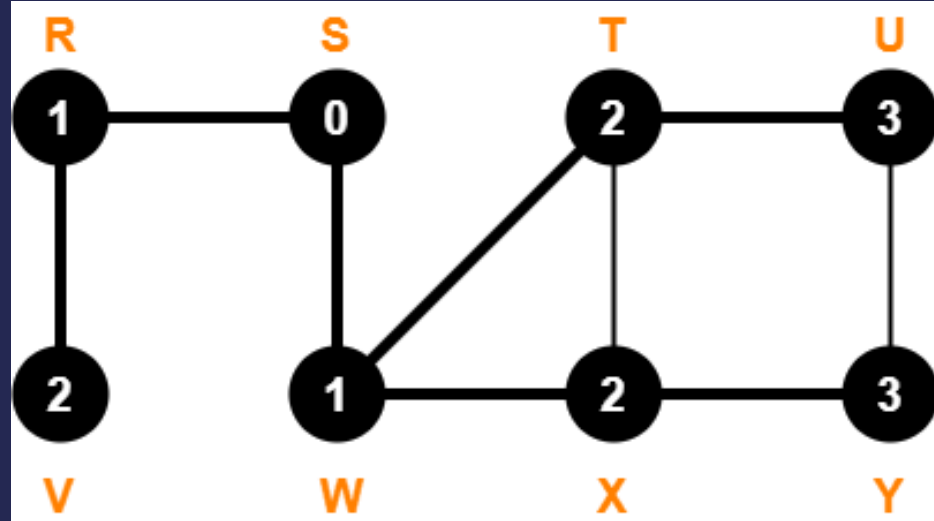
Since, all the vertices have turned black and the queue has got empty, so we stop.





# Breadth first Search

Step 10: **BFS Tree**



———— Tree Edge  
- - - - Cross Edge



# Breadth first Search

**BFS (G,s)**

for each vertex  $u \in G.V - \{s\}$  do

$\text{color}[u] \leftarrow \text{WHITE}$

$d[u] \leftarrow \infty$

$\pi[u] \leftarrow \text{NIL}$

$\text{color}[s] = \text{GREY}$

$d[s] \leftarrow 0$

$\pi[s] \leftarrow \text{NIL}$

$Q \leftarrow \{ \}$

$\text{ENQUEUE}(Q,s)$

While  $Q$  is non-empty do

$u \leftarrow \text{DEQUEUE}(Q)$

    for each  $v$  adjacent to  $u$  do

        if  $\text{color}[v] \leftarrow \text{WHITE}$  then

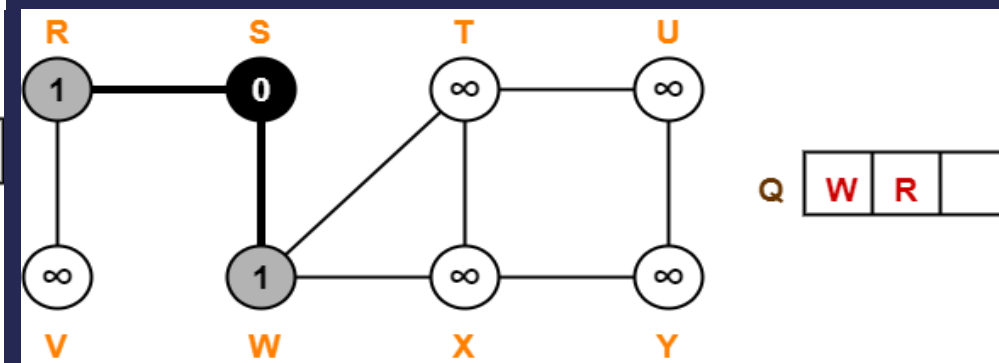
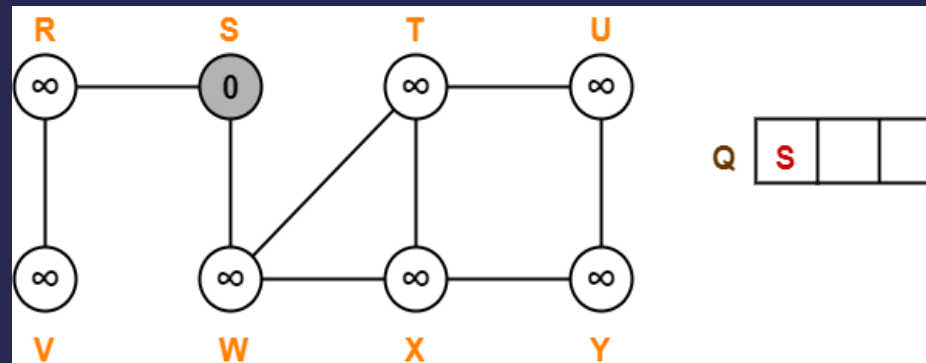
$\text{color}[v] \leftarrow \text{GREY}$

$d[v] \leftarrow d[u] + 1$

$\pi[v] \leftarrow u$

$\text{ENQUEUE}(Q,v)$

$\text{color}[u] \leftarrow \text{BLACK}$





# *GRAPH TRAVERSING*

## *Depth first Search*

*Mr. Prayag Patel*

*Assistant Professor  
IT-ICT Department, LJIET  
[prayag.patel@ljinstitutes.edu.in](mailto:prayag.patel@ljinstitutes.edu.in)*



# *Graph Traversing*

## Two Algorithms

**Breadth first Search**

**Depth first Search**



# *Depth first Search*

- Depth First Search or DFS is a graph traversal algorithm.
- It is used for traversing or searching a graph in a systematic fashion.
- DFS uses a strategy that searches “deeper” in the graph whenever possible.
- Stack data structure is used in the implementation of depth first search.





# Depth first Search

## Types of Edges in DFS-

- After a DFS traversal of any graph G, all its edges can be put in one of the following 4 classes-

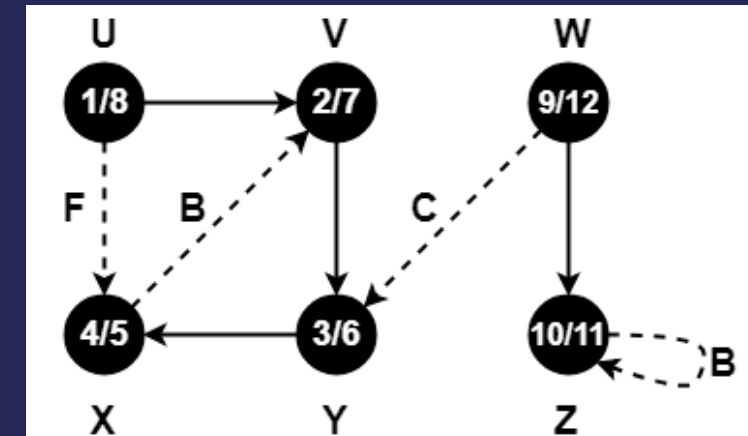
### 1) Tree Edge:

A tree edge is an edge that is included in the DFS tree.

### 2) Back Edge:

An edge from a vertex 'u' to one of its ancestors 'v' and grey at that time is called as a back edge.

A self-loop is considered as a back edge.



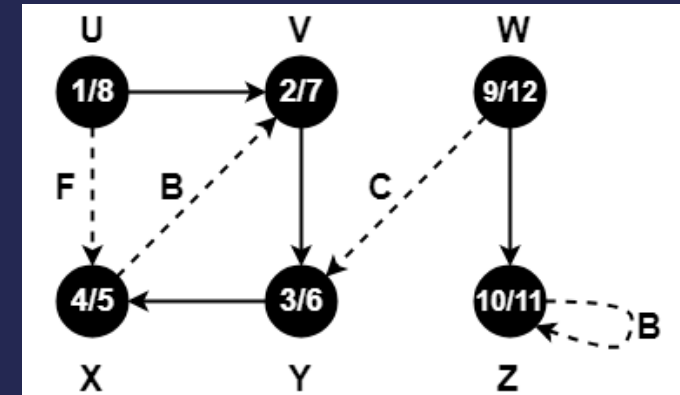


# Depth first Search

## 3) Forward Edge

An edge from a vertex 'u' to one of its descendants 'v' is called as a forward edge.

A forward edge is discovered when DFS tries to extend the visit from a vertex 'u' to a vertex 'v' and finds that  $\text{color}(v) = \text{BLACK}$  and  $d(v) > d(u)$ .



## 4) Cross Edge

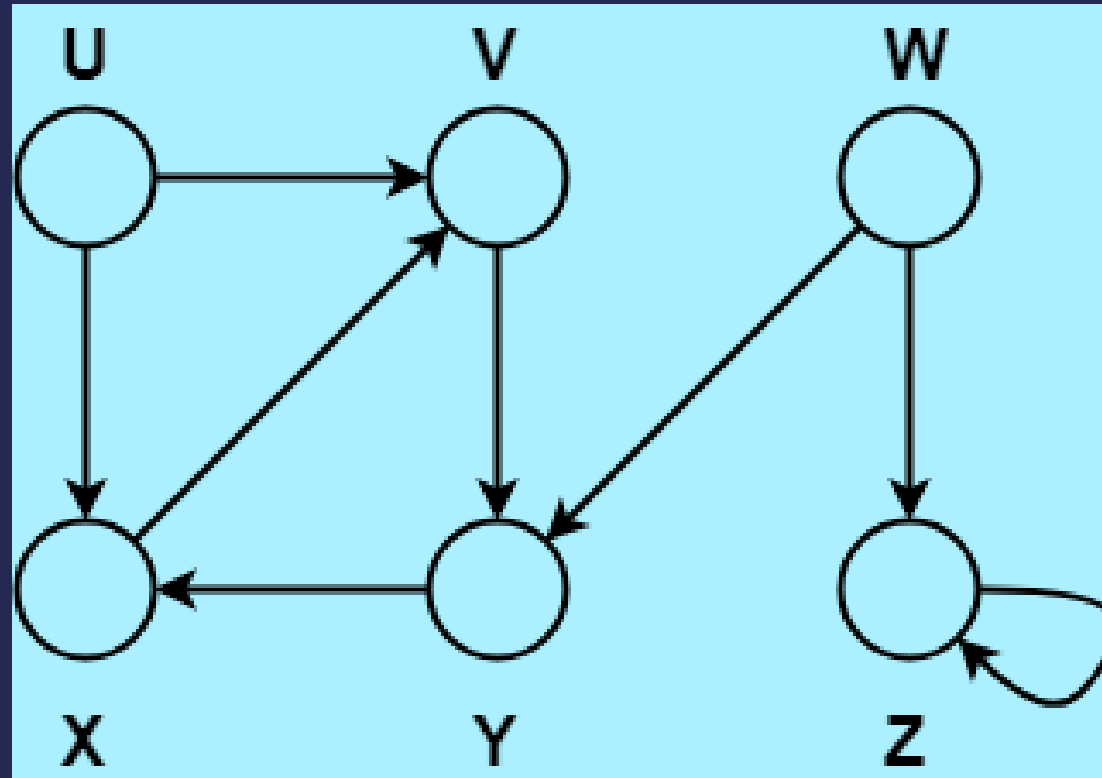
An edge from a vertex 'u' to a vertex 'v' that is neither its ancestor nor its descendant is called as a cross edge.

A cross edge is discovered when DFS tries to extend the visit from a vertex 'u' to a vertex 'v' and finds that  $\text{color}(v) = \text{BLACK}$  and  $d(v) < d(u)$ .



# *Depth first Search*

*Example: Traverse the following graph using Depth First Search Technique.*





# Depth first Search

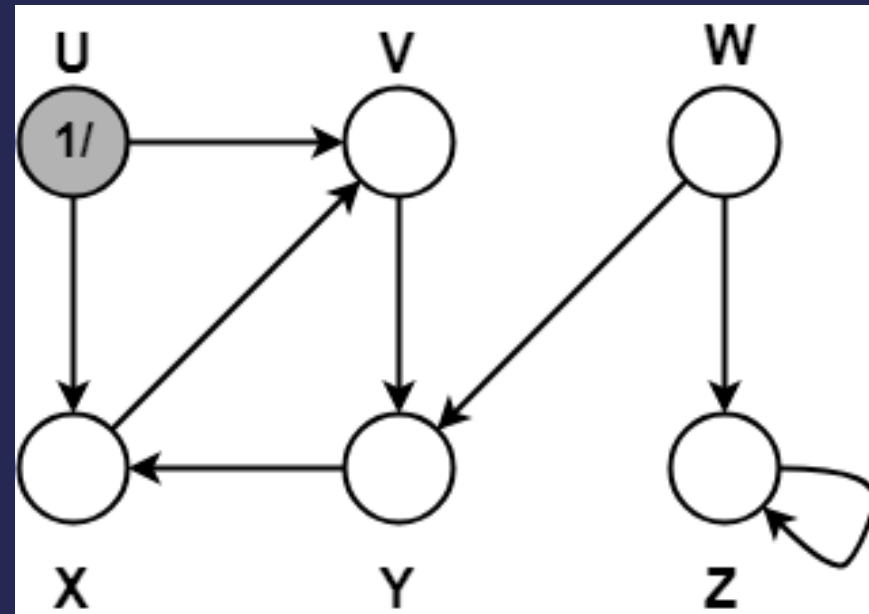
## Step 1:

Initially for all the vertices of the Graph, we set the variables as:

- $\text{color}[u] = \text{WHITE}$
- $\pi[u] = \text{NIL}$
- $\text{time} = 0$

For source vertex U, we initialize the variables as

- $\text{color}[U] = \text{GREY}$
- $\text{time} = 0 + 1 = 1$
- $d[U] = 1$

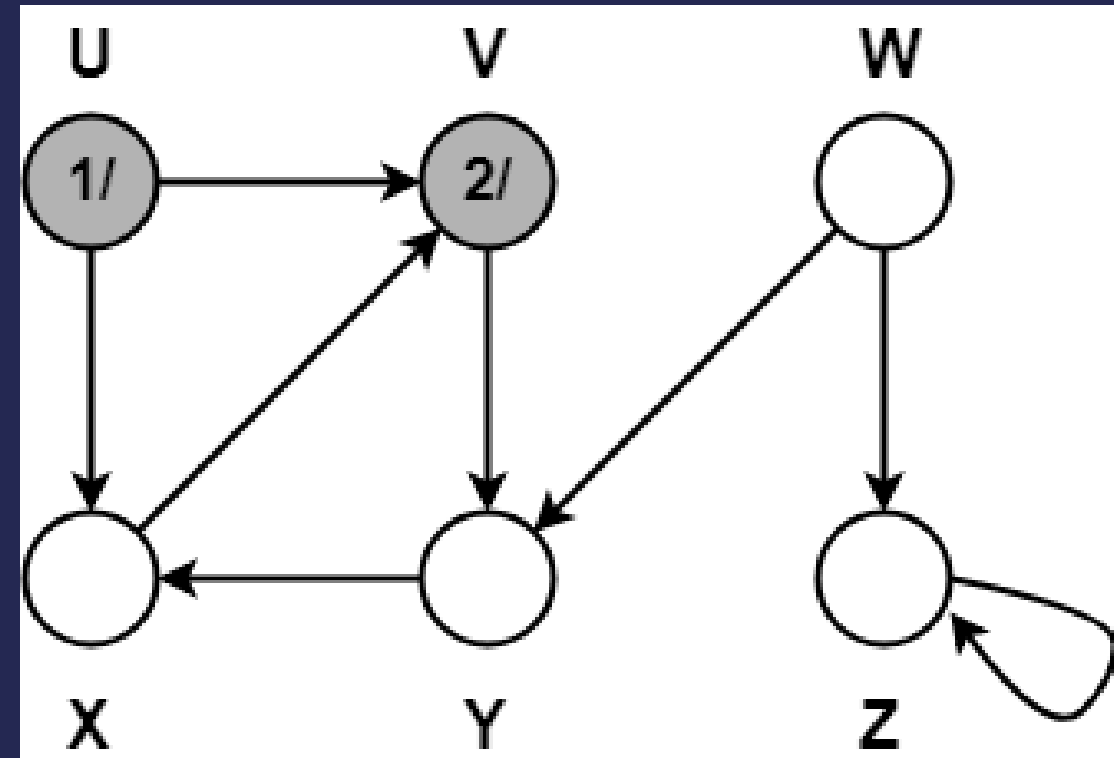




# Depth first Search

## Step 2:

1.  $\pi[V] = U$
2.  $\text{color}[V] = \text{GREY}$
3.  $\text{time} = 1 + 1 = 2$
4.  $d[V] = 2$

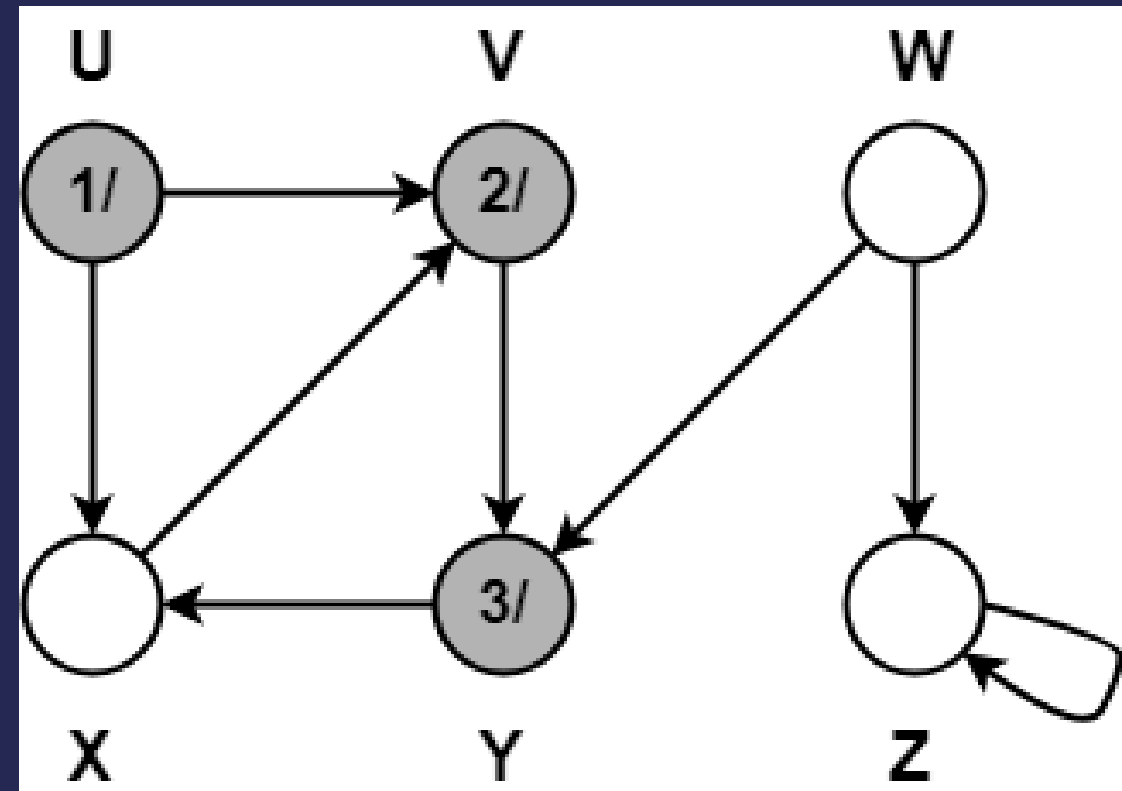




# Depth first Search

## Step 3:

1.  $\pi[Y] = V$
2.  $\text{color}[Y] = \text{GREY}$
3.  $\text{time} = 2 + 1 = 3$
4.  $d[Y] = 3$

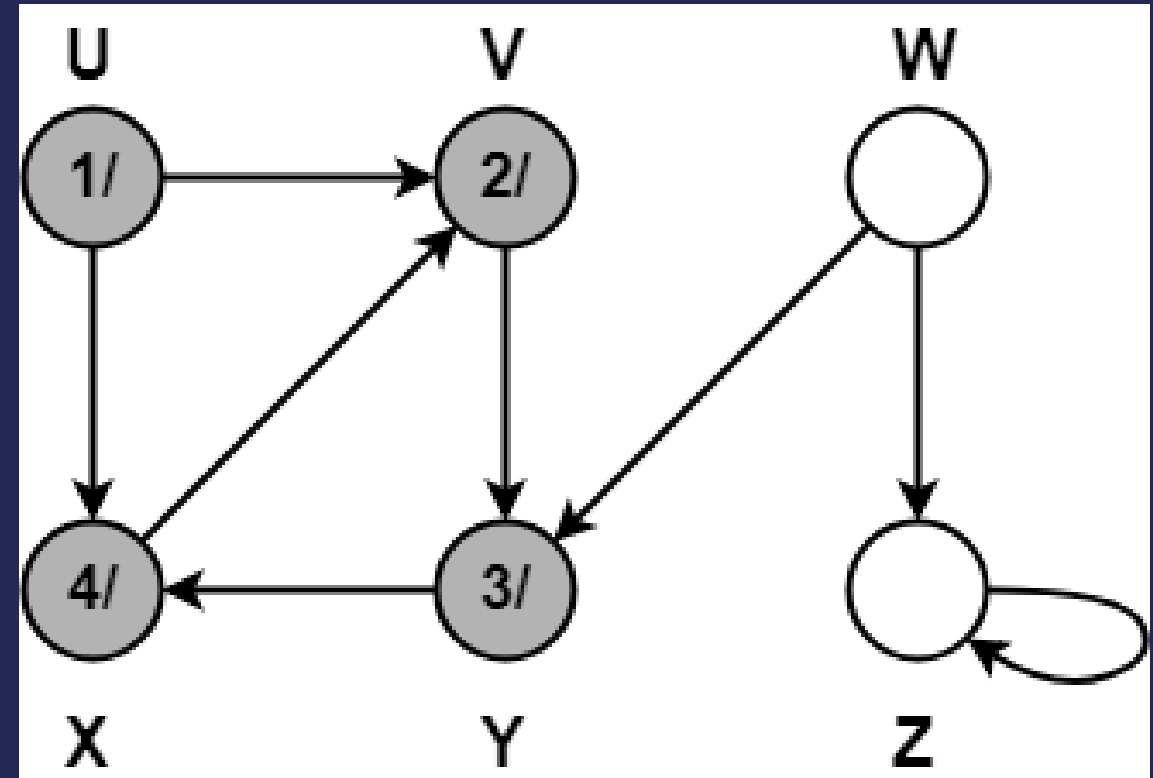




# Depth first Search

## Step 4:

1.  $\pi[X] = Y$
2.  $\text{color}[X] = \text{GREY}$
3.  $\text{time} = 3 + 1 = 4$
4.  $d[X] = 4$

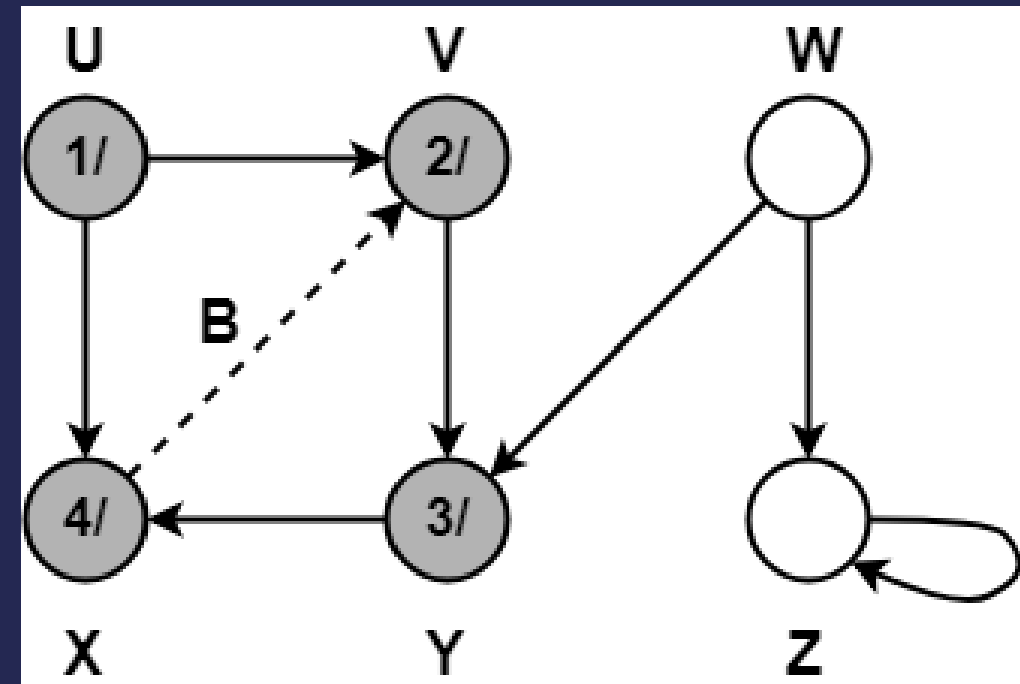




# Depth first Search

## Step 5:

- When DFS tries to extend the visit from vertex X to vertex V, it finds-
  - Vertex V is an ancestor of vertex X since it has already been discovered
  - Vertex V is GREY in color.
- Thus, edge XV is a back edge.



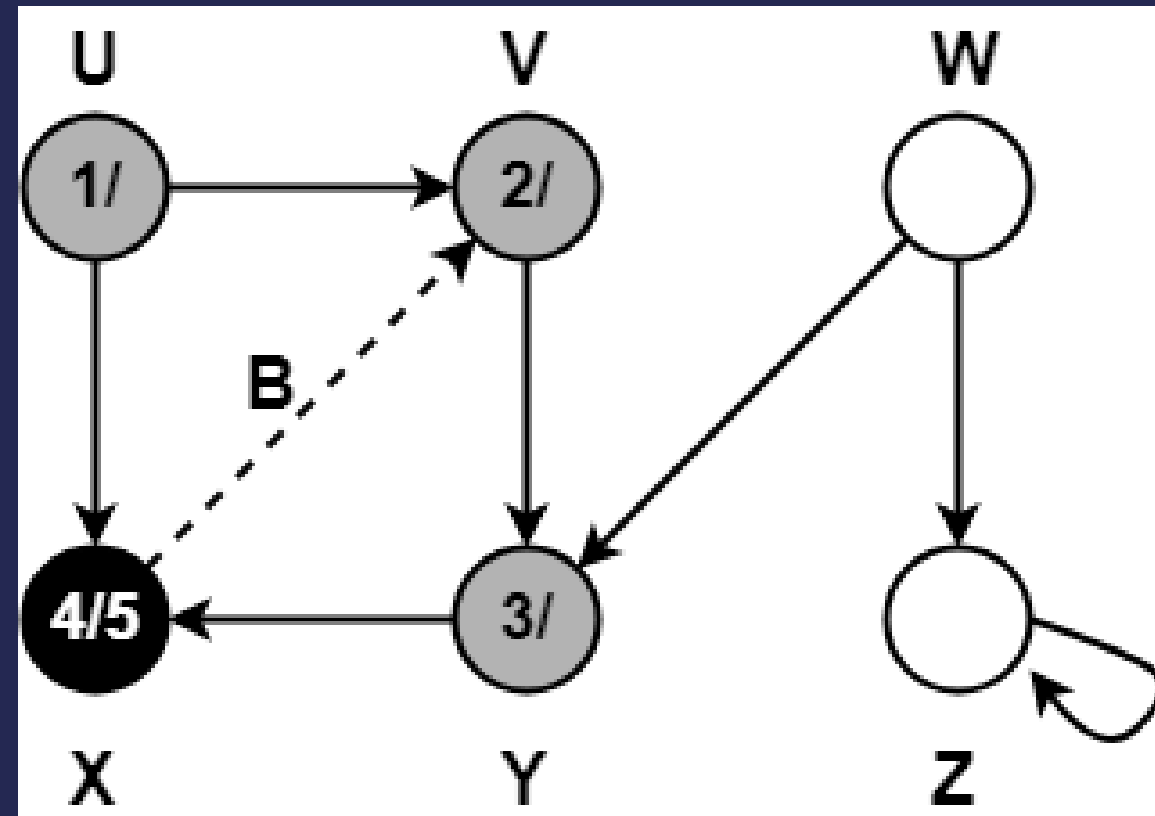




# Depth first Search

## Step 6:

1.  $\text{color}[X] = \text{BLACK}$
2.  $\text{time} = 4 + 1 = 5$
3.  $f[X] = 5$

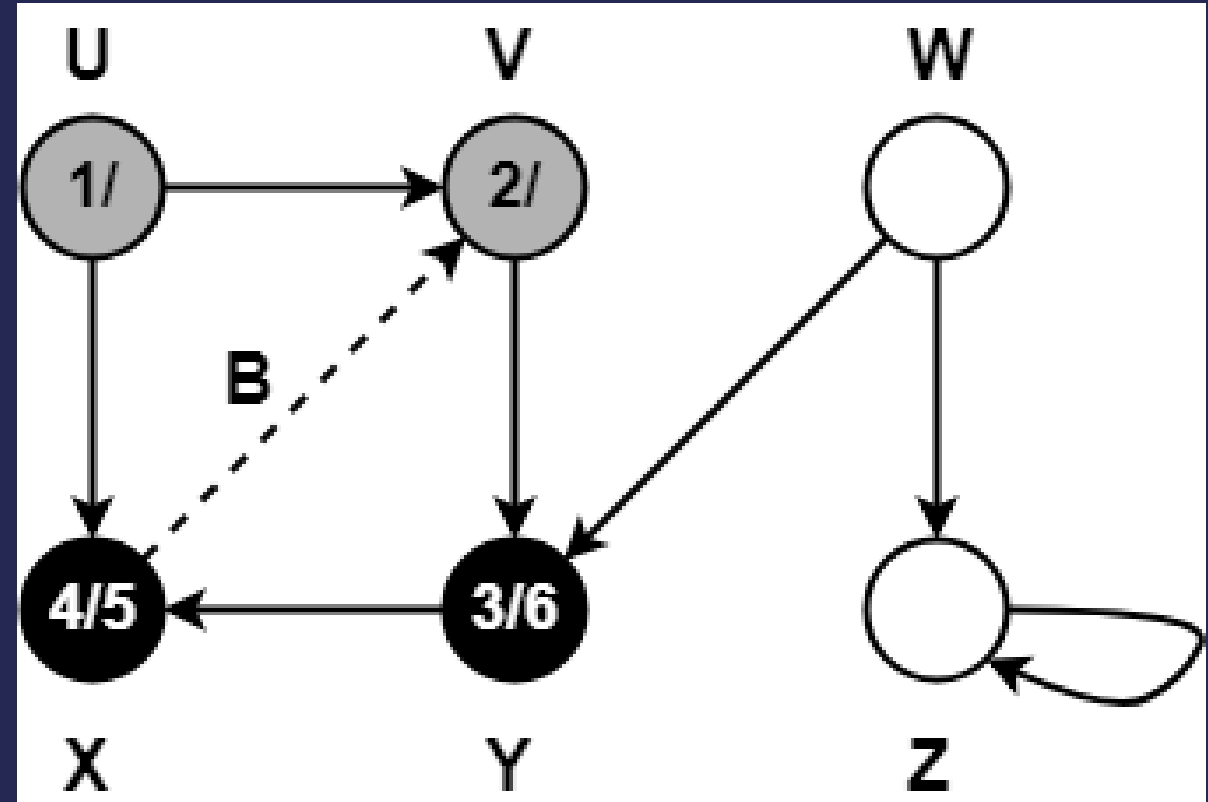




# *Depth first Search*

## Step 7:

1.  $\text{color}[Y] = \text{BLACK}$
2.  $\text{time} = 5 + 1 = 6$
3.  $f[Y] = 6$

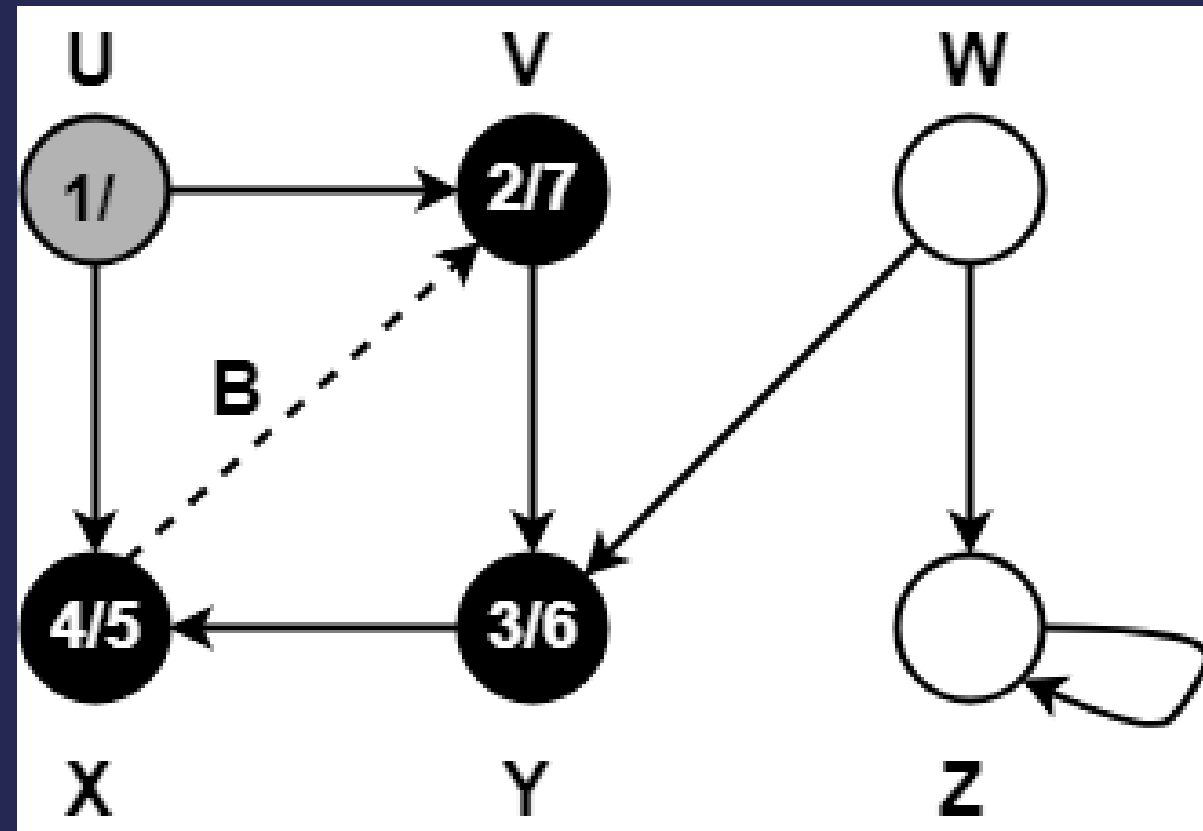




# Depth first Search

## Step 8:

1.  $\text{color}[V] = \text{BLACK}$
2.  $\text{time} = 6 + 1 = 7$
3.  $f[V] = 7$

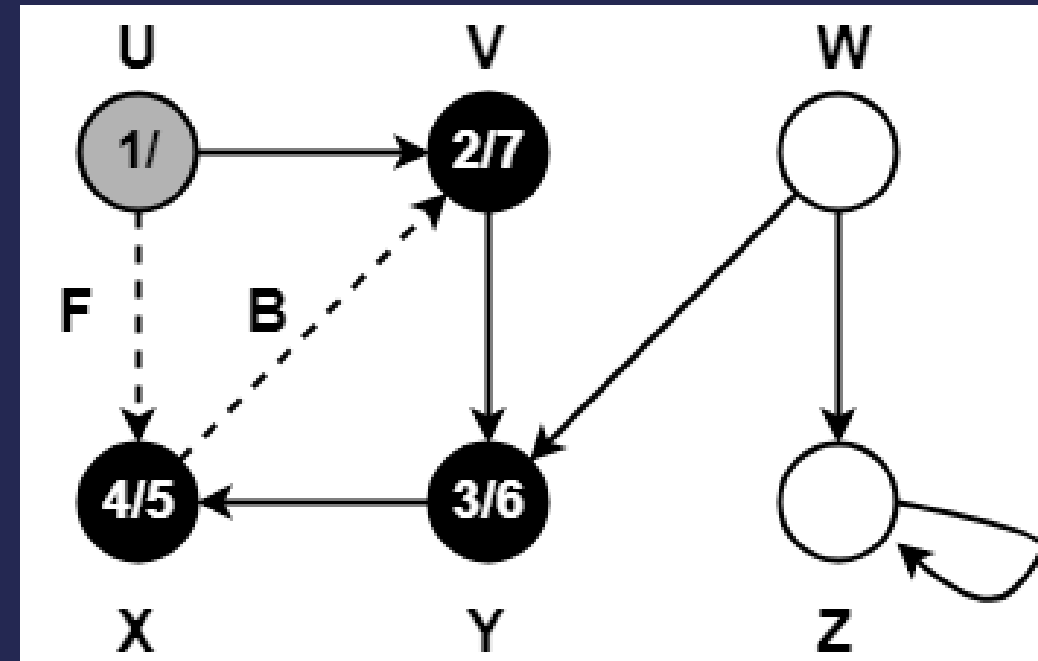




# Depth first Search

## Step 9:

- When DFS tries to extend the visit from vertex U to vertex X, it finds-
  - $\text{Color}(X) = \text{BLACK}$
  - $d(X) > d(U)$
- Thus, edge UX is a forward edge.

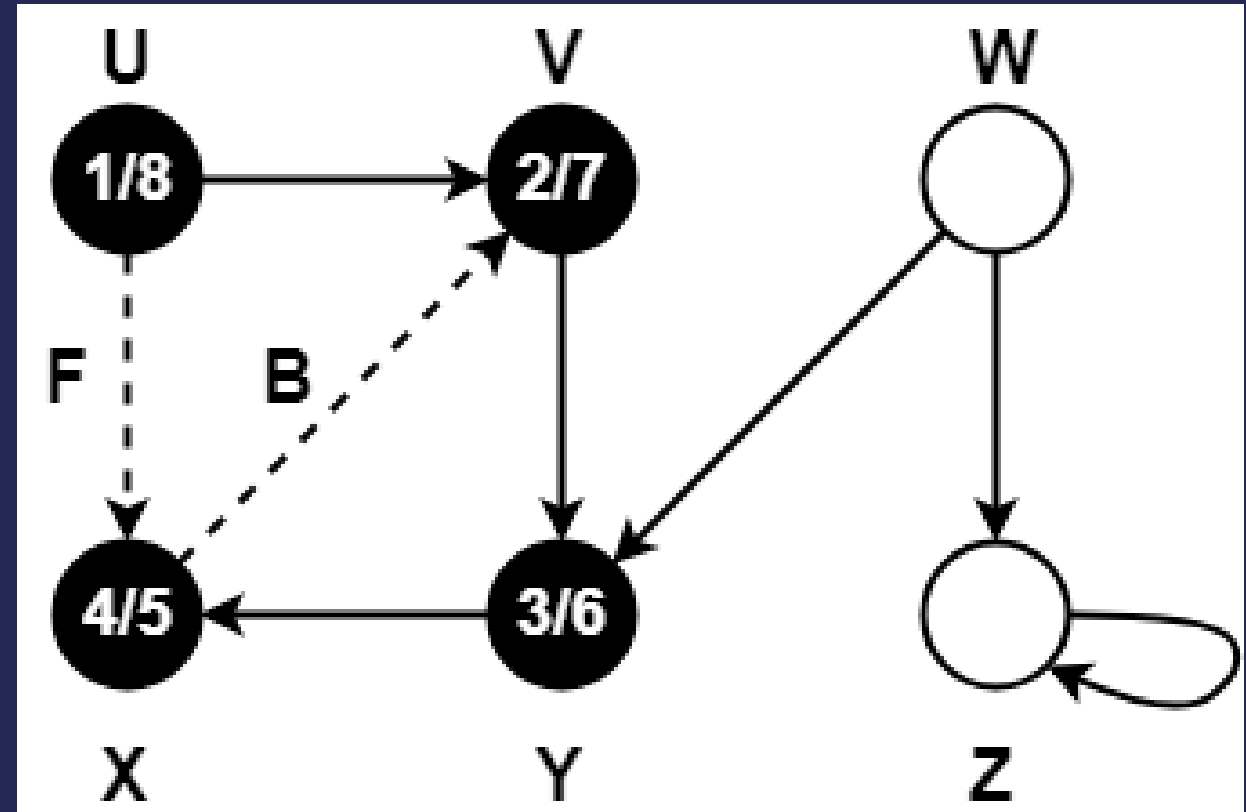




# Depth first Search

Step 10:

1.  $\text{color}[U] = \text{BLACK}$
2.  $\text{time} = 7 + 1 = 8$
3.  $f[U] = 8$

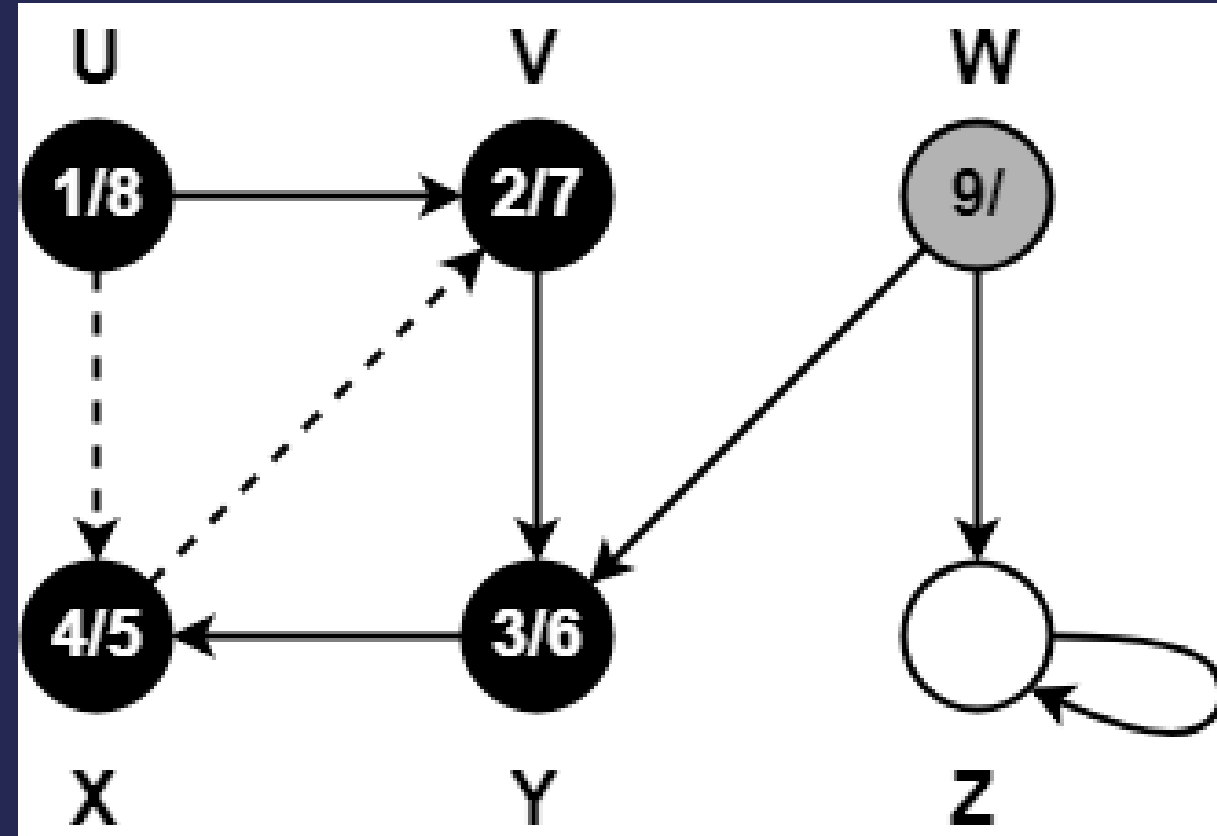




# *Depth first Search*

Step 11:

1.  $\text{color}[W] = \text{GREY}$
2.  $\text{time} = 8 + 1 = 9$
3.  $d[W] = 9$

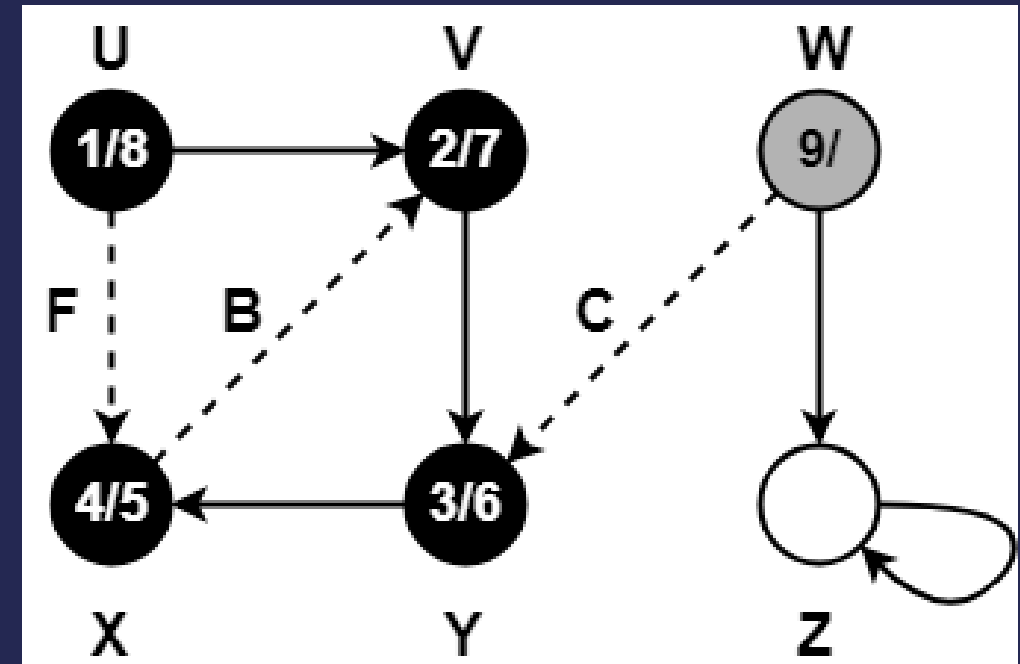




# Depth first Search

## Step 12:

- When DFS tries to extend the visit from vertex W to vertex Y, it finds-
  - $\text{Color}(Y) = \text{BLACK}$
  - $d(Y) < d(W)$
- Thus, edge WY is a cross edge.

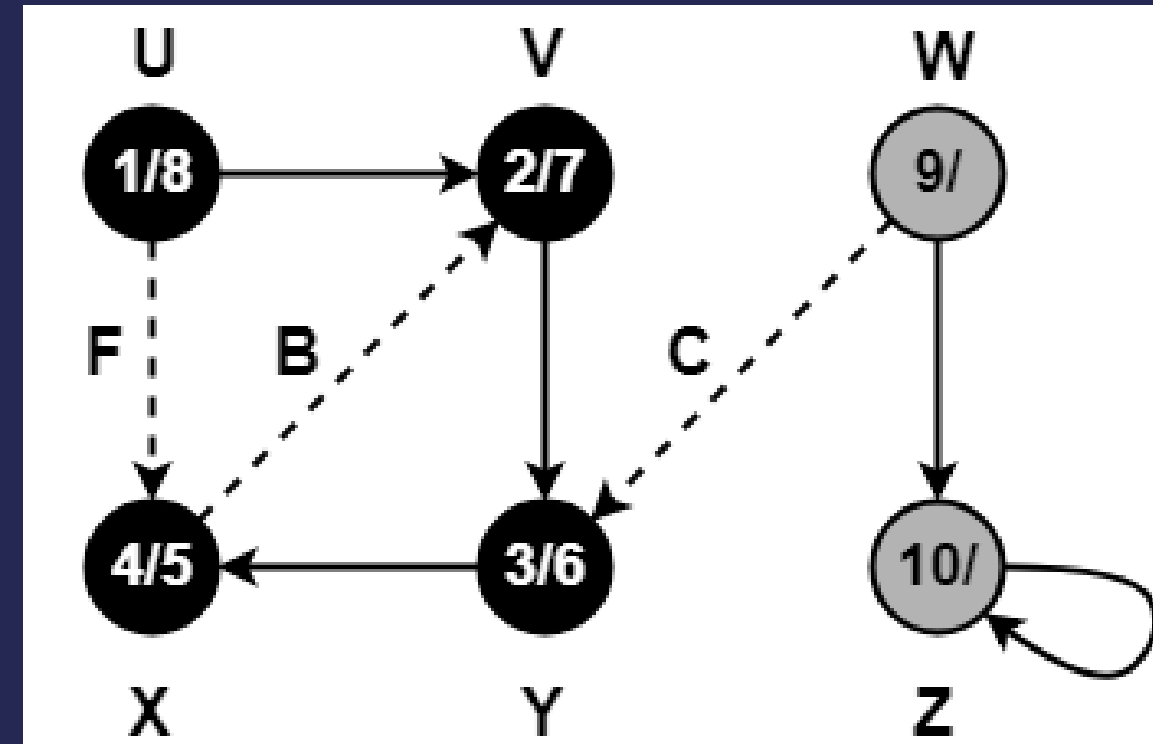




# Depth first Search

## Step 13:

1.  $\pi[Z] = W$
2.  $\text{color}[W] = \text{GREY}$
3.  $\text{time} = 9 + 1 = 10$
4.  $d[W] = 10$



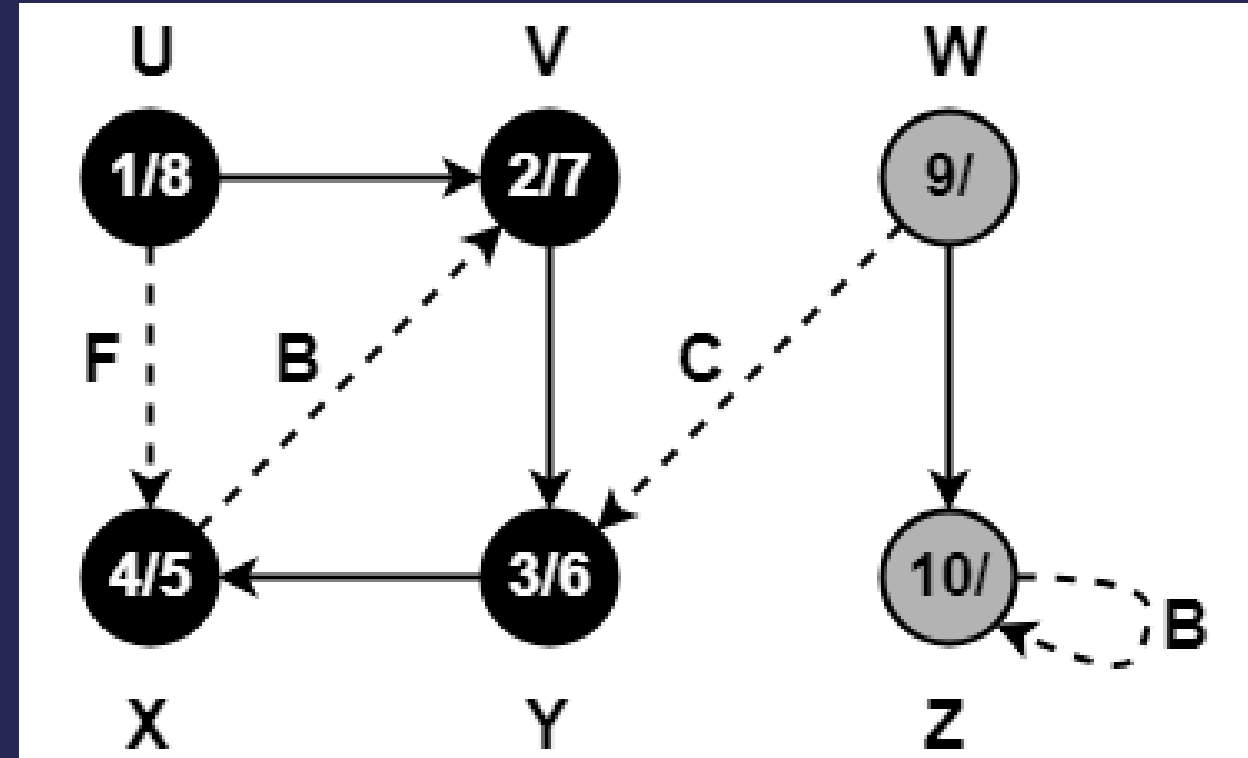




# Depth first Search

## Step 14:

- Since, self-loops are considered as back edges.
- Therefore, self-loop present on vertex Z is considered as a back edge.

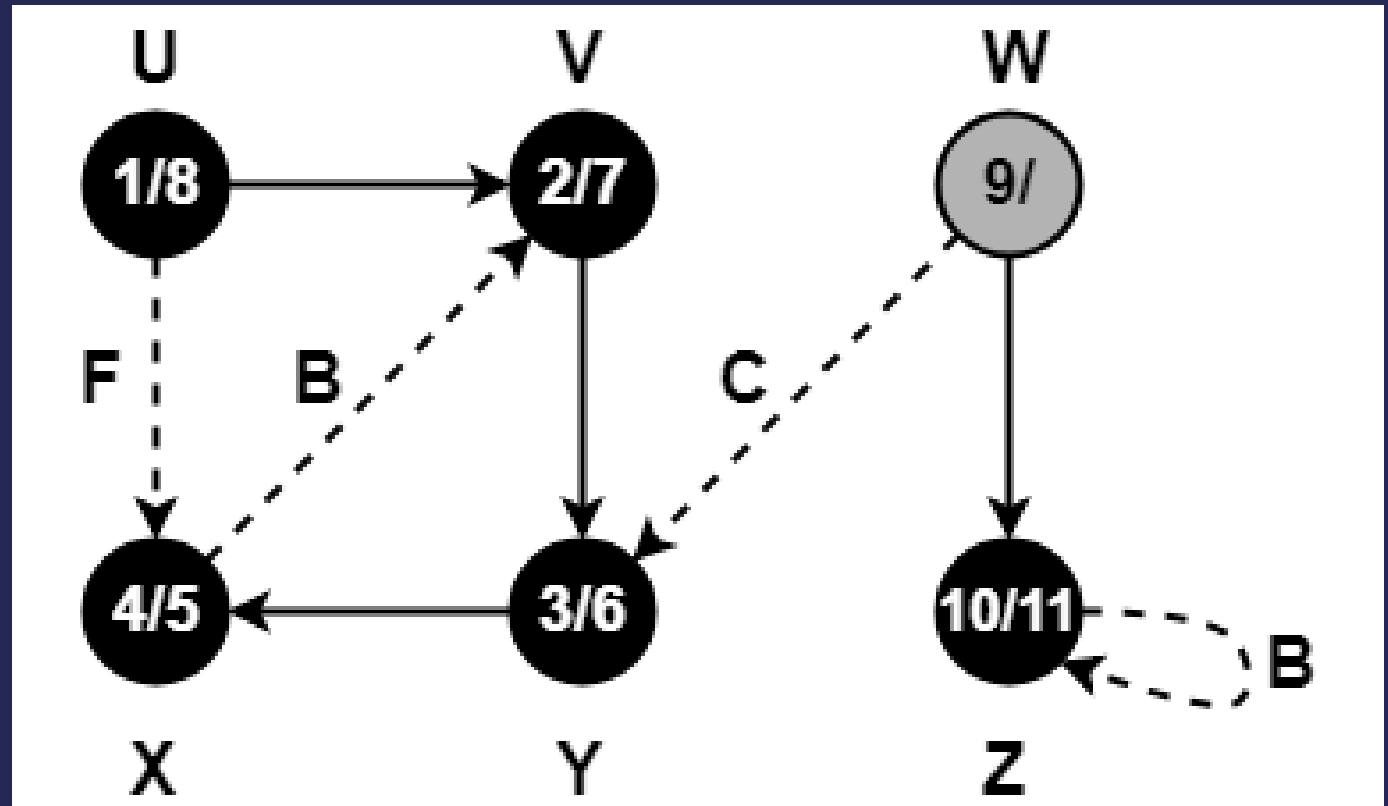




# Depth first Search

Step 15:

1.  $\text{color}[Z] = \text{BLACK}$
2.  $\text{time} = 10 + 1 = 11$
3.  $f[Z] = 11$



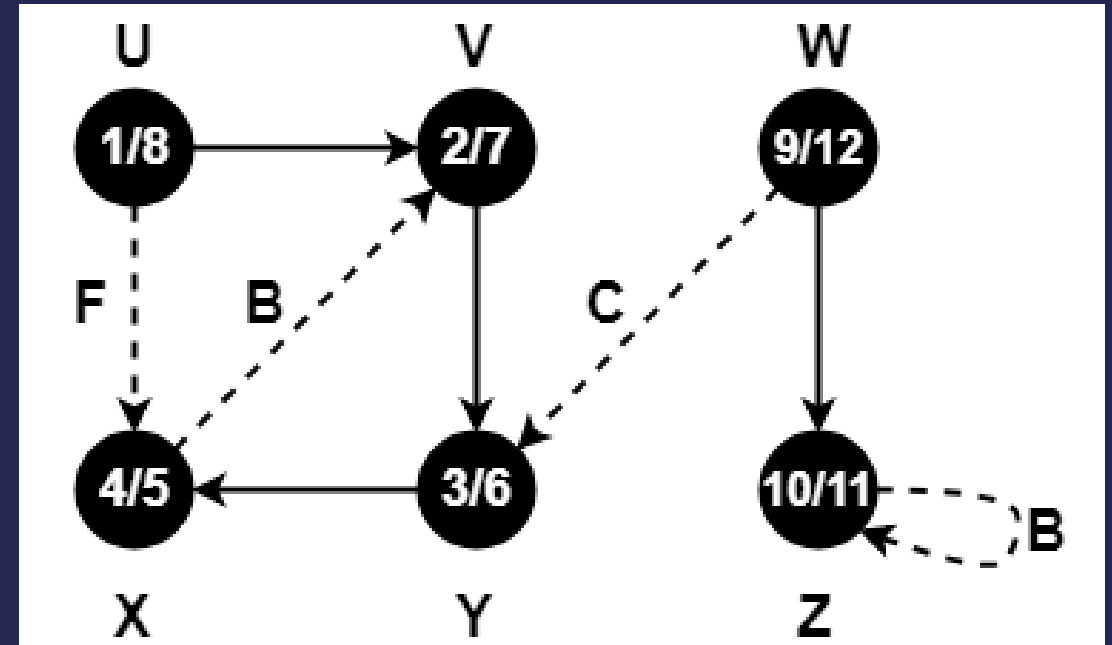


# Depth first Search

## Step 16:

1.  $\text{color}[W] = \text{BLACK}$
2.  $\text{time} = 11 + 1 = 12$
3.  $f[W] = 12$

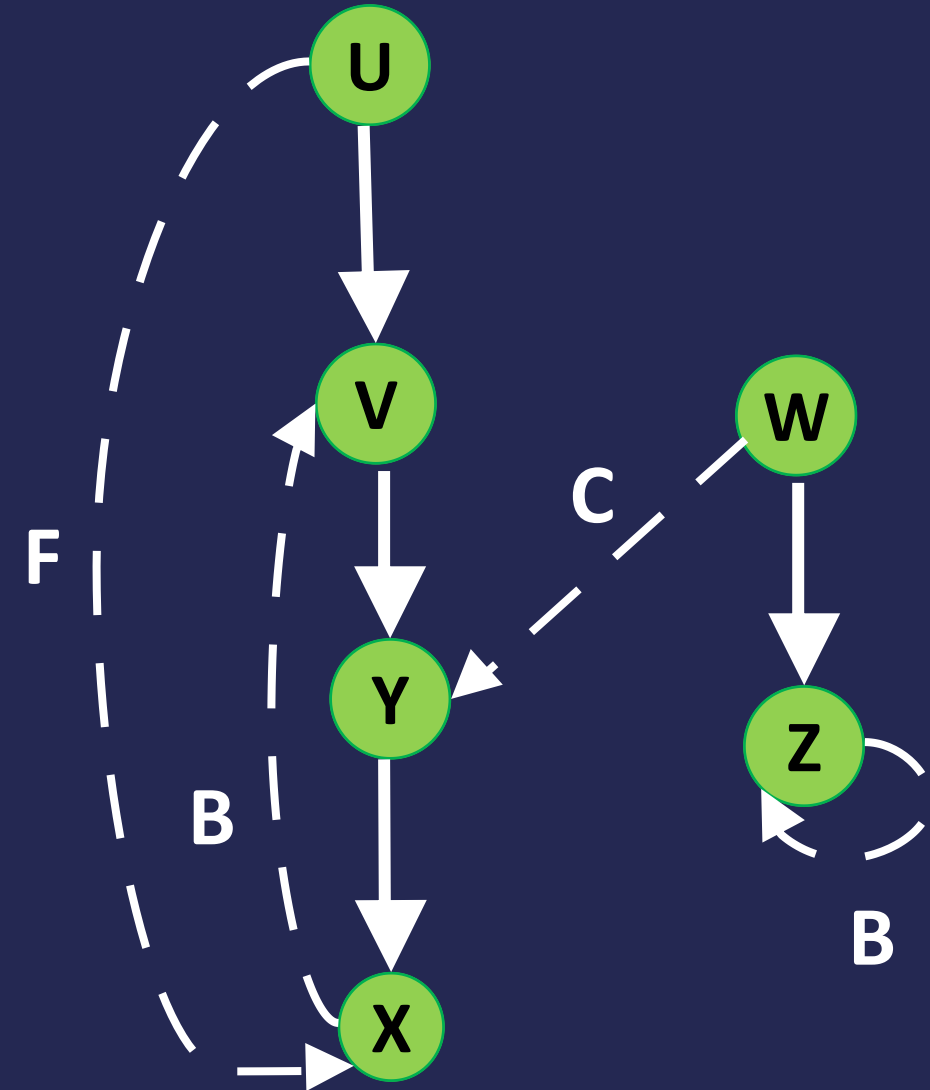
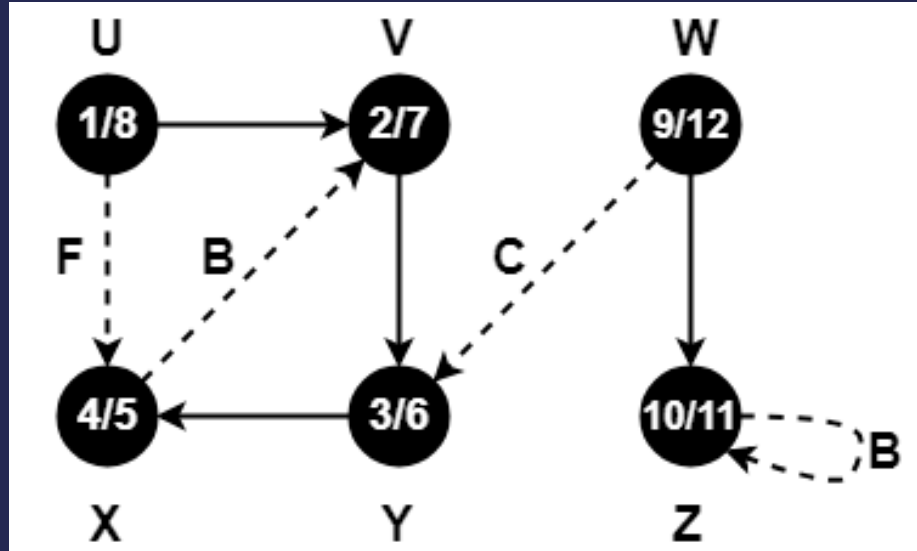
Since all the vertices have turned black, so we stop.





# Depth first Search

## Step 17: DFS Tree



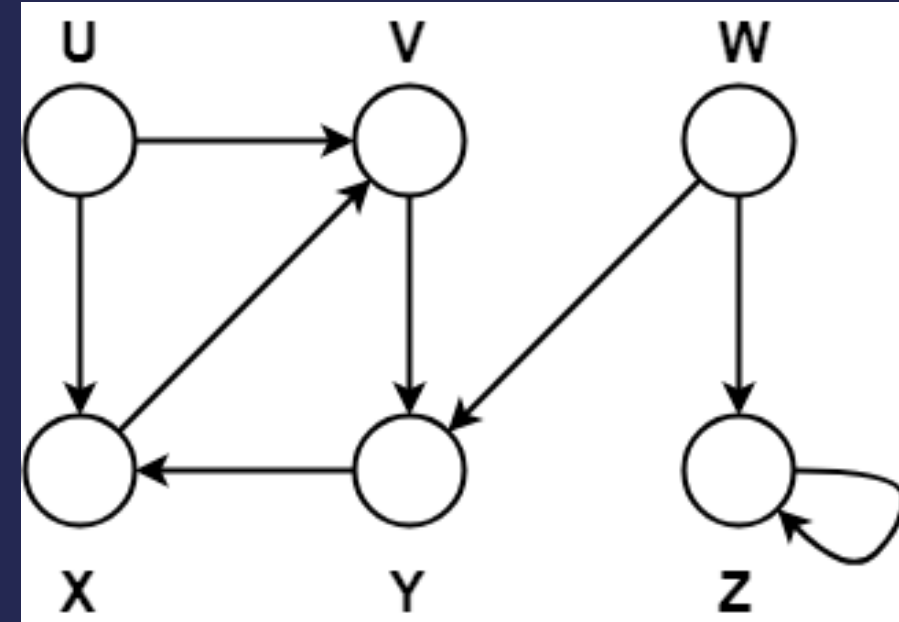


# Depth first Search

## Algorithm

DFS (G)

1. for each vertex  $u$  in  $V[G]$  do
2.      $\text{color}[u] \leftarrow \text{WHITE}$
3.      $\pi[u] \leftarrow \text{NIL}$
4.  $\text{time} \leftarrow 0$
5. for each vertex  $u$  in  $V[G]$  do
6.     if  $\text{color}[u] \leftarrow \text{WHITE}$  then
7.          $\text{Depth\_First\_Search}(G, u)$



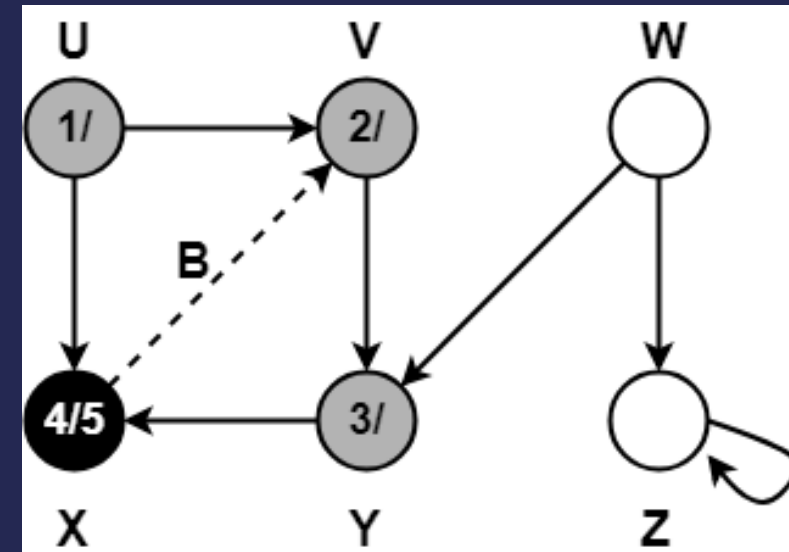
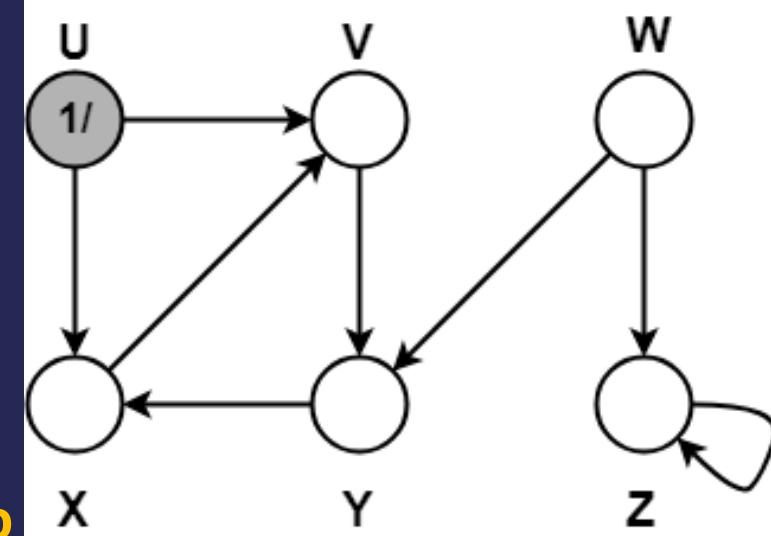


# Depth first Search

## Algorithm

Depth\_First\_Search (G,u)

1.  $\text{time} = \text{time} + 1$
2.  $d[u] = \text{time}$
3.  $\text{color}[u] = \text{GRAY}$
4. **For each adjacent vertex 'v' of 'u' do**
5.     **If**  $\text{color}[v] == \text{white}$  **then**
6.         **set**  $\pi[v] = u$
7.         **Depth\_First\_Search** (G,v)
8.      $\text{color}[u] = \text{BLACK}$
9.      $\text{time} = \text{time} + 1$
10.  $f[u] = \text{time}$





# *TOPOLOGICAL SORT*

*Mr. Prayag Patel*

*Assistant Professor  
IT-ICT Department, LJIET  
[prayag.patel@ljinstitutes.edu.in](mailto:prayag.patel@ljinstitutes.edu.in)*



# Outline

- ❖ *Difference between DFS & BFS*
- ❖ *Application of DFS*
- ❖ *Topological sort*
  - ❖ **DFS Based Algorithm**
  - ❖ **Source Removal Algorithm**





# *Difference between DFS & BFS*

| Sr. No | BFS                                                                                         | DFS                                                                                                                |
|--------|---------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| 1      | BFS, stands for Breadth First Search.                                                       | DFS, stands for Depth First Search.                                                                                |
| 2      | BFS uses Queue to find the shortest path.                                                   | DFS uses Stack to find the shortest path.                                                                          |
| 3      | BFS is better when target is closer to Source.                                              | DFS is better when target is far from source.                                                                      |
| 4      | As BFS considers all neighbor so it is not suitable for decision tree used in puzzle games. | DFS is more suitable for decision tree. As with one decision, we need to traverse further to augment the decision. |
| 5      | BFS is slower than DFS.                                                                     | DFS is faster than BFS.                                                                                            |
| 6      | Time Complexity of BFS = $O(V+E)$ where V is vertices and E is edges.                       | Time Complexity of DFS is also $O(V+E)$ where V is vertices and E is edges.                                        |

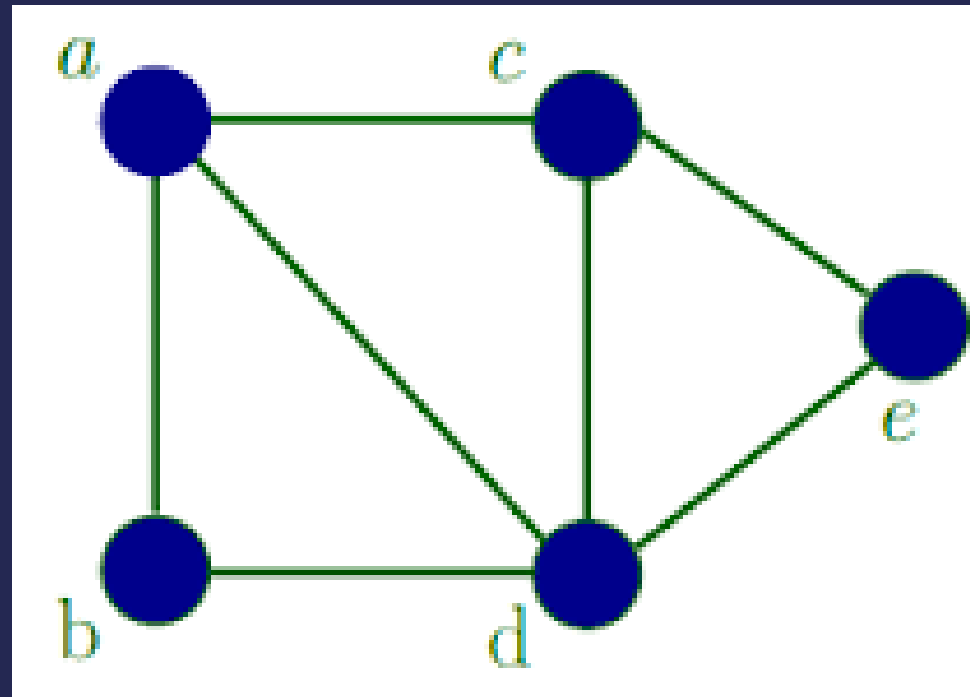


# Application of DFS

## Three Application of DFS:

1) DFS is used for checking connectivity of a graph.

If all the vertices of graph are visited then graph is said to be a connected graph.



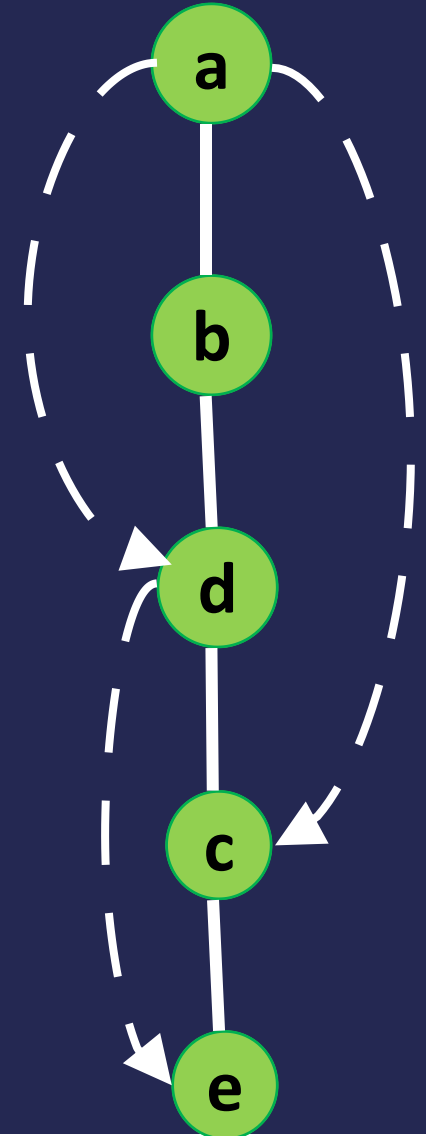
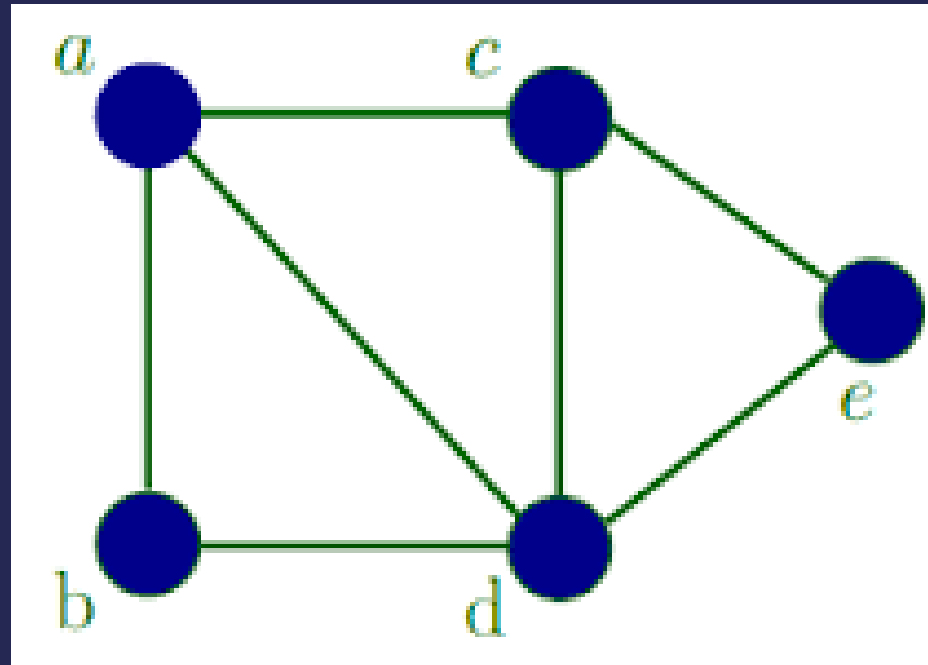


# Application of DFS

## Three Application of DFS:

2) DFS is used for checking Acyclicity of a graph.

If DFS forest does not have back edge then the graph is said to be Acyclic.



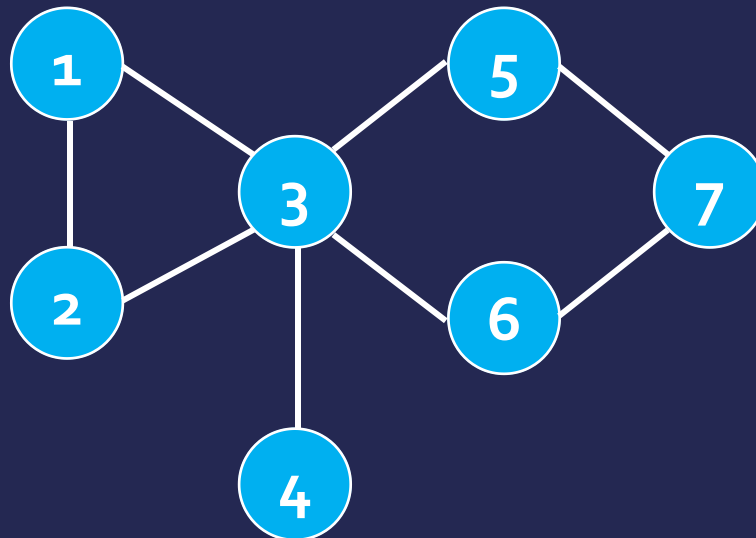


# Application of DFS

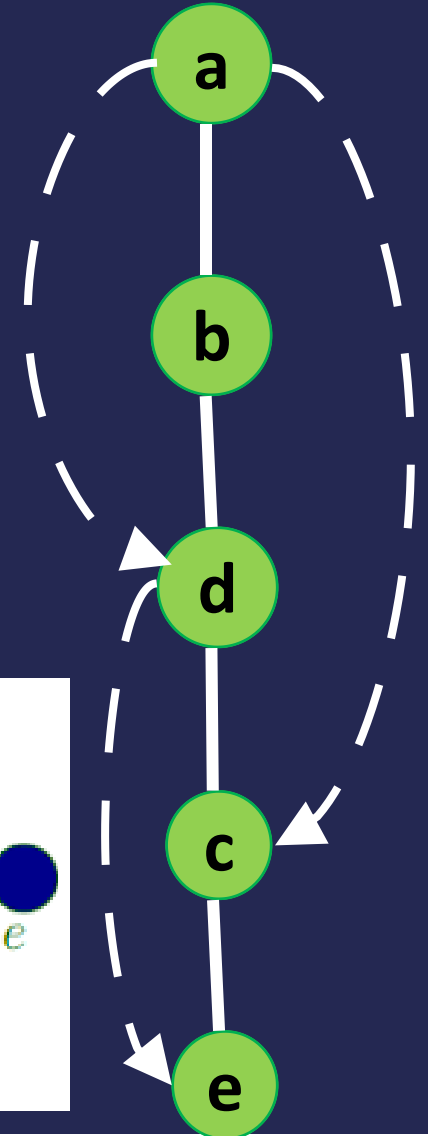
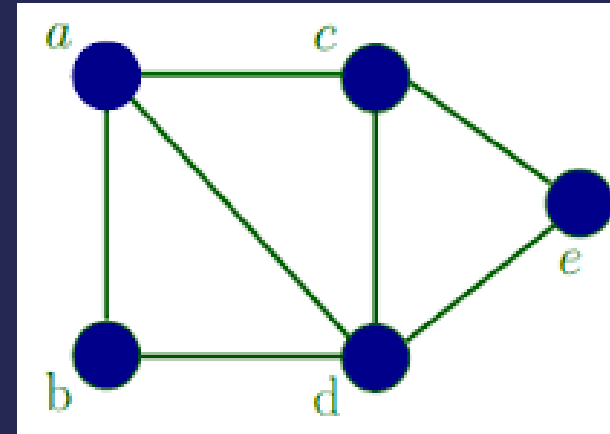
## Three Application of DFS:

3) DFS is used to find Articulation point.

A vertex of connected graph is said to be its articulation point if its removal with all its incident edges breaks the graph into disjoint pieces.



Articulation Points: 3

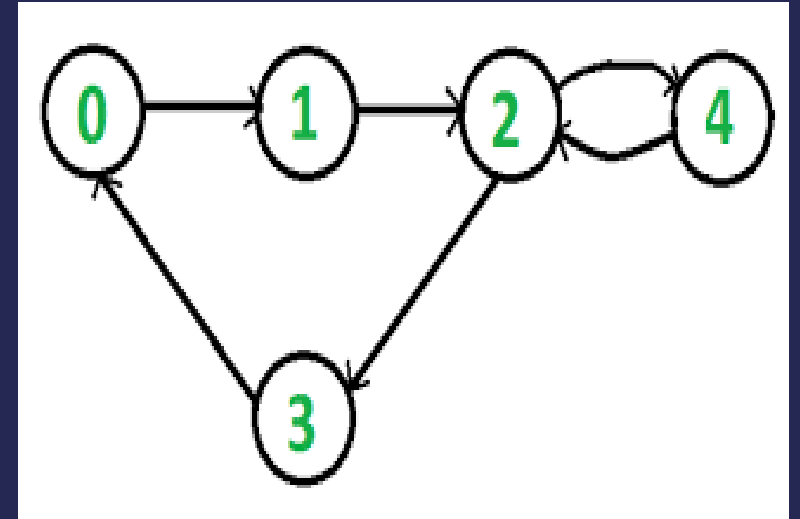
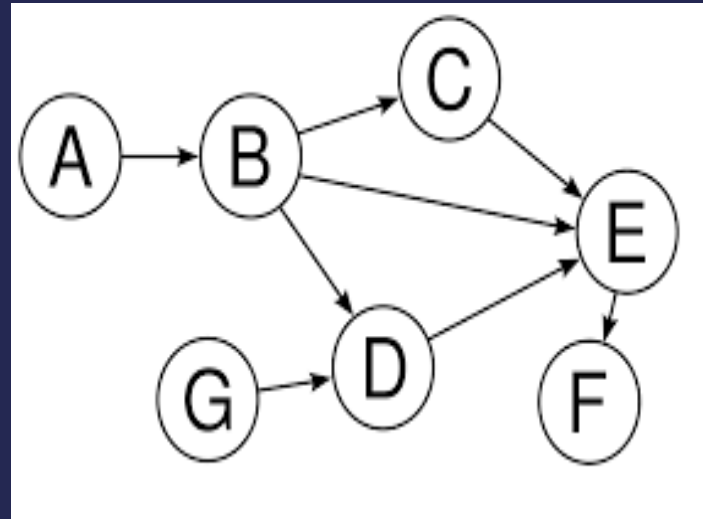




# *Topological sort*

## Definition of DAG:

A directed Acyclic graph is a directed graph with no cycles.

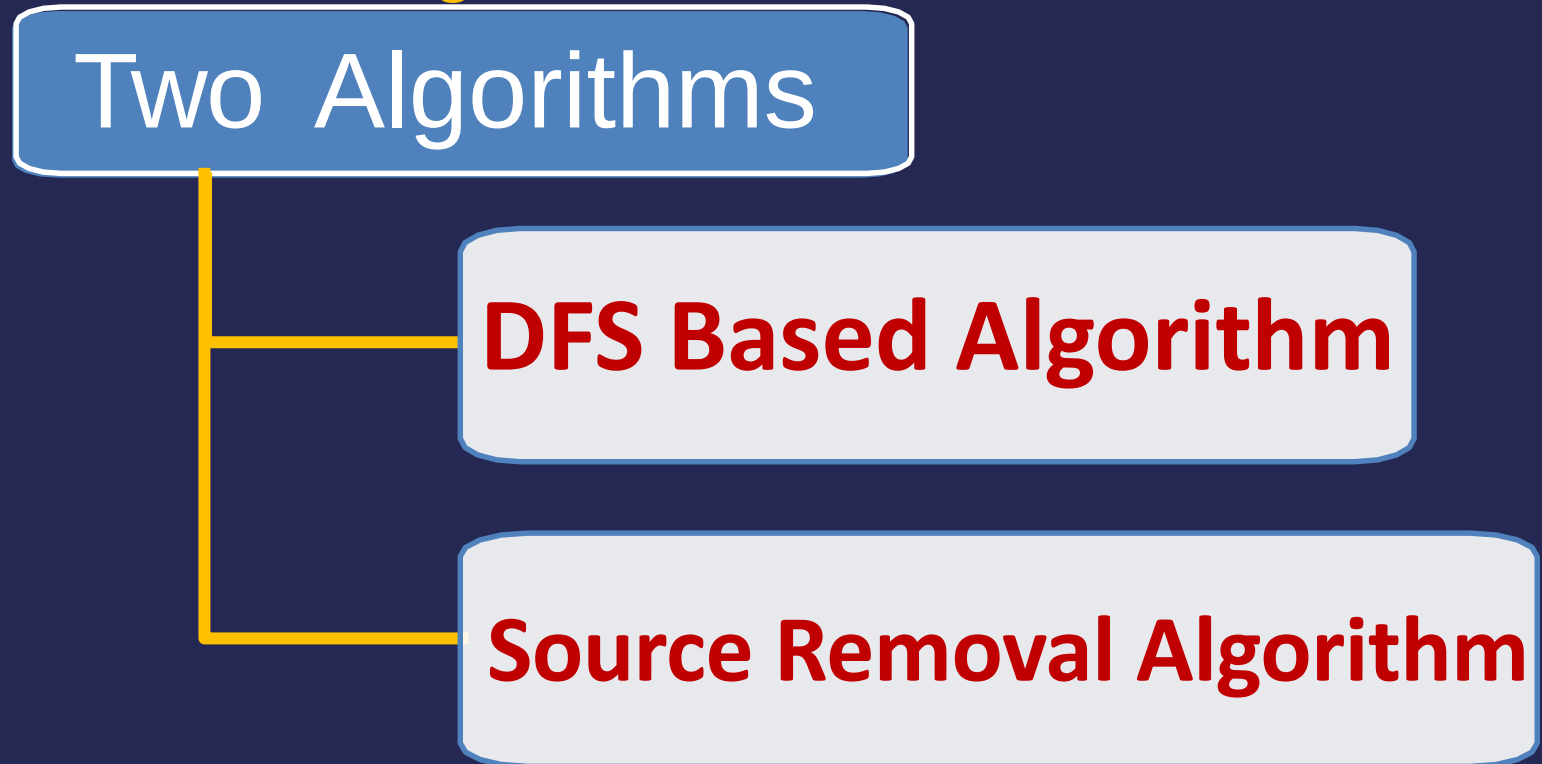




# *Topological sort*

## Topological Sort:

A topological sort of a DAG  $G = (V, E)$  is a linear ordering of all its vertices such that if  $G$  contains an edge  $(u, v)$ , then **u appears before v in the ordering.**





# *DFS Based Algorithm*

## **DFS based Algorithm for Topological order:**

- TOPOLOGICAL-SORT( $G$ ):
  - 1) call DFS( $G$ ) to compute **finishing** times  $f[v]$  for each vertex  $v$
  - 2) as each vertex is finished, insert it onto the **front** of a linked list
  - 3) return the linked list of vertices
- Note that the result is just a list of vertices in order of **decreasing** finish times  $f[]$

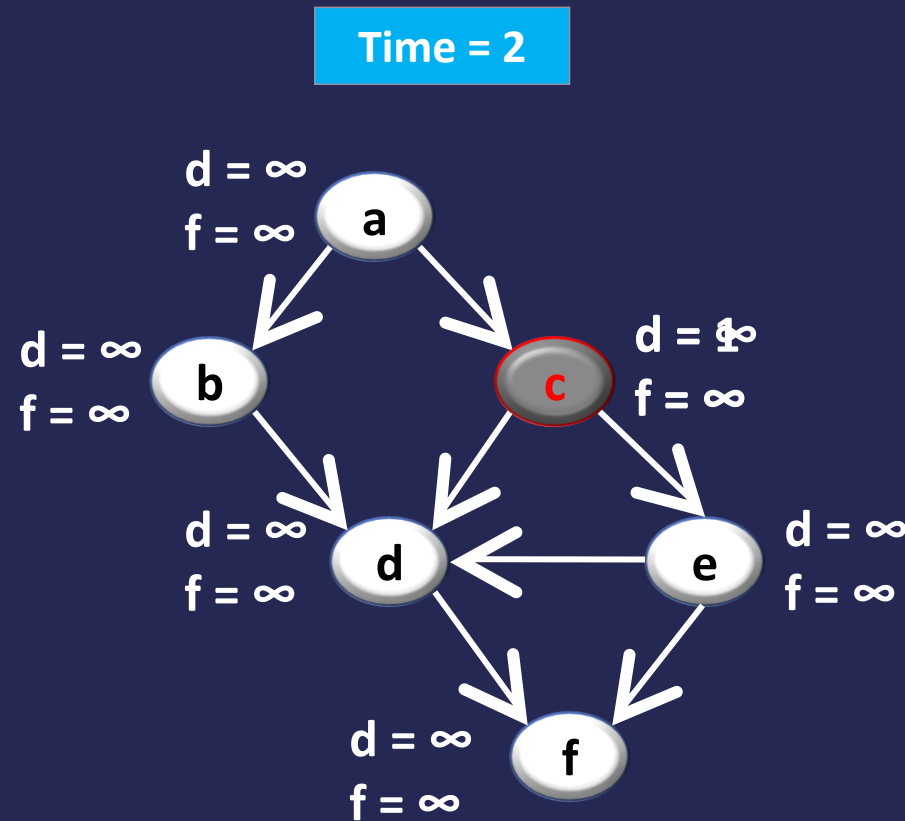


# DFS based Algorithm

1) Call DFS(**G**) to compute the finishing times **f[v]**

Let's say we start the DFS from the vertex **c**

Next we discover the vertex **d**





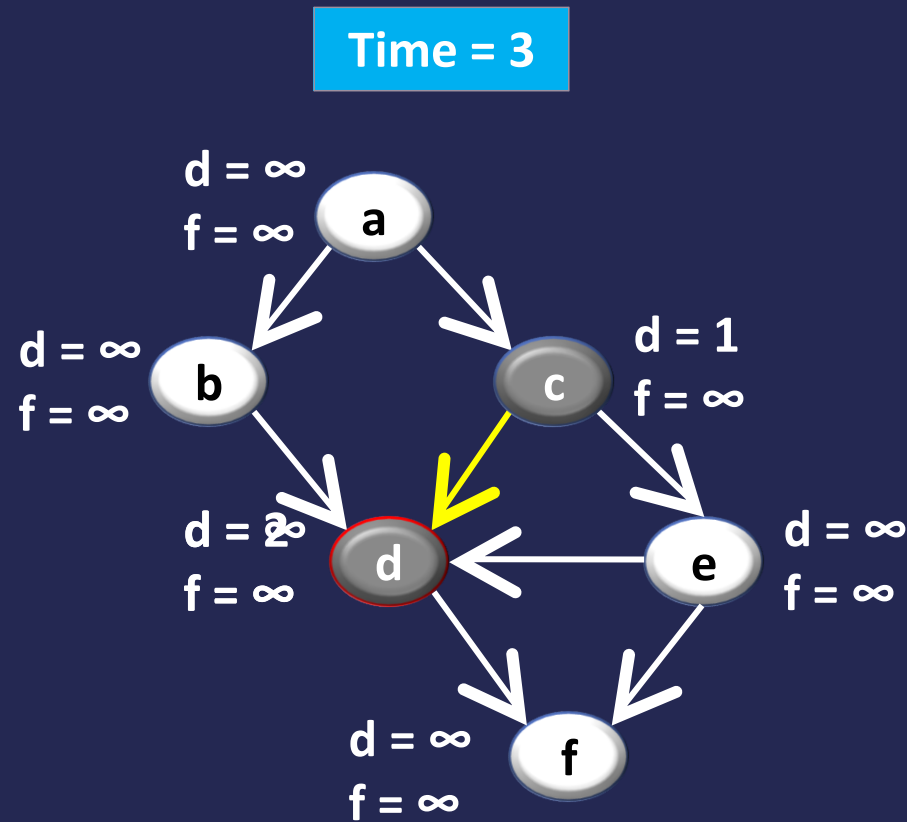


# DFS based Algorithm

1) Call DFS(**G**) to compute the finishing times **f[v]**

Let's say we start the DFS from the vertex **c**

Next we discover the vertex **d**





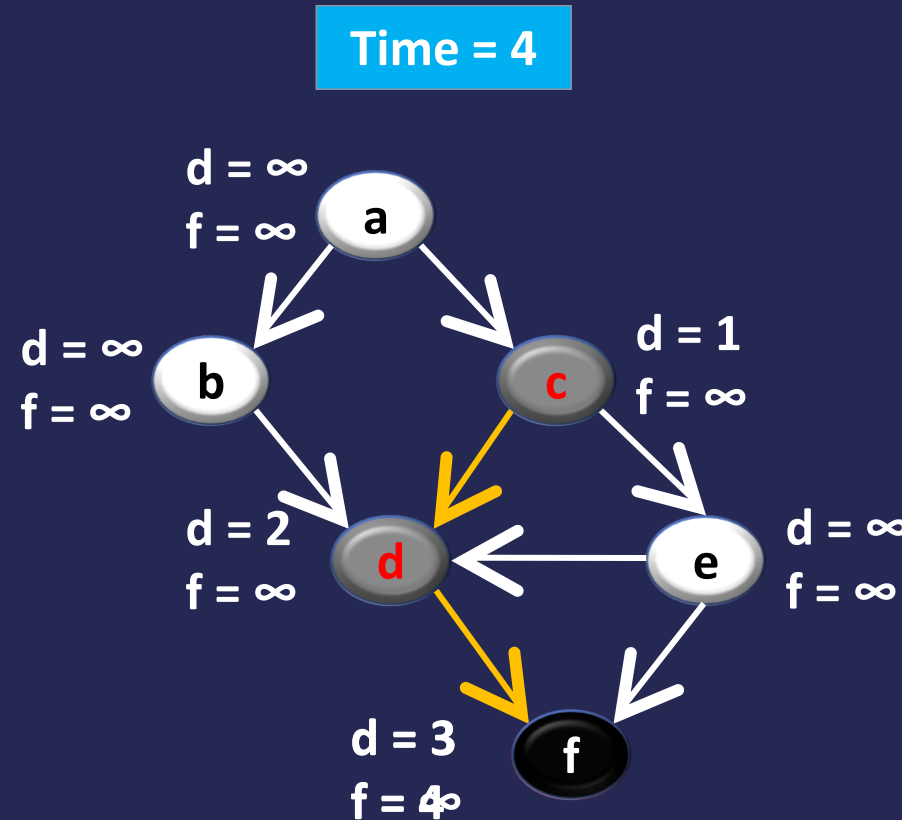
# DFS based Algorithm

1) Call DFS(**G**) to compute the finishing times **f[v]**

2) as each vertex is finished, insert it onto the **front** of a linked list

Next we discover the vertex **f**

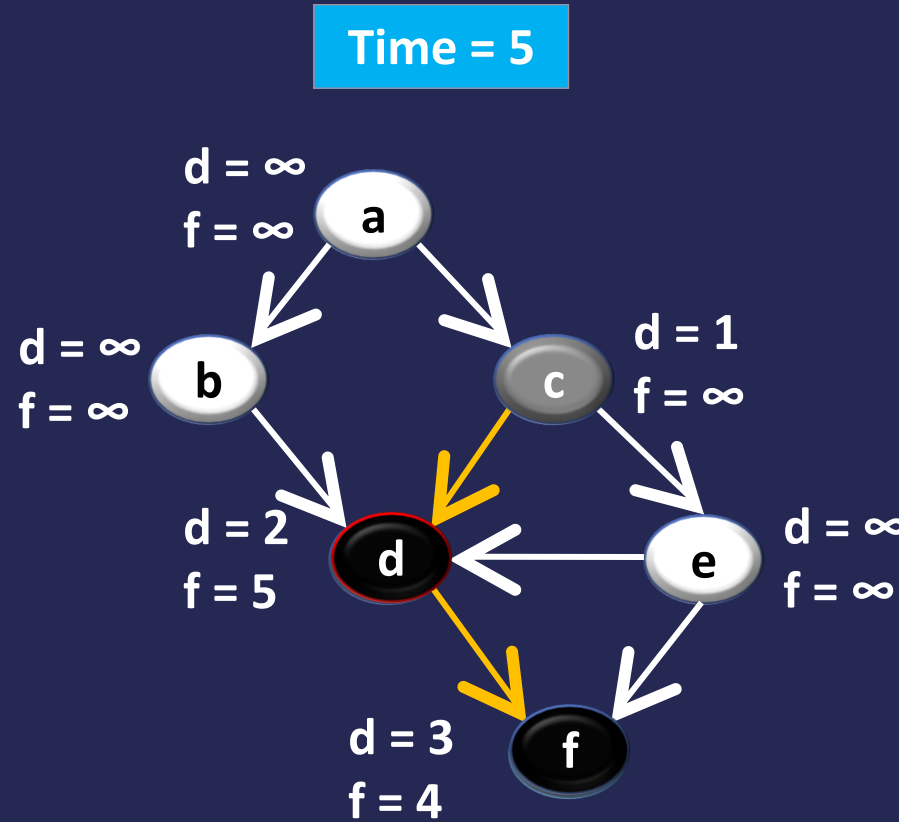
**f** is done, move back to **d**





# DFS based Algorithm

1) Call DFS(**G**) to compute the finishing times **f[v]**



Let's say we start the DFS from the vertex **c**

Next we discover the vertex **d**

Next we discover the vertex **f**

**f** is done, move back to **d**

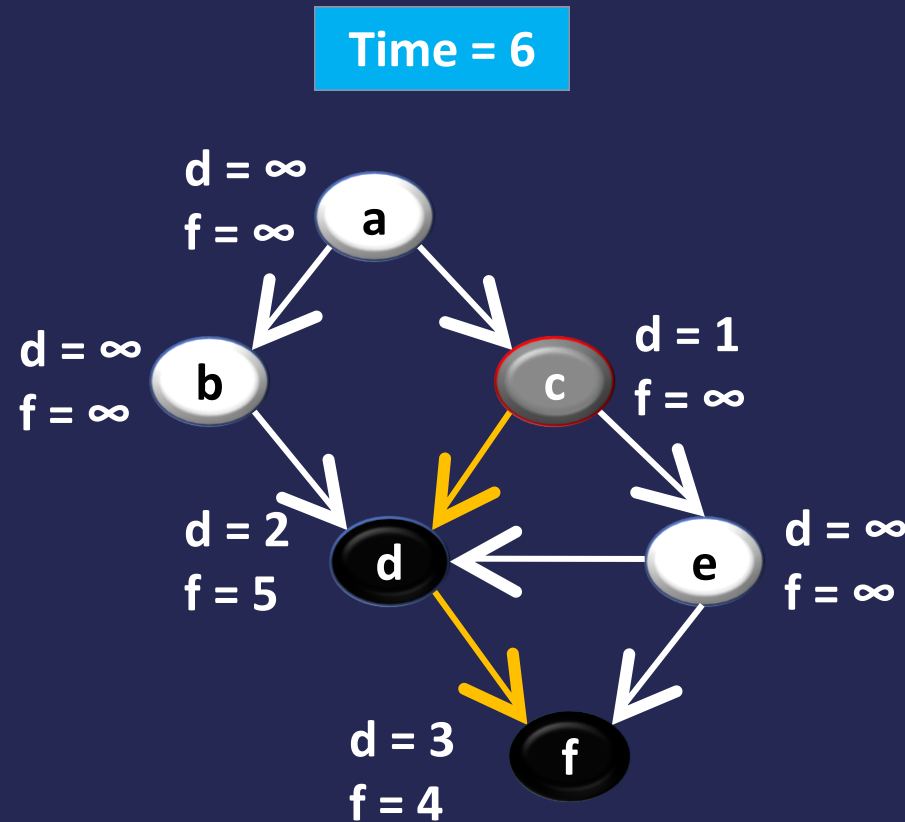
**d** is done, move back to **c**





# DFS based Algorithm

1) Call DFS(**G**) to compute the finishing times **f[v]**



Let's say we start the DFS from the vertex **c**

Next we discover the vertex **d**

Next we discover the vertex **f**

**f** is done, move back to **d**

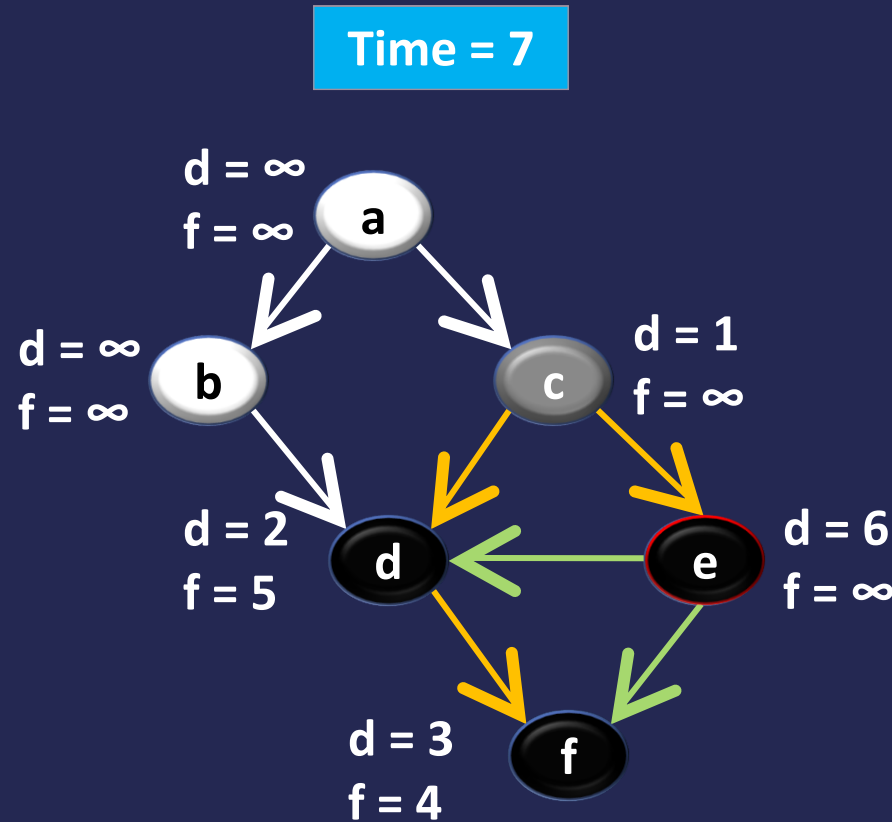
**d** is done, move back to **c**

Next we discover the vertex **e**



# DFS based Algorithm

1) Call DFS(**G**) to compute the finishing times **f[v]**



Let's say we start the DFS from the vertex **c**

Next we discover the vertex **d**

Both edges from **e** are **cross edges**

**d** is done, move back to **c**

Next we discover the vertex **e**

**e** is done, move back to **c**

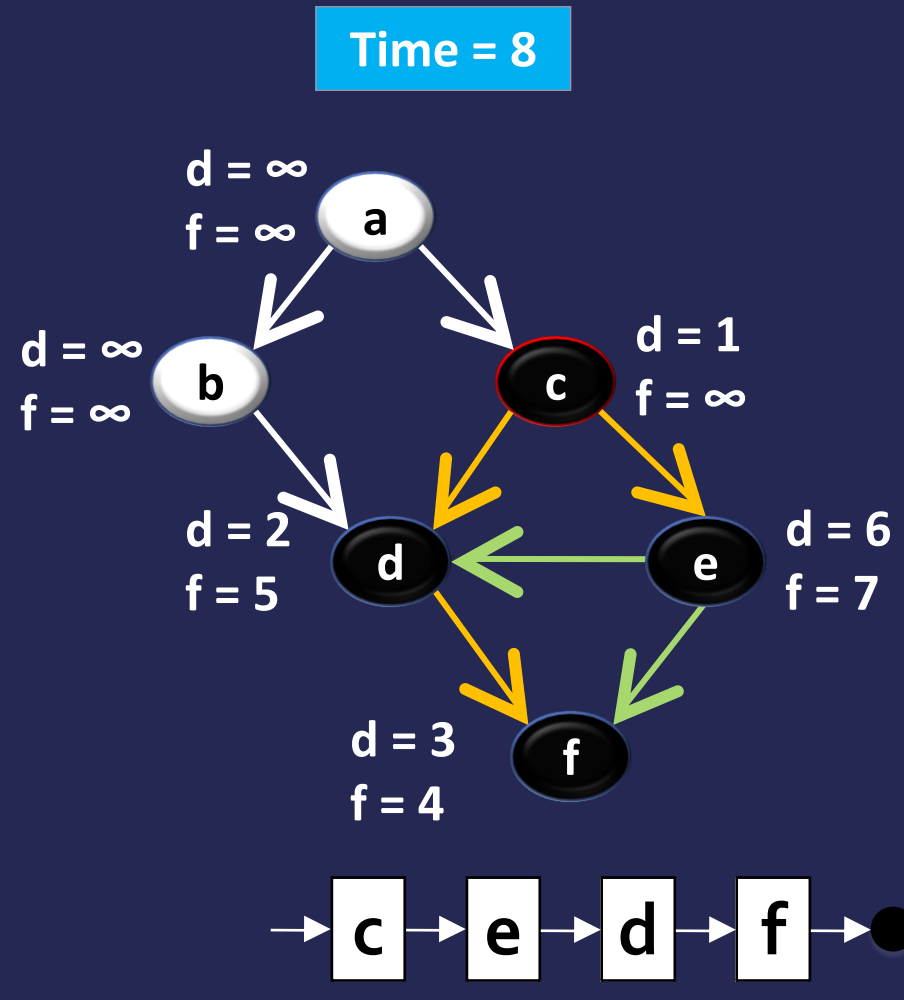


# DFS based Algorithm

1) Call DFS(**G**) to compute the finishing times **f[v]**

Let's say we start the DFS from the vertex **c**

Just a note: If there was (**c,f**) edge in the graph, it would be classified as a **forward edge** (in this particular DFS run)



**d** is done, move back to **c**

Next we discover the vertex **e**

**e** is done, move back to **c**

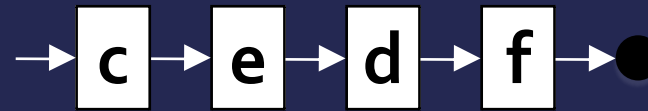
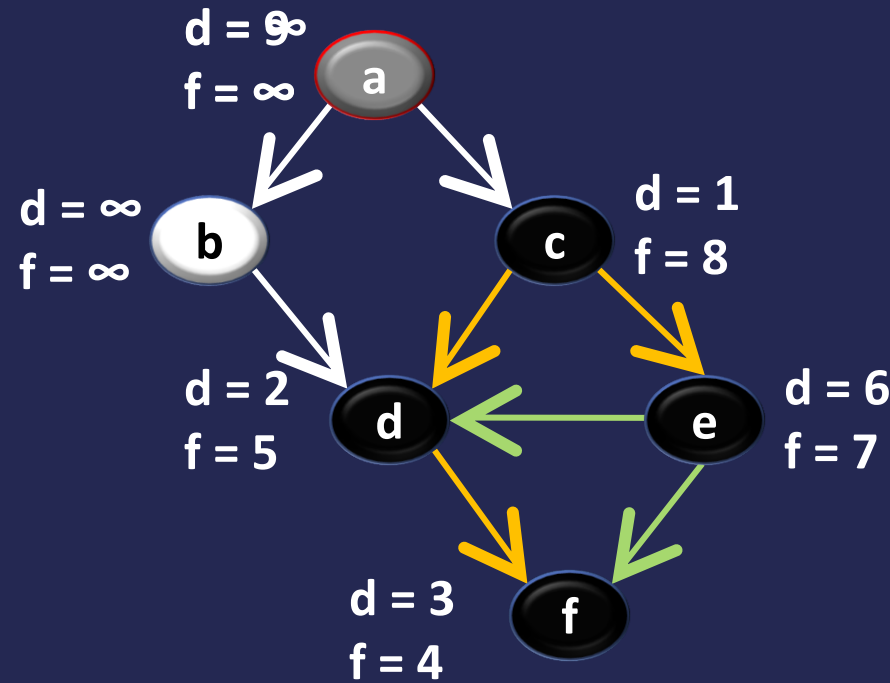
**c** is done as well



# DFS based Algorithm

1) Call DFS(**G**) to compute the finishing times **f[v]**

Time = 9



Let's now call DFS visit from the vertex **a**

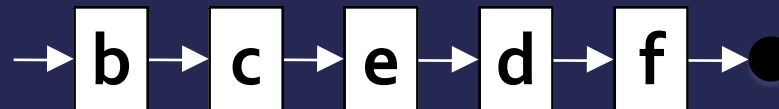
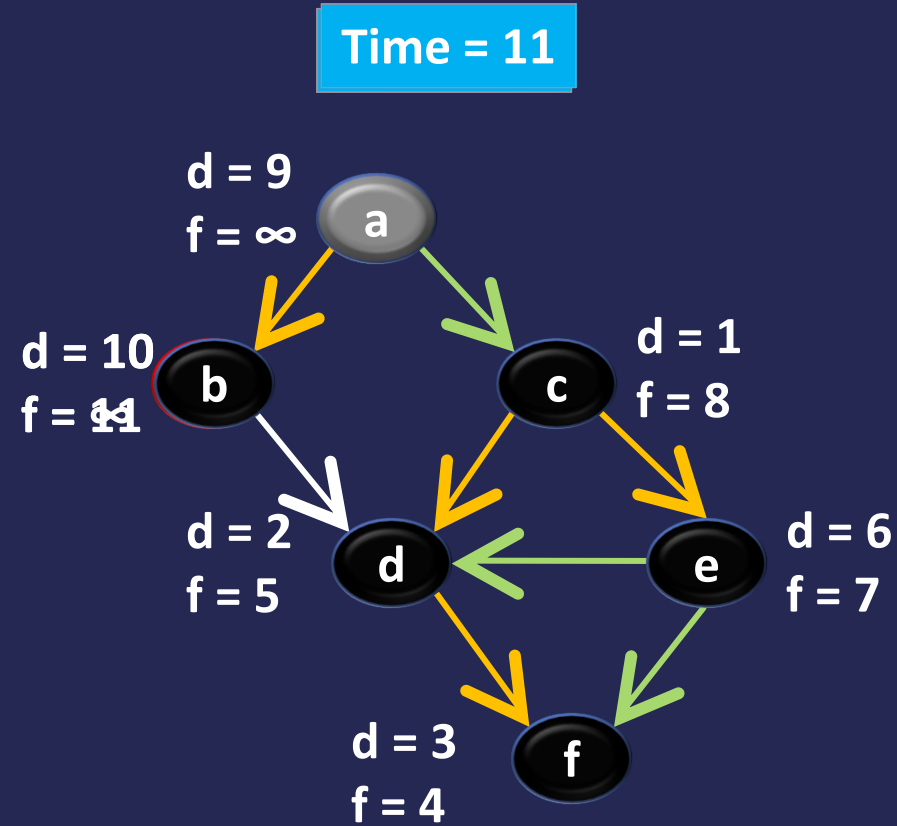
Next we discover the vertex **c**, but **c** was already processed  $\Rightarrow$  (**a,c**) is a cross edge

Next we discover the vertex **b**



# DFS based Algorithm

1) Call DFS(**G**) to compute the finishing times **f[v]**



Let's now call DFS visit from the vertex **a**

Next we discover the vertex **c**, but **c** was already processed  $\Rightarrow$  (**a,c**) is a cross edge

Next we discover the vertex **b**

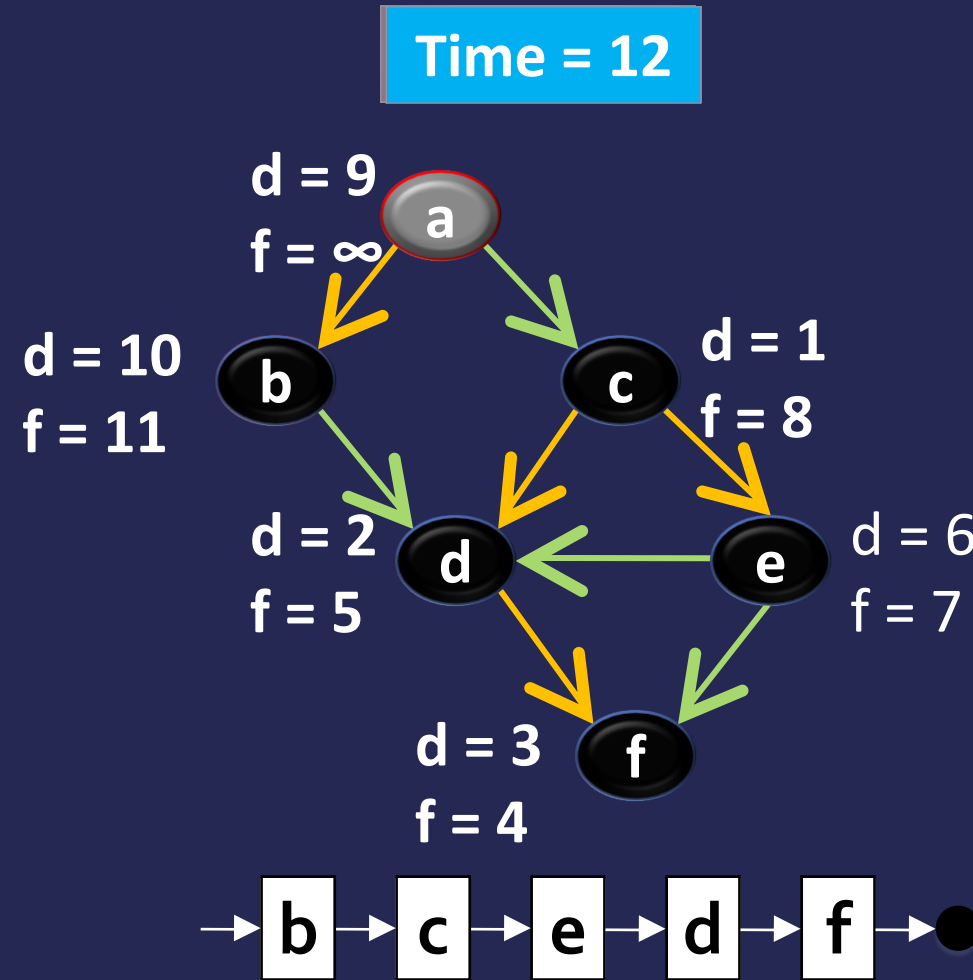
**b** is done as (**b,d**) is a cross edge  $\Rightarrow$  now move back to **c**





# DFS based Algorithm

1) Call DFS(**G**) to compute the finishing times **f[v]**



Let's now call DFS visit from the vertex **a**

Next we discover the vertex **c**, but **c** was already processed  $\Rightarrow$  (**a,c**) is a cross edge

Next we discover the vertex **b**

**b** is done as (**b,d**) is a cross edge  $\Rightarrow$  now move back to **c**

**a** is done as well

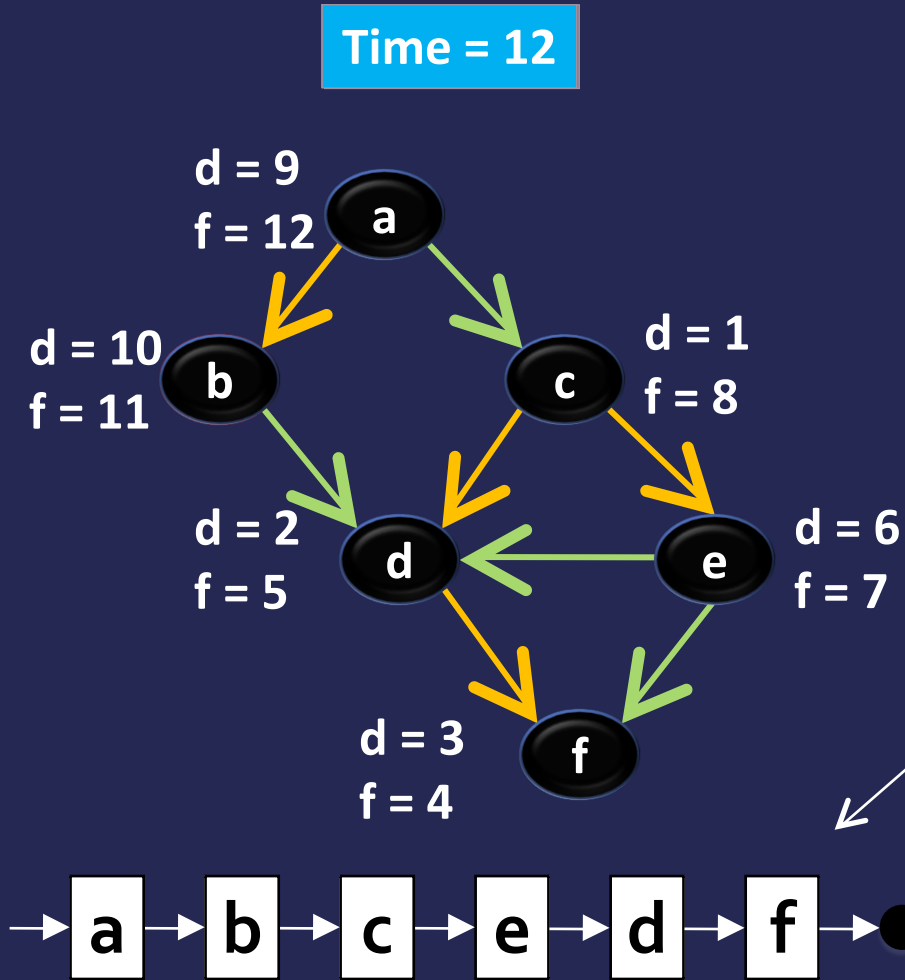


# DFS based Algorithm

1) Call DFS(**G**) to compute the finishing times **f[v]**

**WE HAVE THE RESULT!**

3) return the linked list of vertices



$\Rightarrow (a,c)$  is a cross edge

Next we discover the vertex **b**

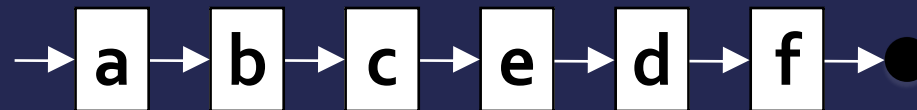
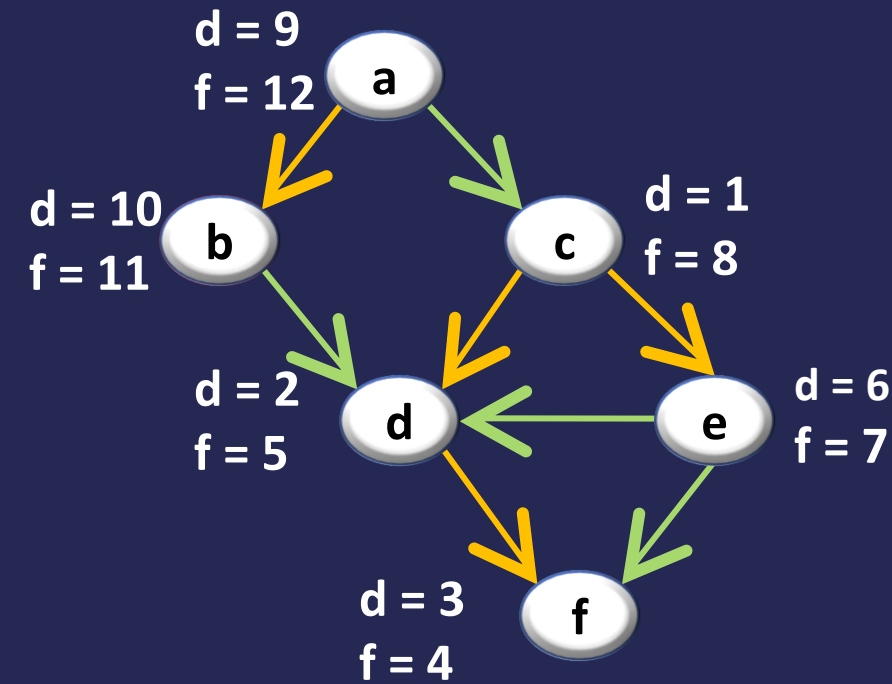
**b** is done as  $(b,d)$  is a cross edge  $\Rightarrow$  now move back to **c**

**a** is done as well



# DFS based Algorithm

Time = 12



The linked list is sorted in **decreasing** order of finishing times  $f[]$

Try yourself with different vertex order for DFS visit

Note: If you redraw the graph so that all vertices are in a line ordered by a valid topological sort, then all edges point „from left to right“



# *Topological sort*

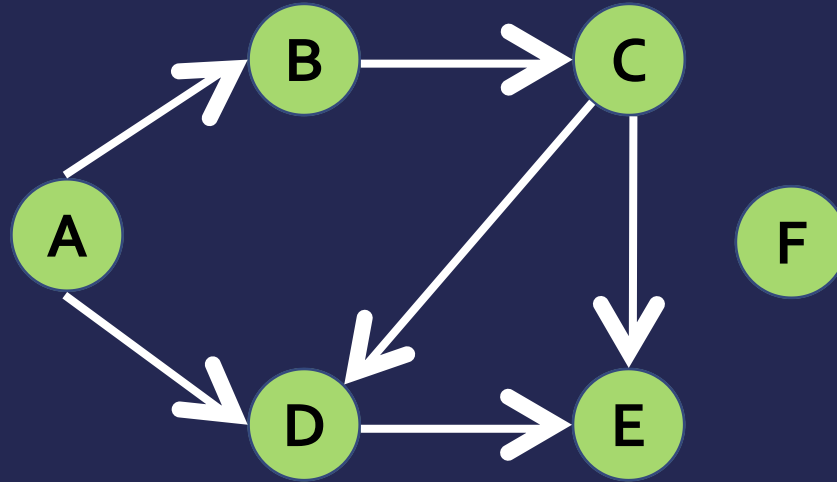
## Two Algorithms

**DFS Based Algorithm**

**Source Removal Algorithm**



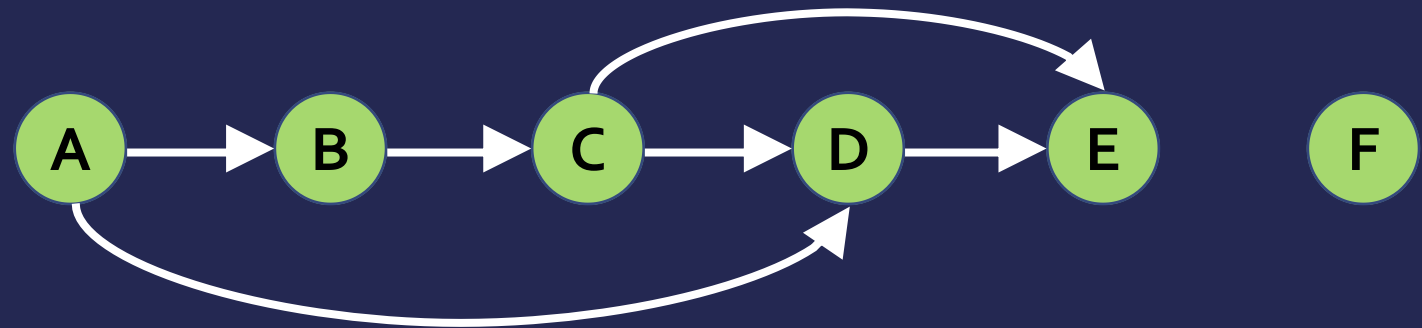
# Source Removal Algorithm



Identify nodes having in degree '0'

Select a node and delete it with its edges then add node to output

Output:





# *ARTICULATION POINT & STRONGLY CONNECTED COMPONENTS*

*Mr. Prayag Patel*

*Assistant Professor  
IT-ICT Department, LJIE  
[prayag.patel@ljinstitutes.edu.in](mailto:prayag.patel@ljinstitutes.edu.in)*



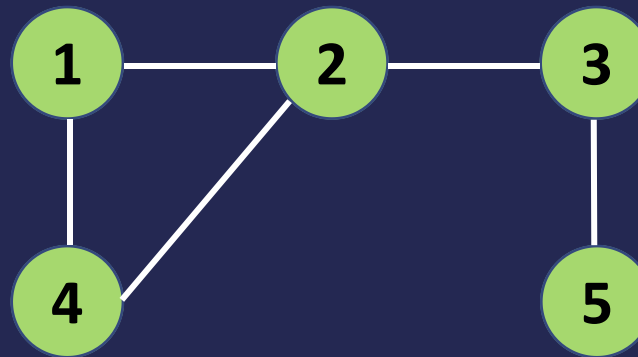
# *Outline*

- ❖ *Articulation Point*
- ❖ Strongly Connected components

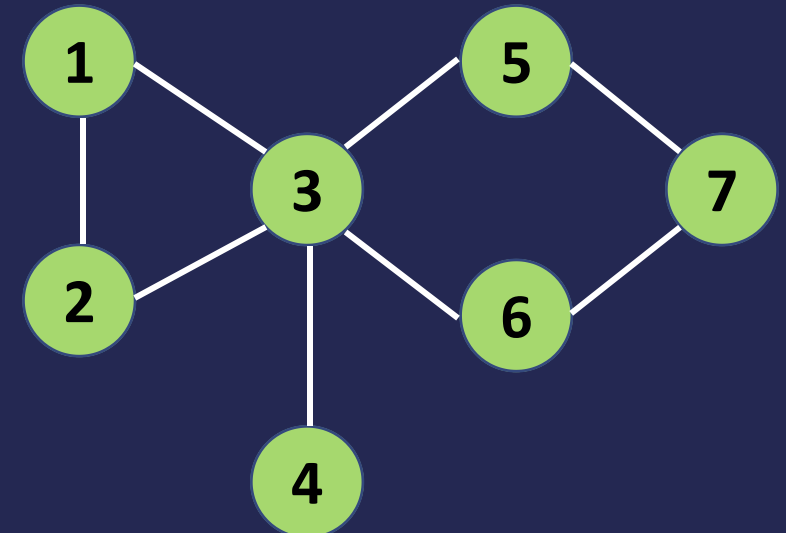


# Articulation Point

- An **articulation point** in a connected graph is a vertex , that if deleted then disconnects the graph.
- In a connected network, single points whose failure would split network into two or more disconnected components.



Articulation Points: 2,3



Articulation Points: 3





# Articulation Point

- In DFS tree, Check following conditions for articulation point:
  - 1)  $u$  is root of DFS tree and it has at least two children.
  - 2) A leaf node of DFS tree is not an Articulation Point.
  - 3)  $u$  is not root of DFS tree and it has a child  $v$  such that no vertex in subtree rooted with  $v$  has a back edge to one of the ancestors (in DFS tree) of  $u$ .

$$\text{low}[u] = \min\{\text{dfn}[u], \min\{\text{low}[w]\}, \min\{\text{dfn}[w]\}\}$$

$w$  is a child of  $u$ .

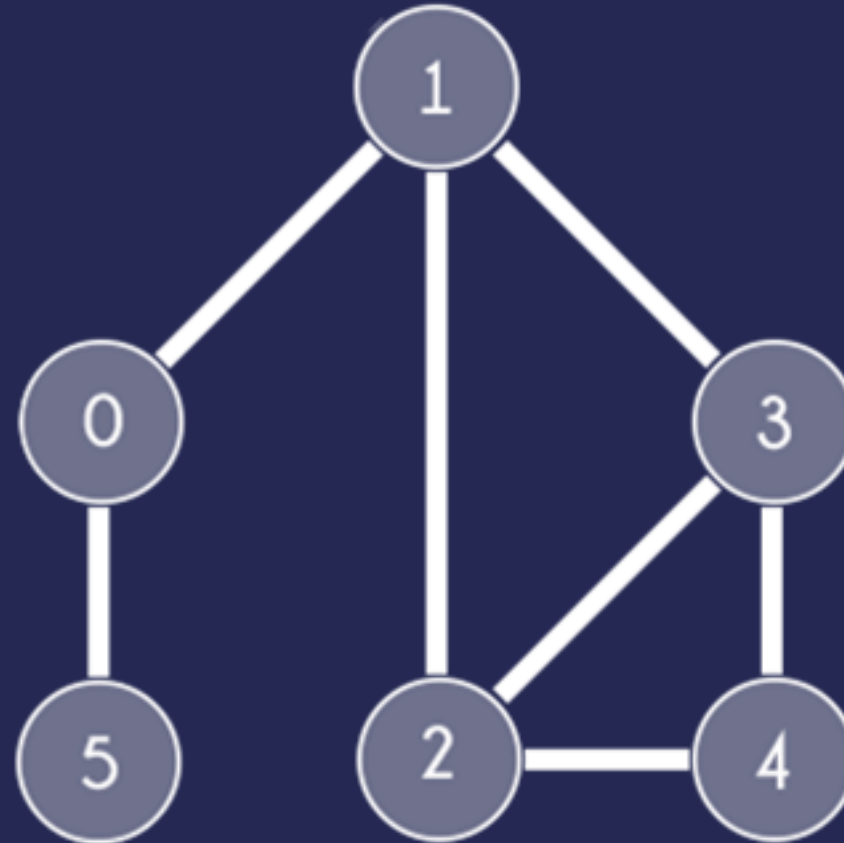
$(u, w)$  is a back edge.

Vertex  $u$  is an articulation point if  $w$  is child of  $u$  such that  $\text{low}[w] \geq \text{dfn}[u]$



# *Articulation Point*

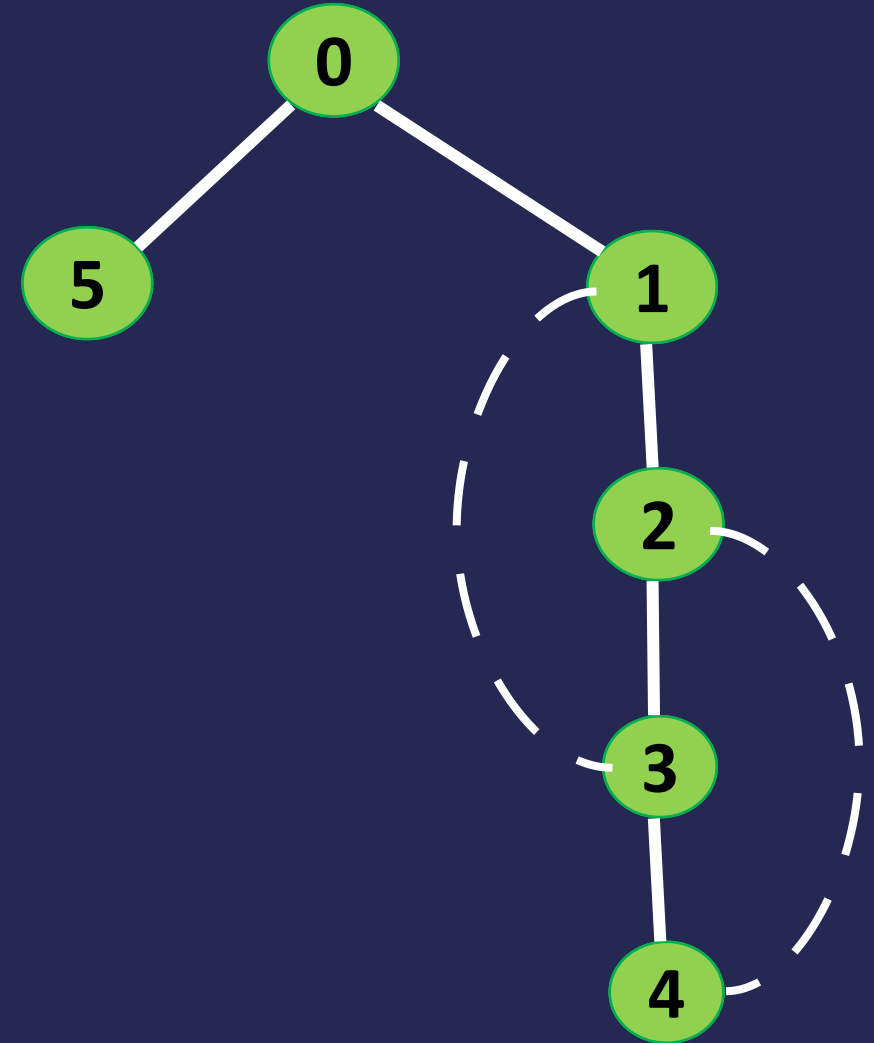
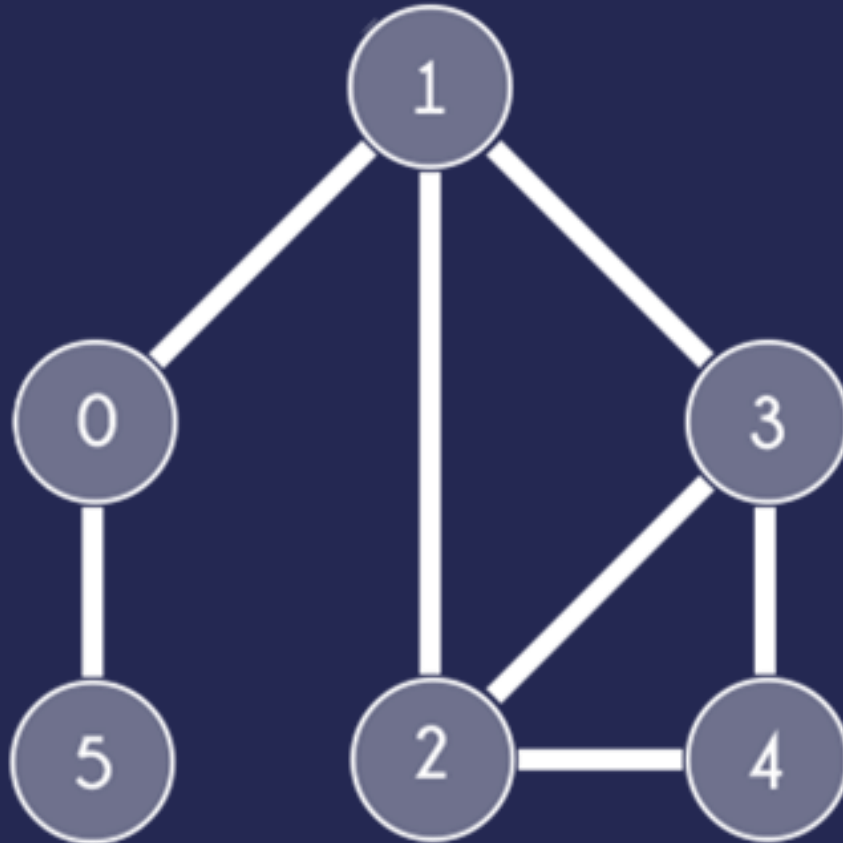
Example: Find an Articulation point for the given graph.





# Articulation Point

Step 1: DFS Tree

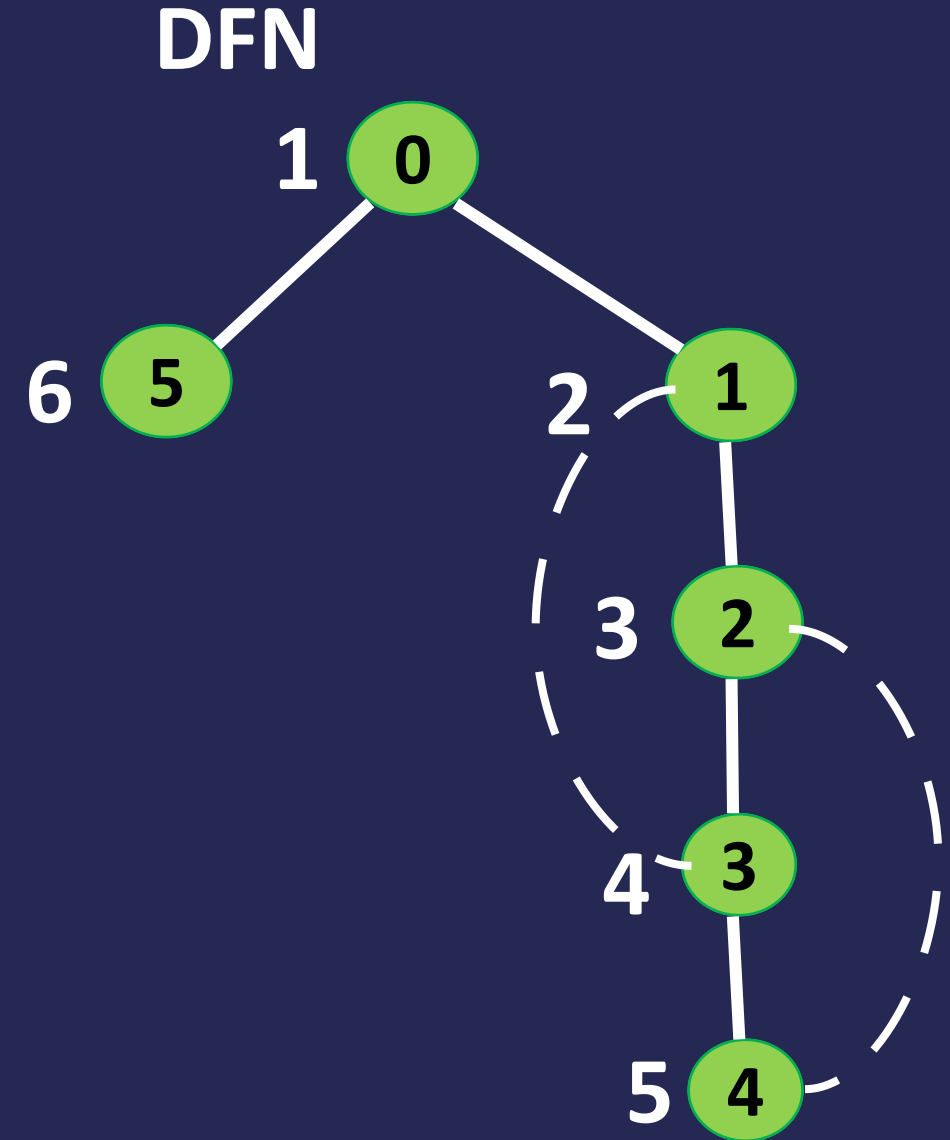




# Articulation Point

Step 2:

Assign dfn number





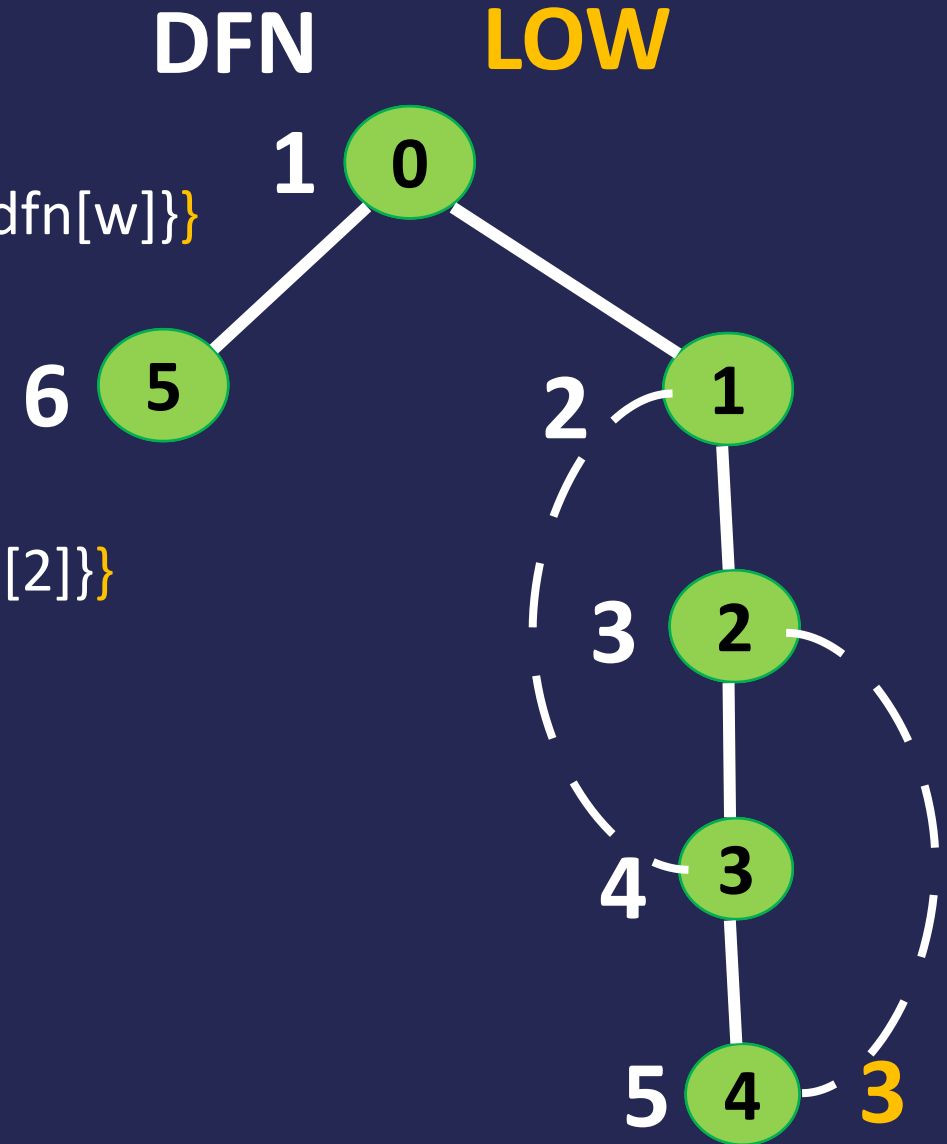
# Articulation Point

Step 3: Find low

$$\text{low}[u] = \min\{\text{dfn}[u], \min\{\text{low}[w]\}, \min\{\text{dfn}[w]\}\}$$

For node 4

$$\begin{aligned}\text{low}[4] &= \min\{\text{dfn}[4], -, \min\{\text{dfn}[2]\}\} \\ &= \min\{5, -, 3\} \\ &= 3\end{aligned}$$





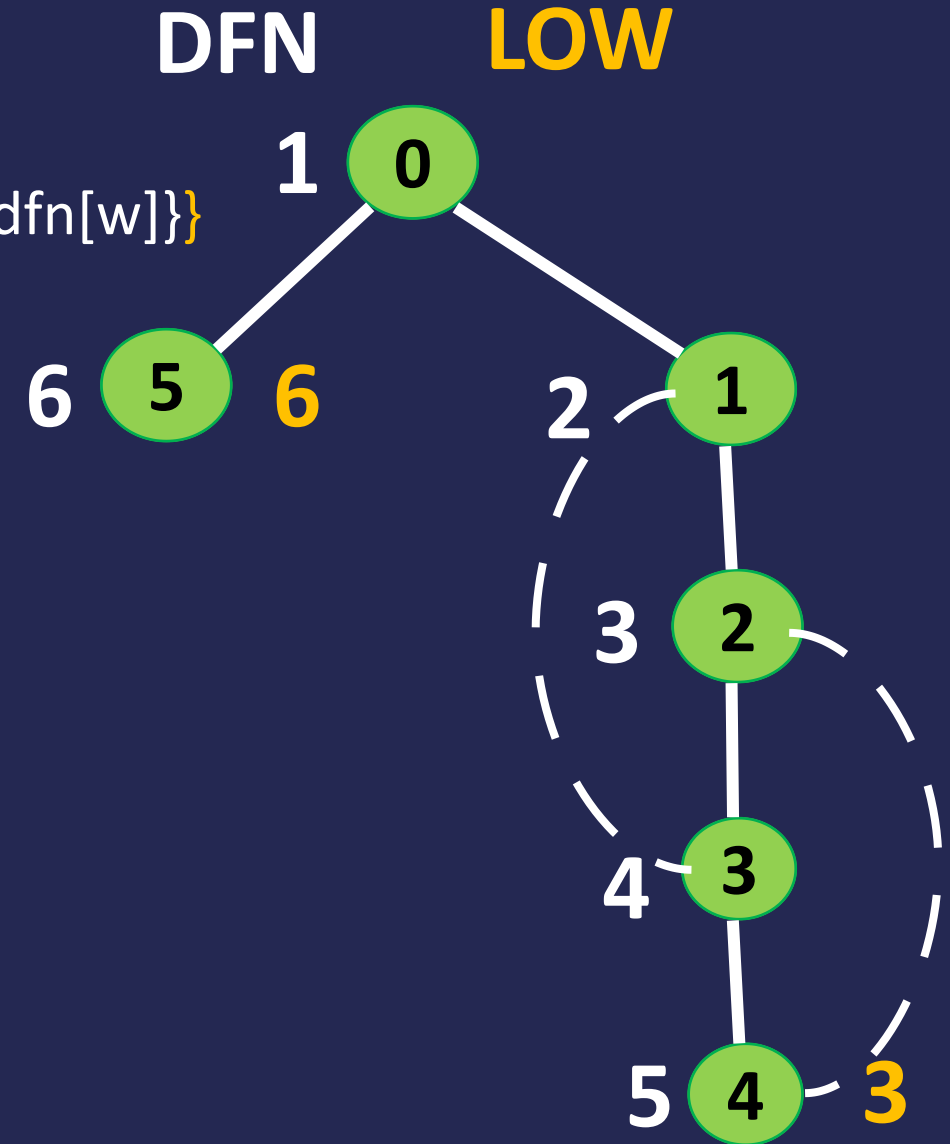
# Articulation Point

Step 3: Find low

$$\text{low}[u] = \min\{\text{dfn}[u], \min\{\text{low}[w]\}, \min\{\text{dfn}[w]\}\}$$

For node 5

$$\begin{aligned}\text{low}[5] &= \min\{\text{dfn}[5], -, -\} \\ &= \min\{6, -, -\} \\ &= 6\end{aligned}$$





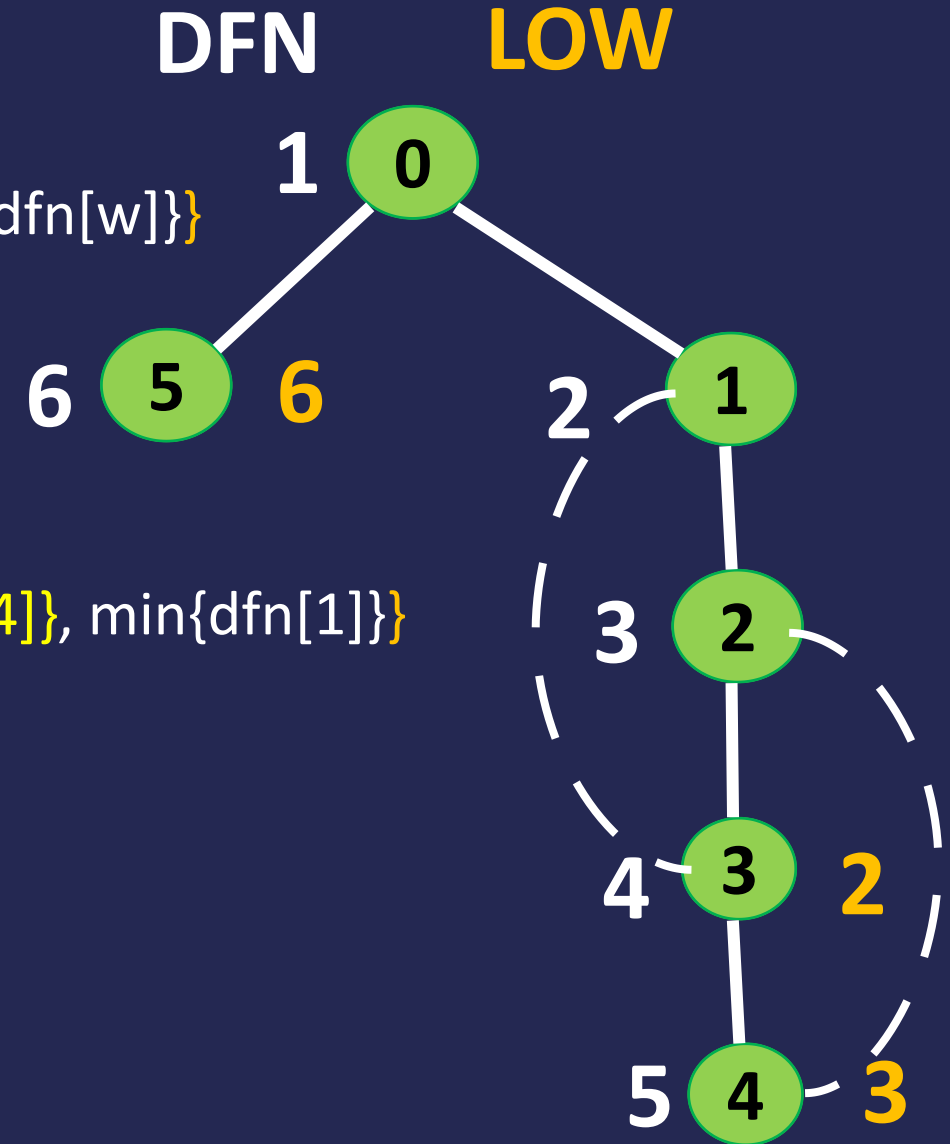
# Articulation Point

Step 3: Find low

$$\text{low}[u] = \min\{\text{dfn}[u], \min\{\text{low}[w]\}, \min\{\text{dfn}[w]\}\}$$

For node 3

$$\begin{aligned}\text{low}[3] &= \min\{\text{dfn}[3], \min\{\text{low}[4]\}, \min\{\text{dfn}[1]\}\} \\ &= \min\{4, 3, 2\} \\ &= 2\end{aligned}$$





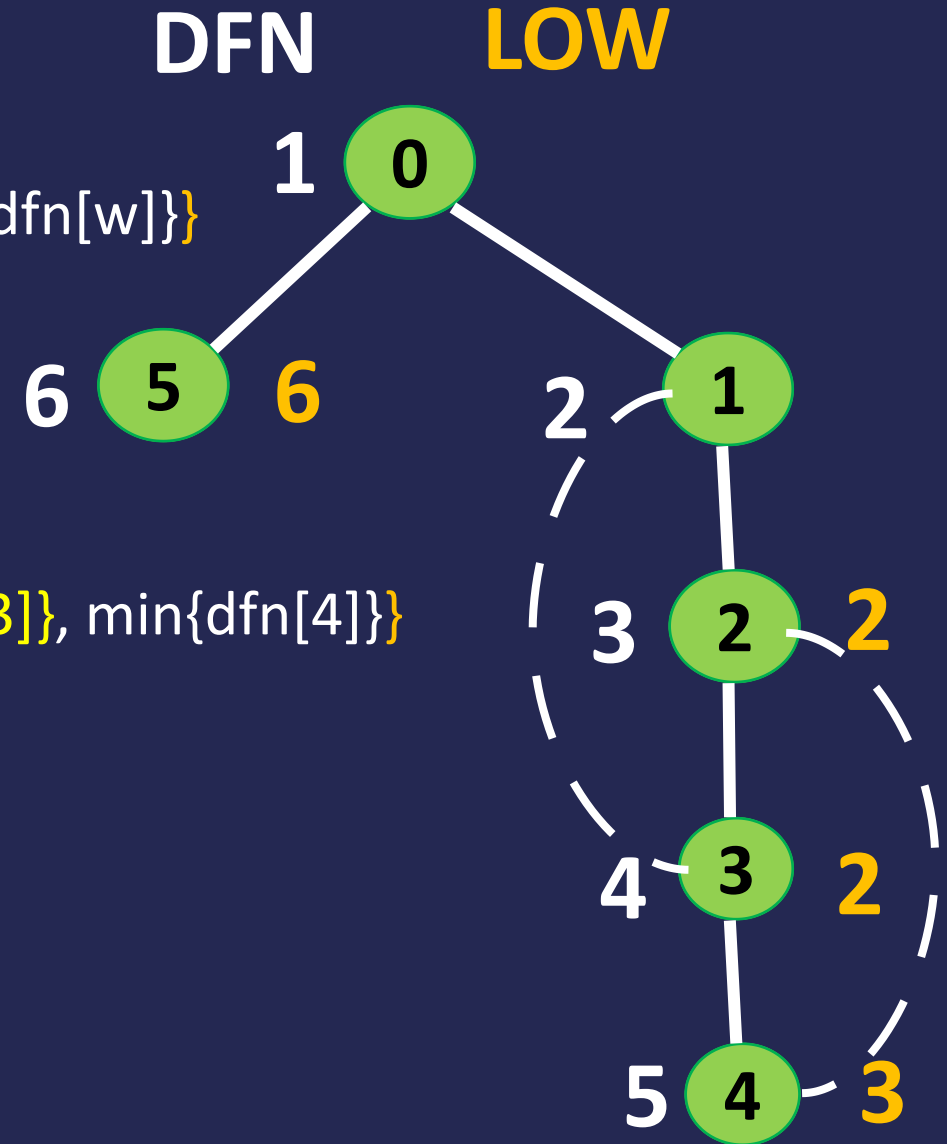
# Articulation Point

Step 3: Find low

$$\text{low}[u] = \min\{\text{dfn}[u], \min\{\text{low}[w]\}, \min\{\text{dfn}[w]\}\}$$

For node 2

$$\begin{aligned}\text{low}[2] &= \min\{\text{dfn}[2], \min\{\text{low}[3]\}, \min\{\text{dfn}[4]\}\} \\ &= \min\{3, 2, 5\} \\ &= 2\end{aligned}$$







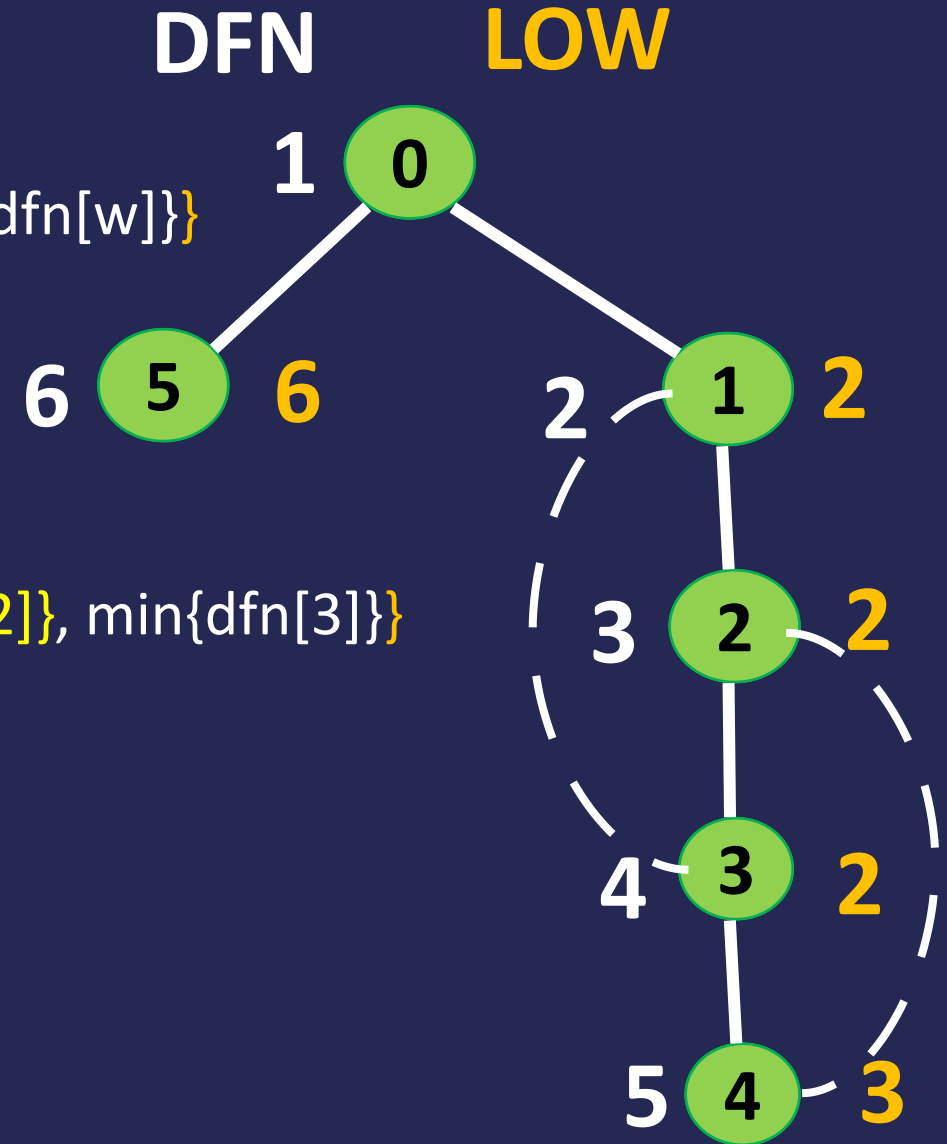
# Articulation Point

Step 3: Find low

$$\text{low}[u] = \min\{\text{dfn}[u], \min\{\text{low}[w]\}, \min\{\text{dfn}[w]\}\}$$

For node 1

$$\begin{aligned}\text{low}[1] &= \min\{\text{dfn}[1], \min\{\text{low}[2]\}, \min\{\text{dfn}[3]\}\} \\ &= \min\{2, 2, 4\} \\ &= 2\end{aligned}$$





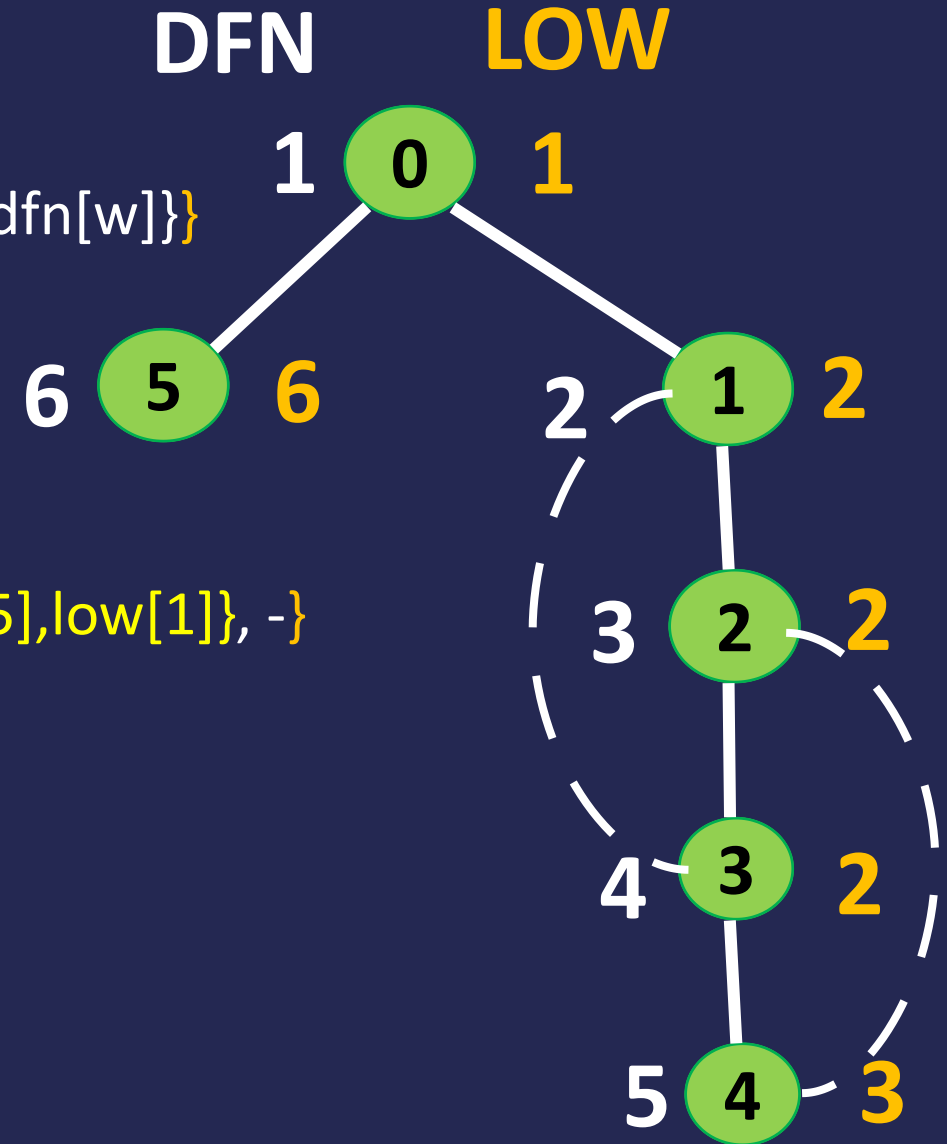
# Articulation Point

Step 3: Find low

$$\text{low}[u] = \min\{\text{dfn}[u], \min\{\text{low}[w]\}, \min\{\text{dfn}[w]\}\}$$

For node 0

$$\begin{aligned}\text{low}[0] &= \min\{\text{dfn}[0], \min\{\text{low}[5], \text{low}[1]\}, -\} \\ &= \min\{1, \min\{6, 2\}, -\} \\ &= 1\end{aligned}$$





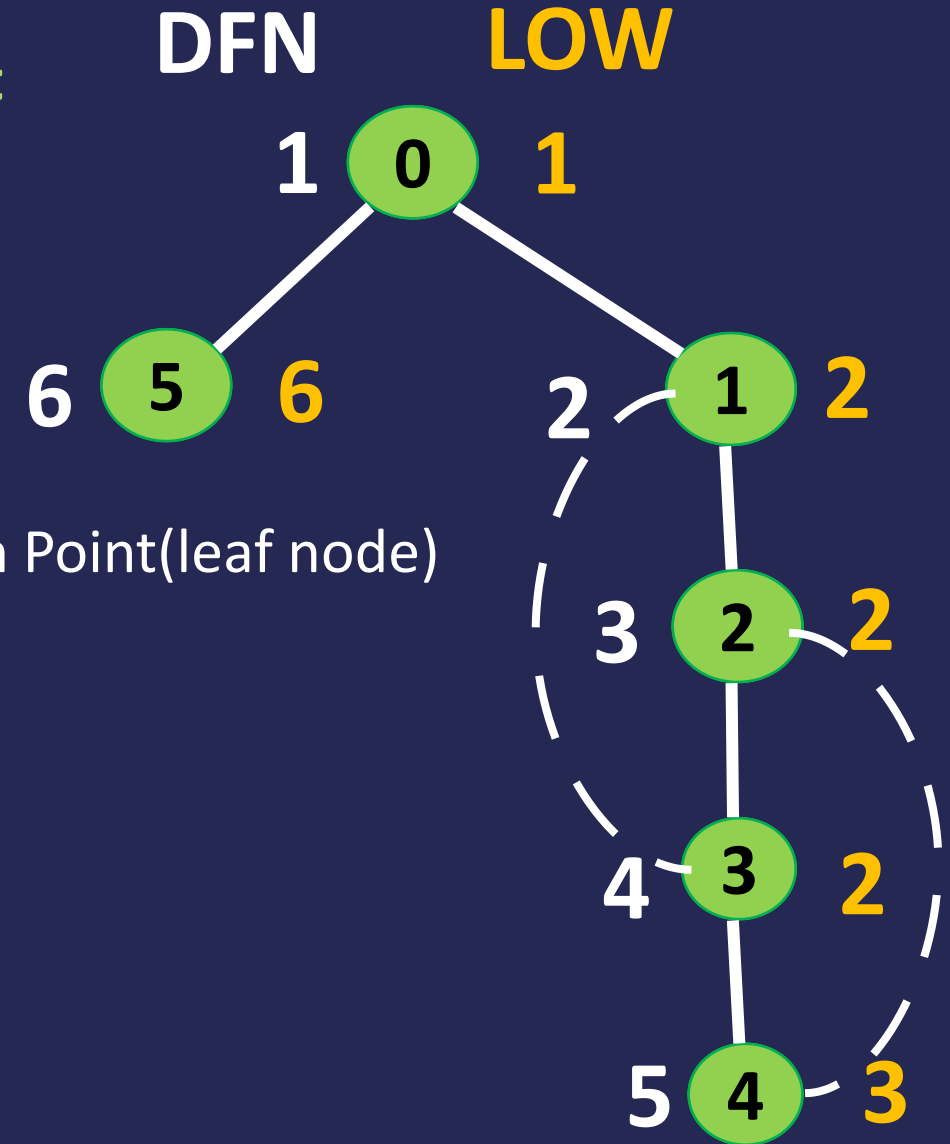
# Articulation Point

## Step 4: Find Articulation Point

Vertex  $u$  is an articulation point if  $w$  is child of  $u$  such that

$$\text{low}[w] \geq \text{dfn}[u]$$

Node 5 and Node 4 are not an Articulation Point(leaf node)  
So check for Node 0, 1, 2 and 3.





# Articulation Point

## Step 4: Find Articulation Point

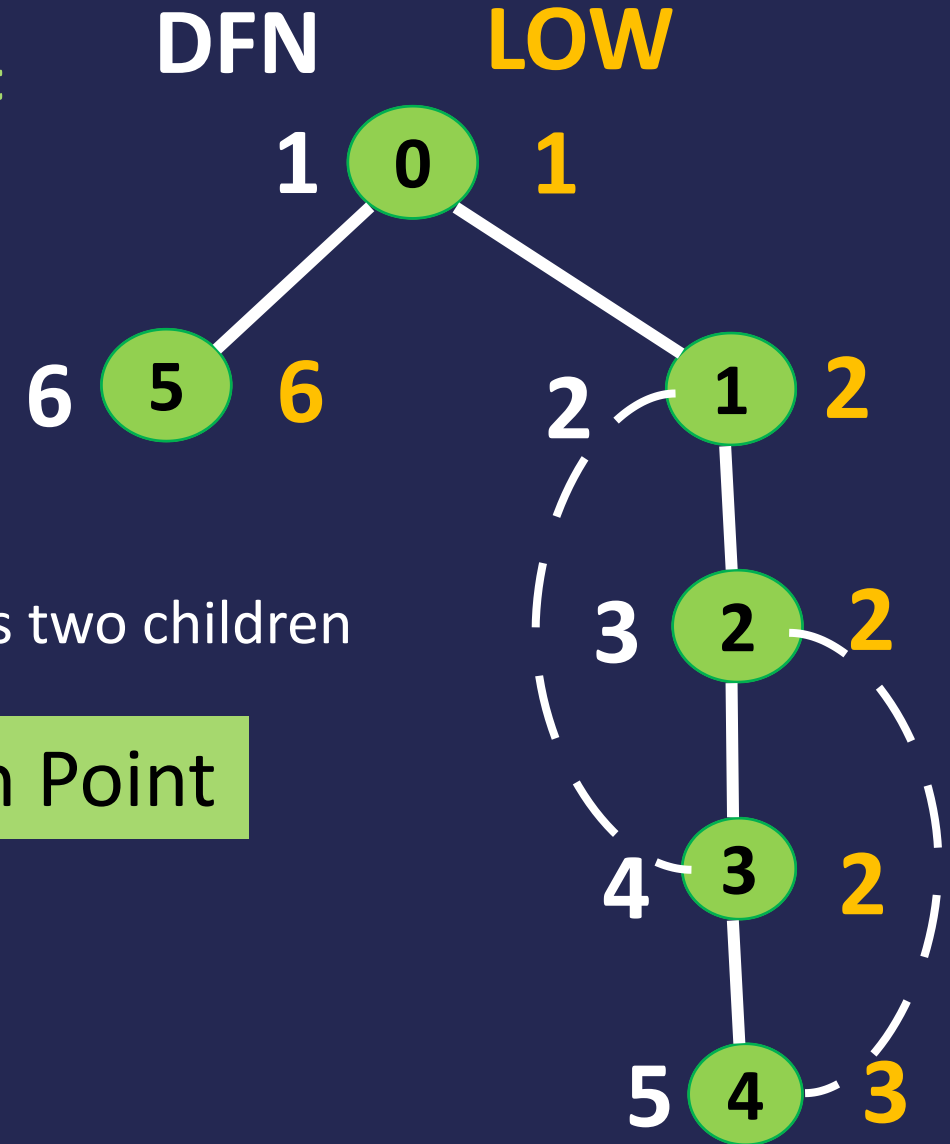
Vertex  $u$  is an articulation point if  $w$  is child of  $u$  such that

$$\text{low}[w] \geq \text{dfn}[u]$$

For Node 0

Node 0 is root of DFS tree and it has two children

Node 0 is an Articulation Point





# Articulation Point

## Step 4: Find Articulation Point

Vertex  $u$  is an articulation point if  $w$  is child of  $u$  such that

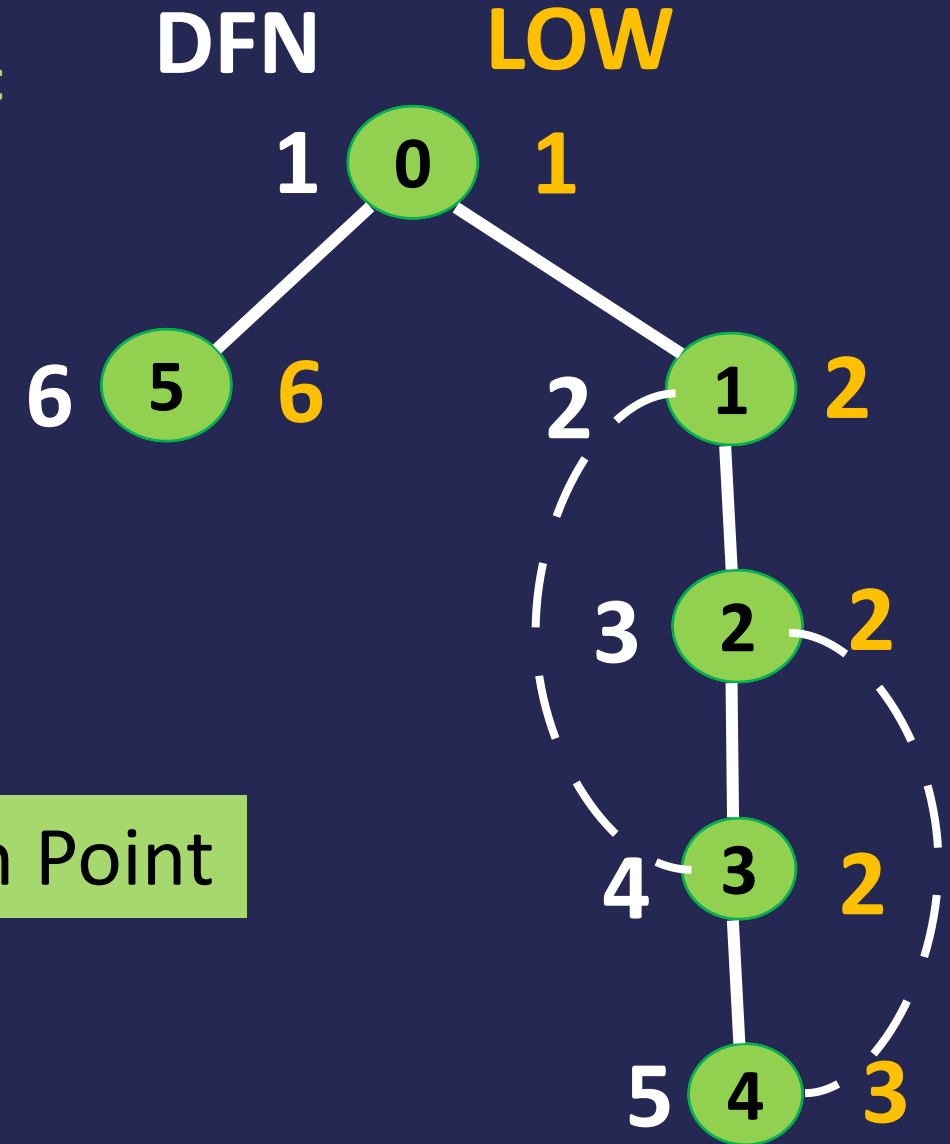
$$\text{low}[w] \geq \text{dfn}[u]$$

For Node 1

$$\text{Low}[2] \geq \text{dfn}[1]$$

$$2 \geq 2 \text{ YES}$$

Node 1 is an Articulation Point





# Articulation Point

## Step 4: Find Articulation Point

Vertex  $u$  is an articulation point if  $w$  is child of  $u$  such that

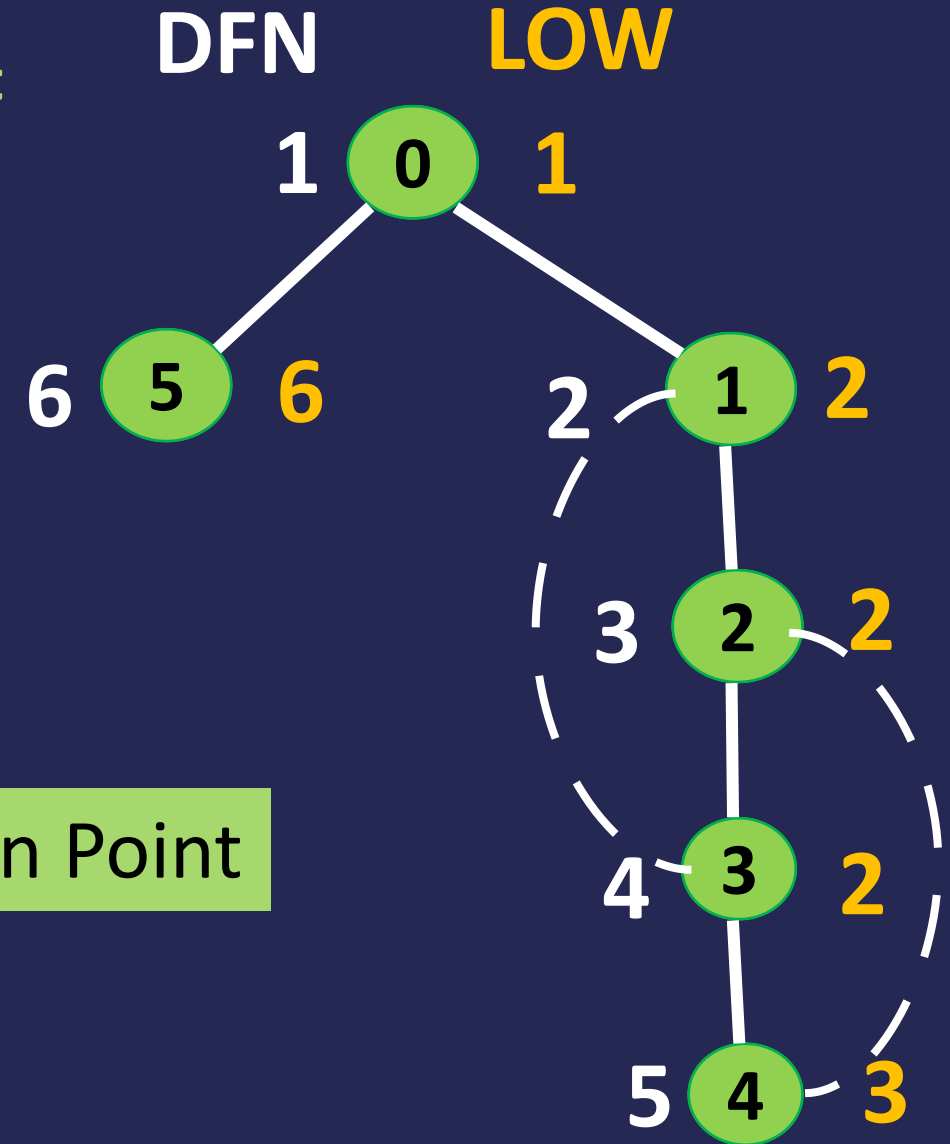
$$\text{low}[w] \geq \text{dfn}[u]$$

For Node 2

$$\text{Low}[3] \geq \text{dfn}[2]$$

$$2 \geq 3 \quad \text{NO}$$

Node 2 is not an Articulation Point





# Articulation Point

## Step 4: Find Articulation Point

Vertex  $u$  is an articulation point if  $w$  is child of  $u$  such that

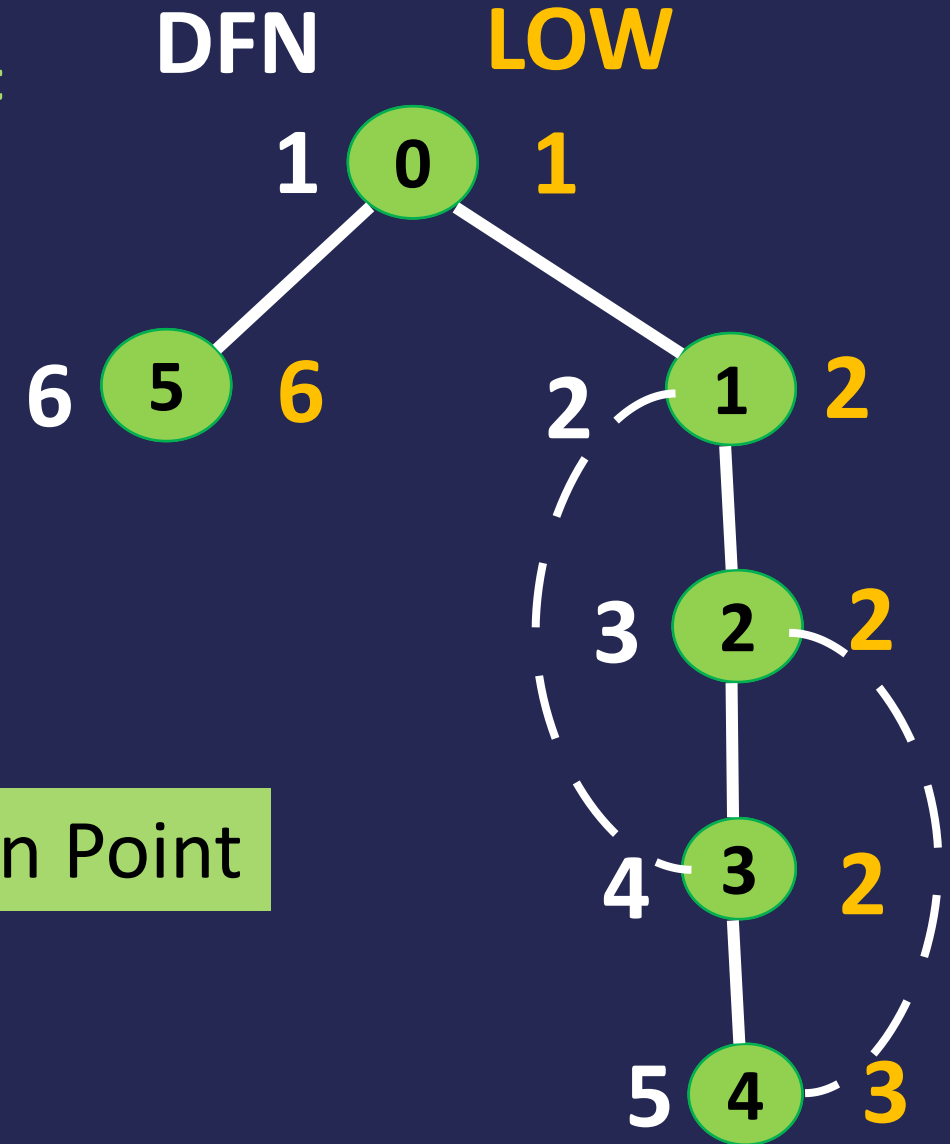
$$\text{low}[w] \geq \text{dfn}[u]$$

For Node 3

$$\text{Low}[4] \geq \text{dfn}[3]$$

$$3 \geq 4 \quad \text{NO}$$

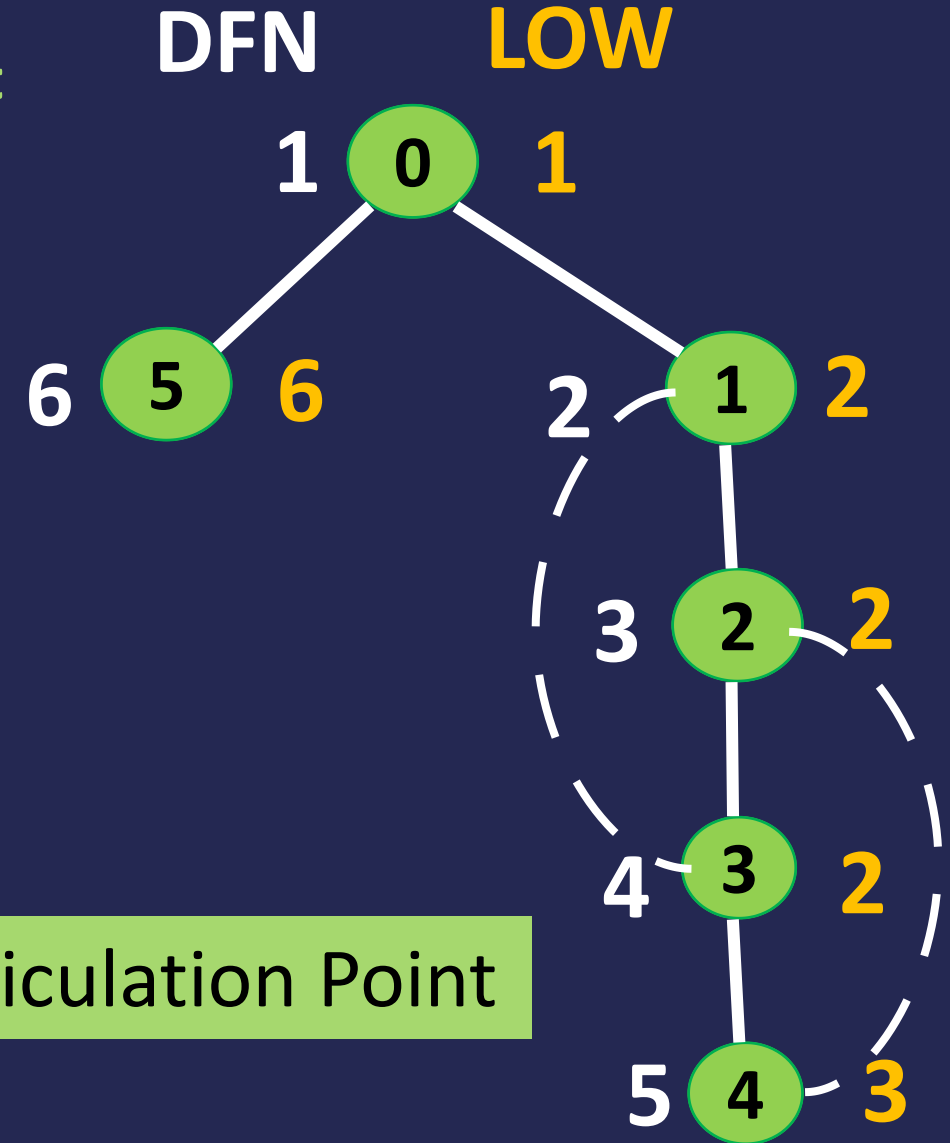
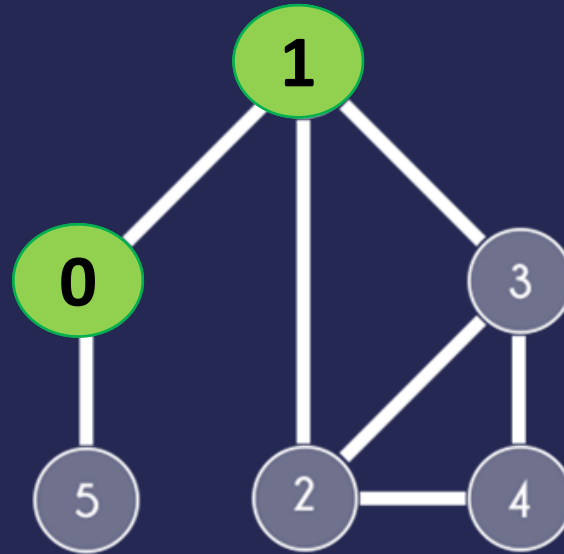
Node 3 is not an Articulation Point





# Articulation Point

Step 4: Find Articulation Point



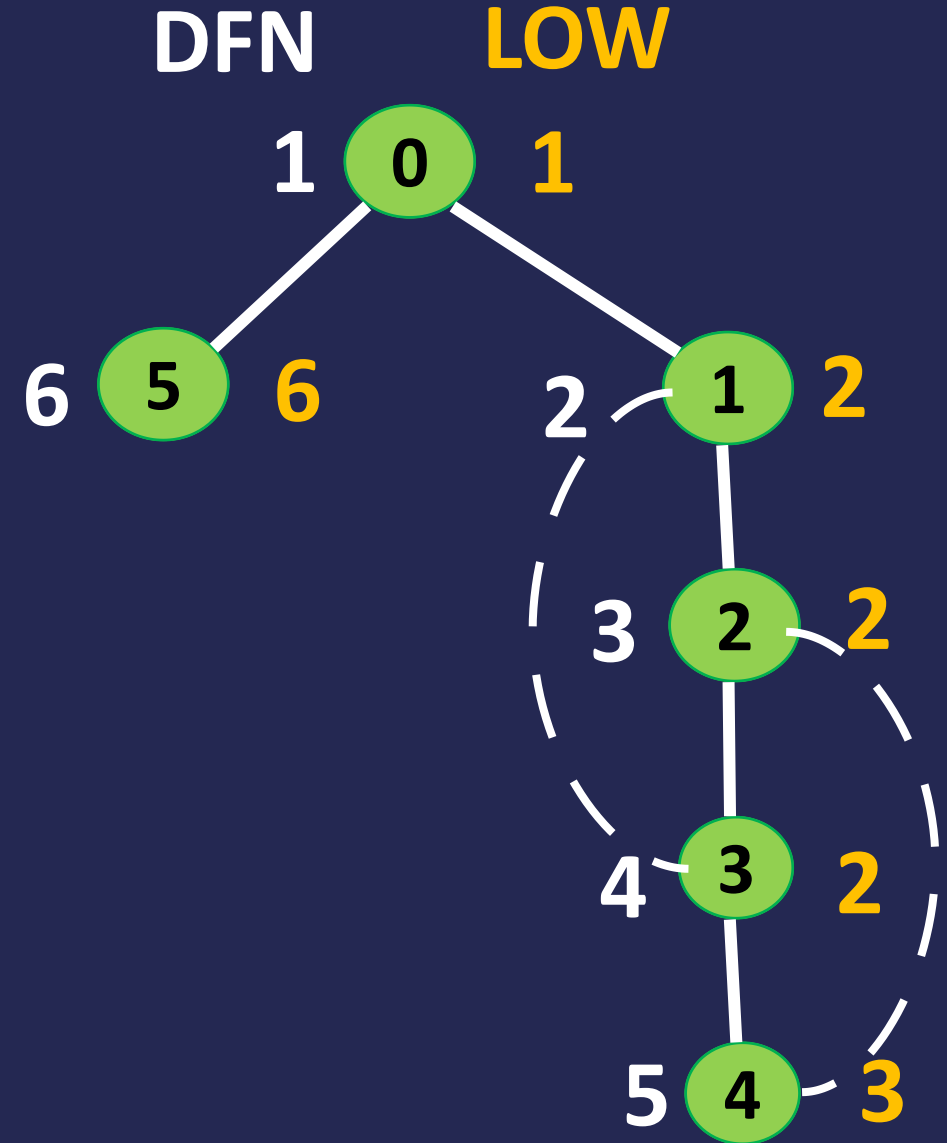
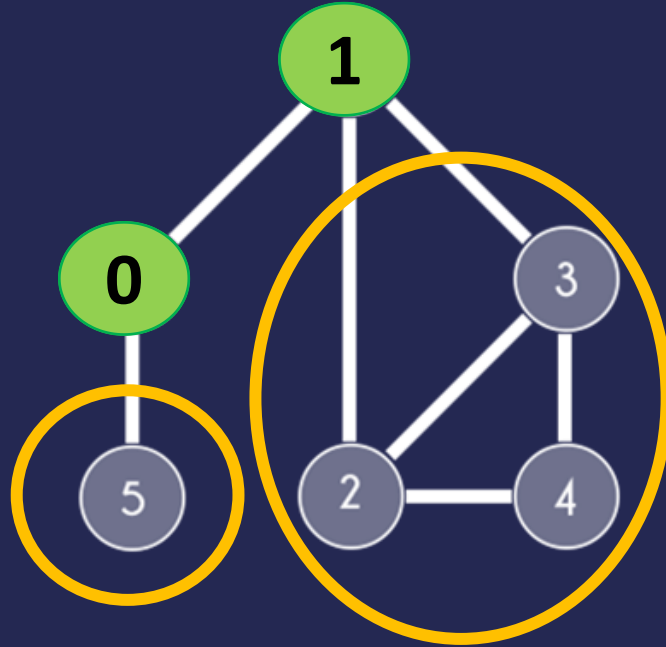
Node 0 and Node 1 are an Articulation Point





# Articulation Point

Bi connected components:

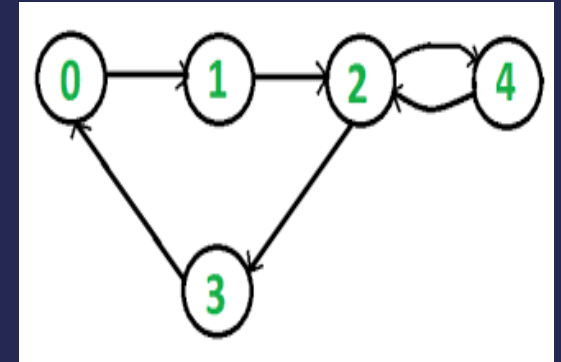




# Strongly Connected components

## Strongly connected components:

A directed graph is called strongly connected if for every pair of vertices  $u$  and  $v$  there is a path from  $u$  to  $v$  and a path from  $v$  to  $u$ .



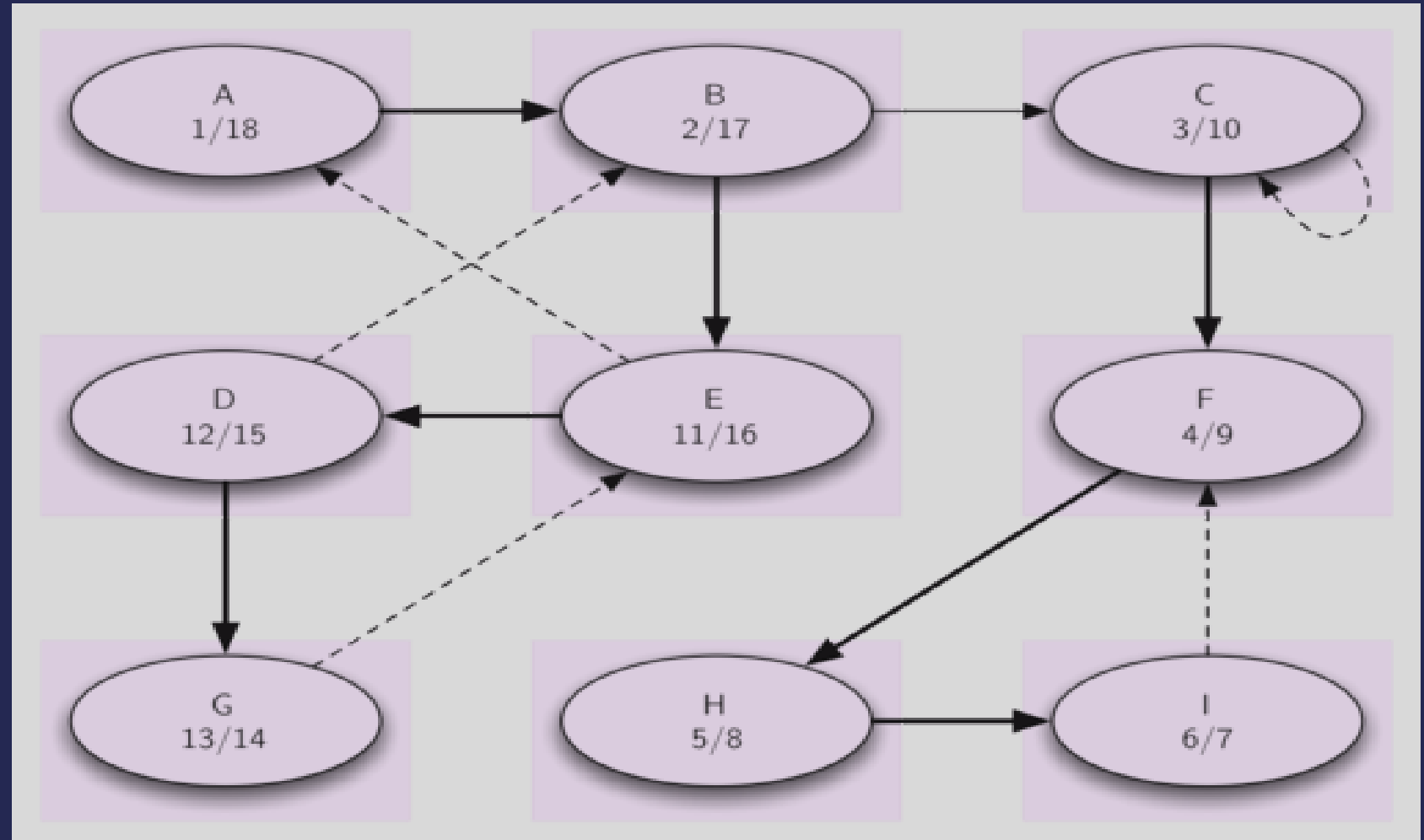
Algorithm to compute the strongly connected components for a graph.

1. Call dfs for the graph  $G$  to compute the finish times for each vertex.
2. **Compute  $G^T$ .**
3. Call dfs for the graph  $G^T$  but in the main loop of DFS explore each vertex in decreasing order of finish time.
4. **Each tree in the forest computed in step 3 is a strongly connected component.**



# Strongly Connected components

**Step-1** : Call dfs for the graph G to compute the finish times for each vertex.

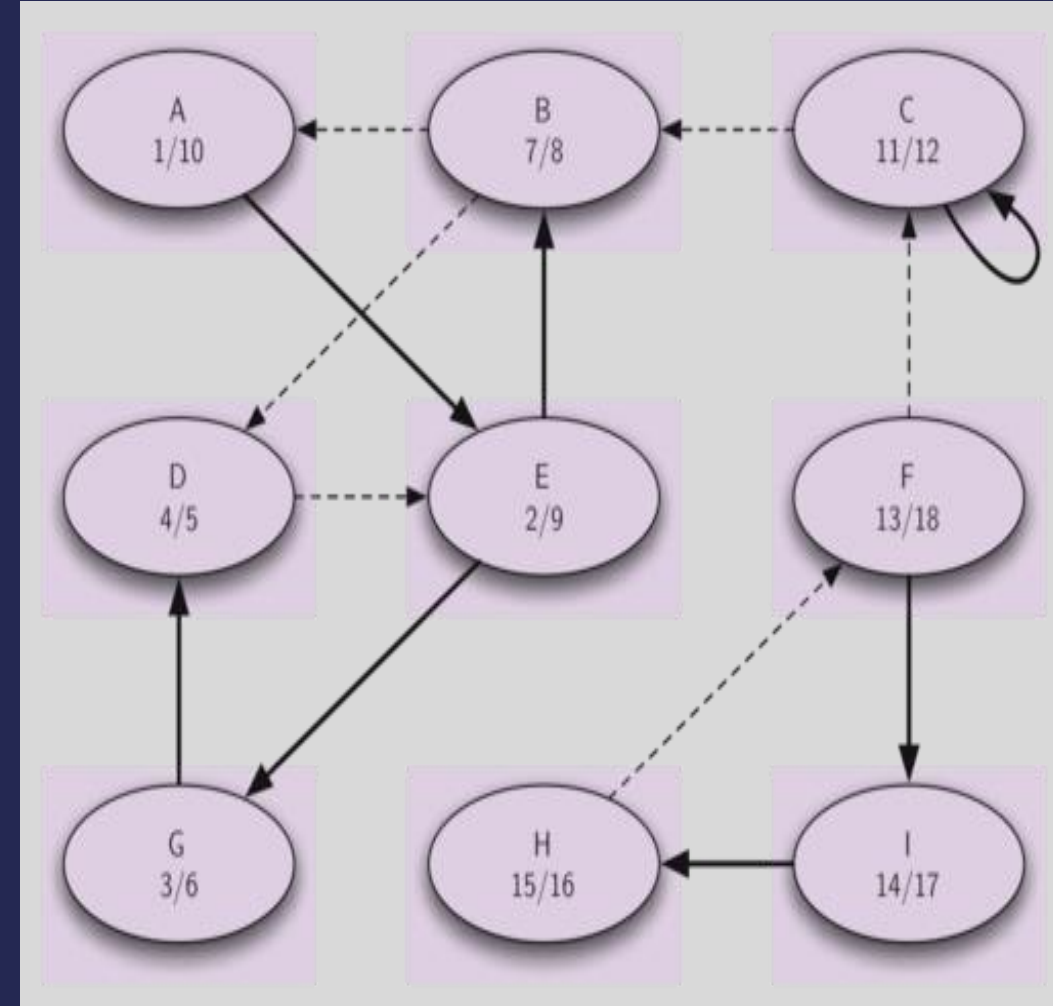




The graph consists of 9 nodes, each represented by a light purple oval with a black border. The nodes are arranged in a 3x3 grid. Each node contains a letter and a fraction. The edges are as follows:

- Solid arrows: A → B, B → C, B → E, E → D, D → G, F → H, H → I.
- Dashed arrows: A → E, D → B, G → E, I → F.
- Self-loop: C → C.

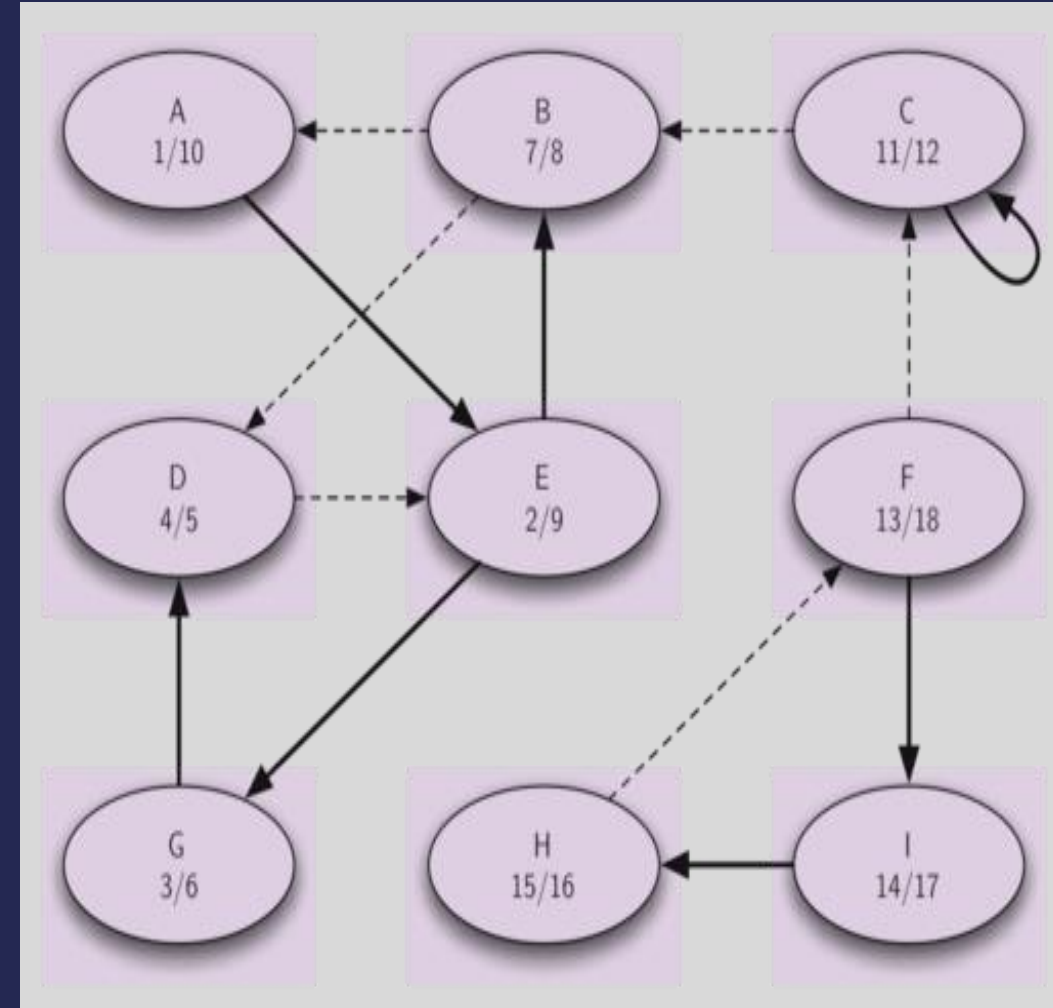
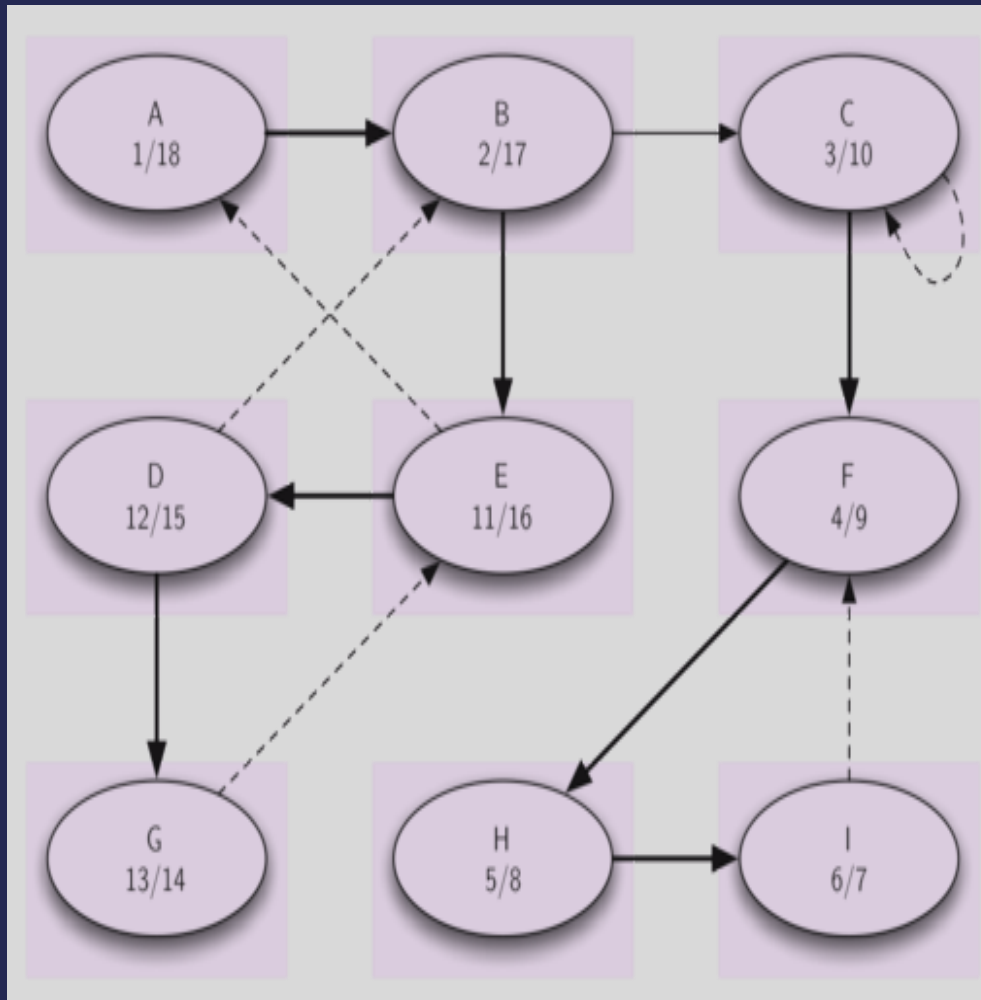
```
graph LR; A((A  
1/18)) --> B((B  
2/17)); B --> C((C  
3/10)); B --> E((E  
11/16)); C --> C; C --> F((F  
4/9)); D((D  
12/15)) --> E; E --> D; E --> G((G  
13/14)); F --> H((H  
5/8)); H --> I((I  
6/7)); I -.-> F; A -.-> E; D -.-> B; G -.-> E;
```





# Strongly Connected components

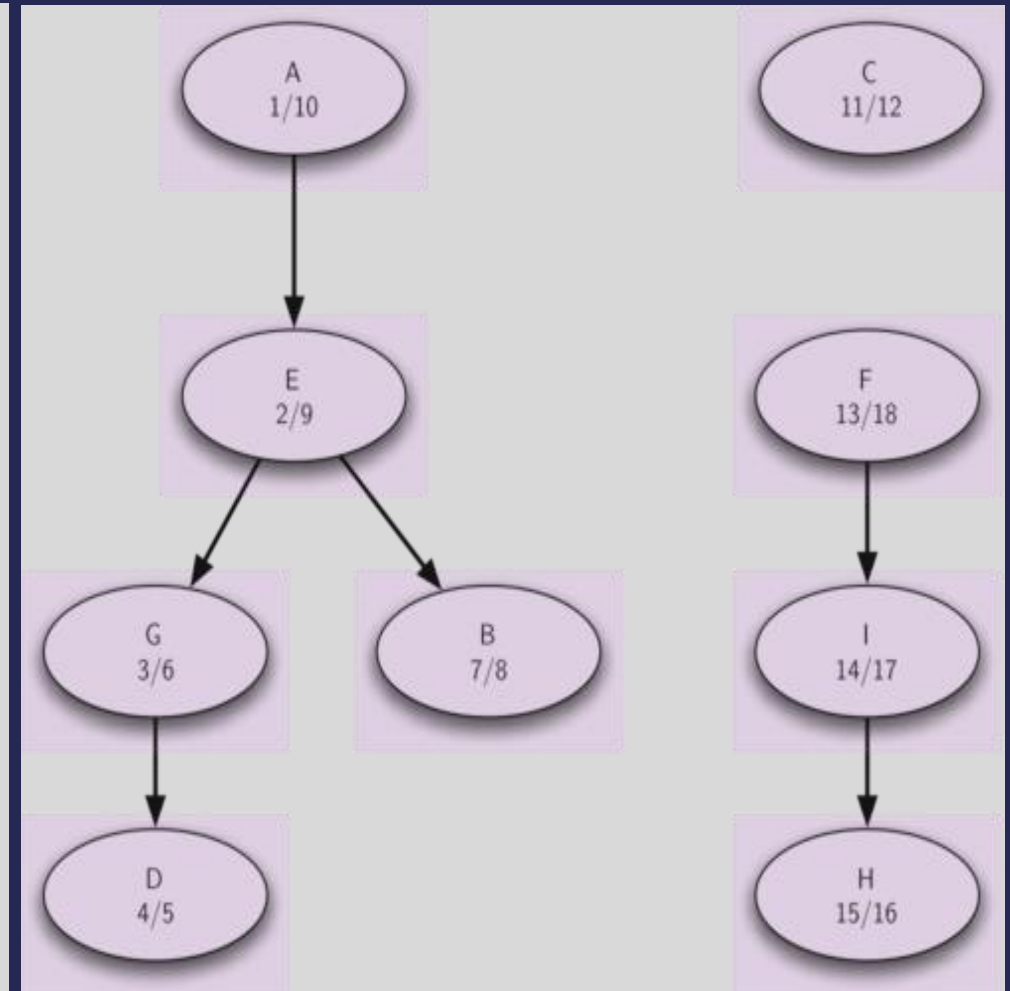
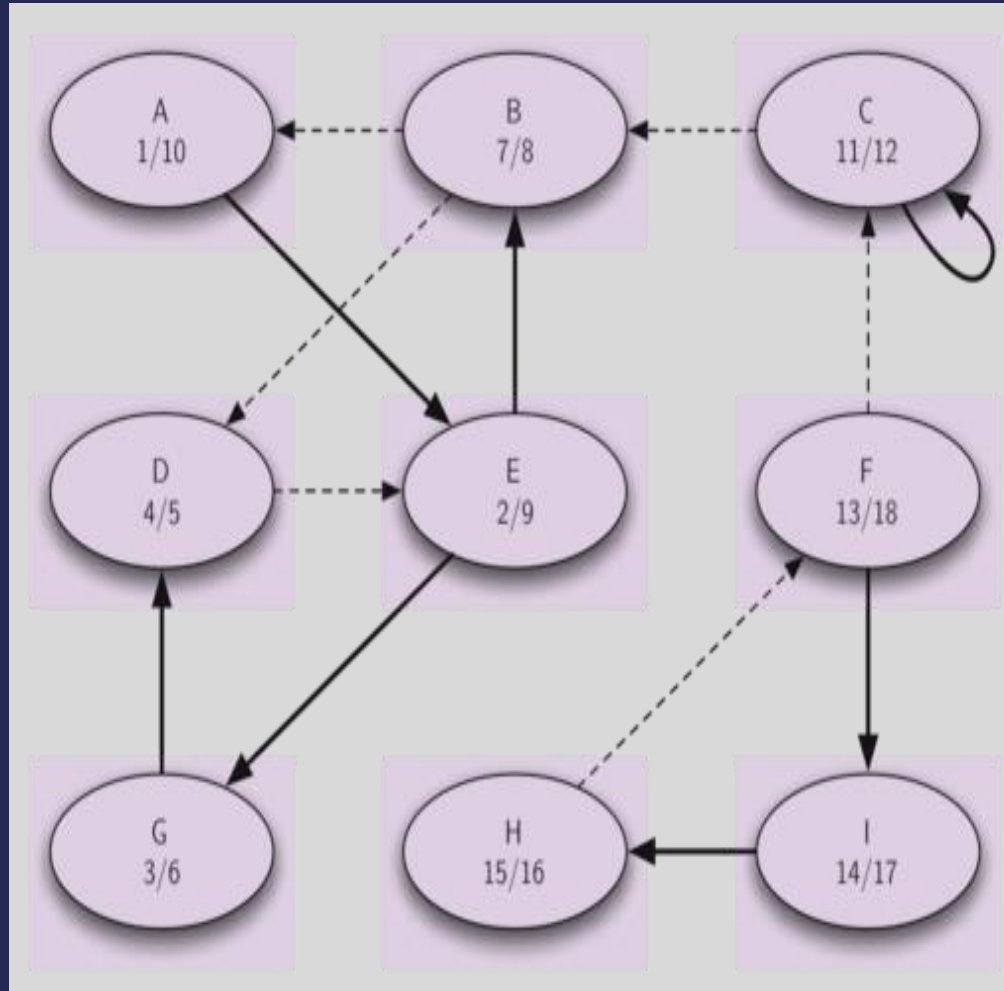
**Step-3 :** Call dfs for the graph  $G^T$  explore each vertex in decreasing order of finish time.





# Strongly Connected components

**Step-4 : Each tree in the forest computed in step 3 is a strongly connected component.**

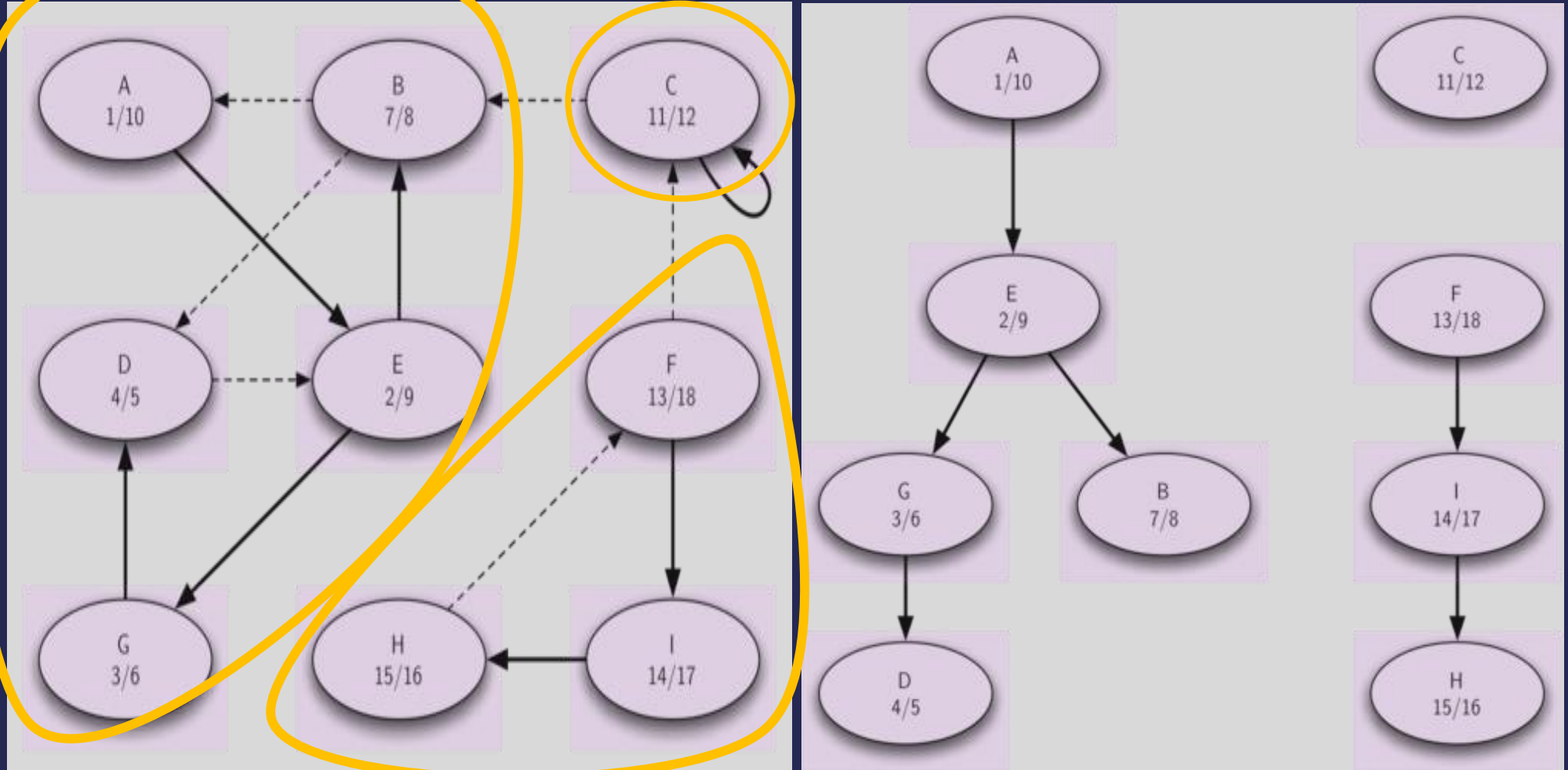






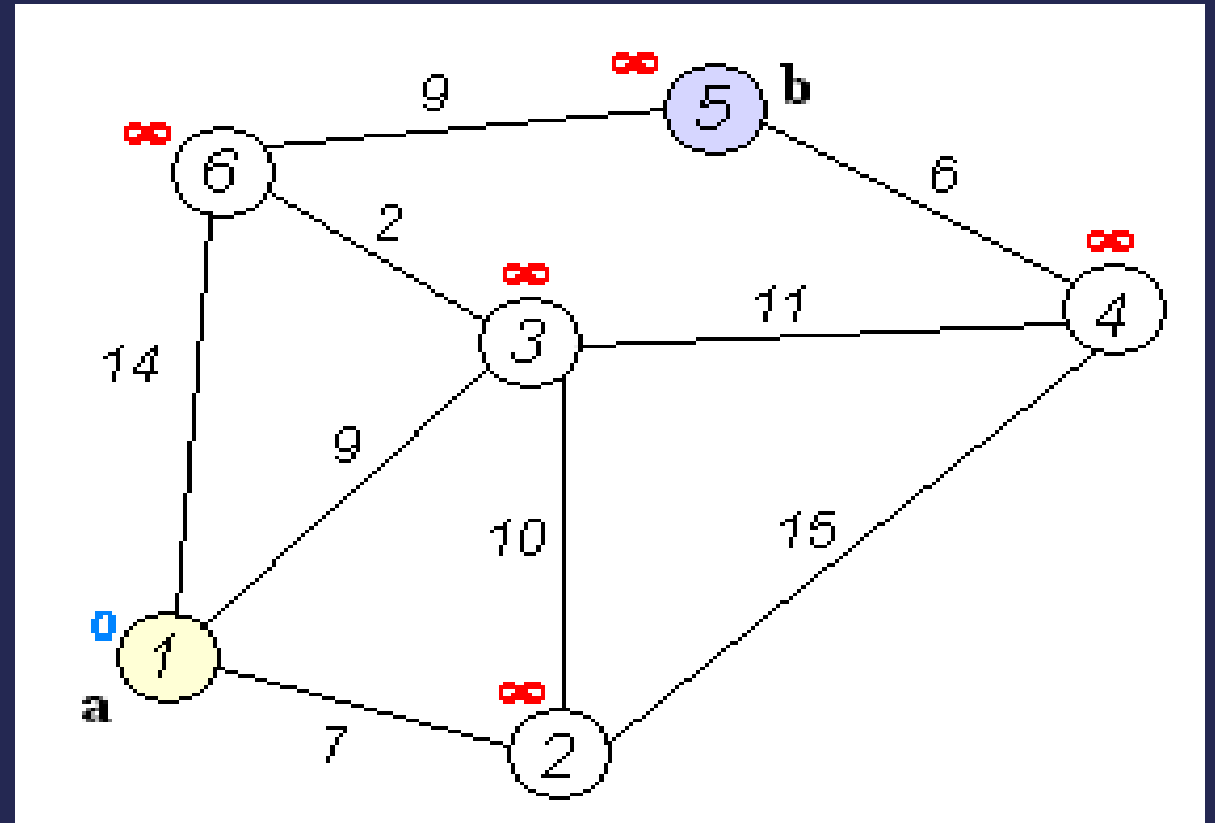
# Strongly Connected components

Step-4 : Each tree in the forest computed in step 3 is a strongly connected component.





# DIJKSTRA ALGORITHM



Assistant Professor

IT-ICT Department, LJIET

[prayag.patel@ljinstitutes.edu.in](mailto:prayag.patel@ljinstitutes.edu.in)

Mr. Prayag Patel

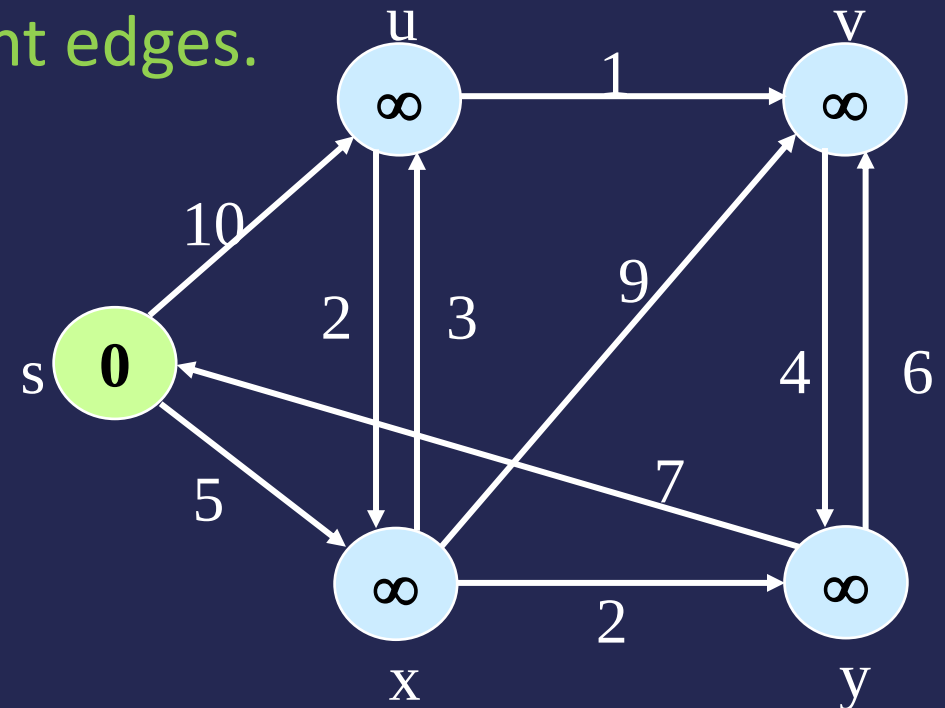




# Dijkstra Algorithm

## ■ Problem Definition

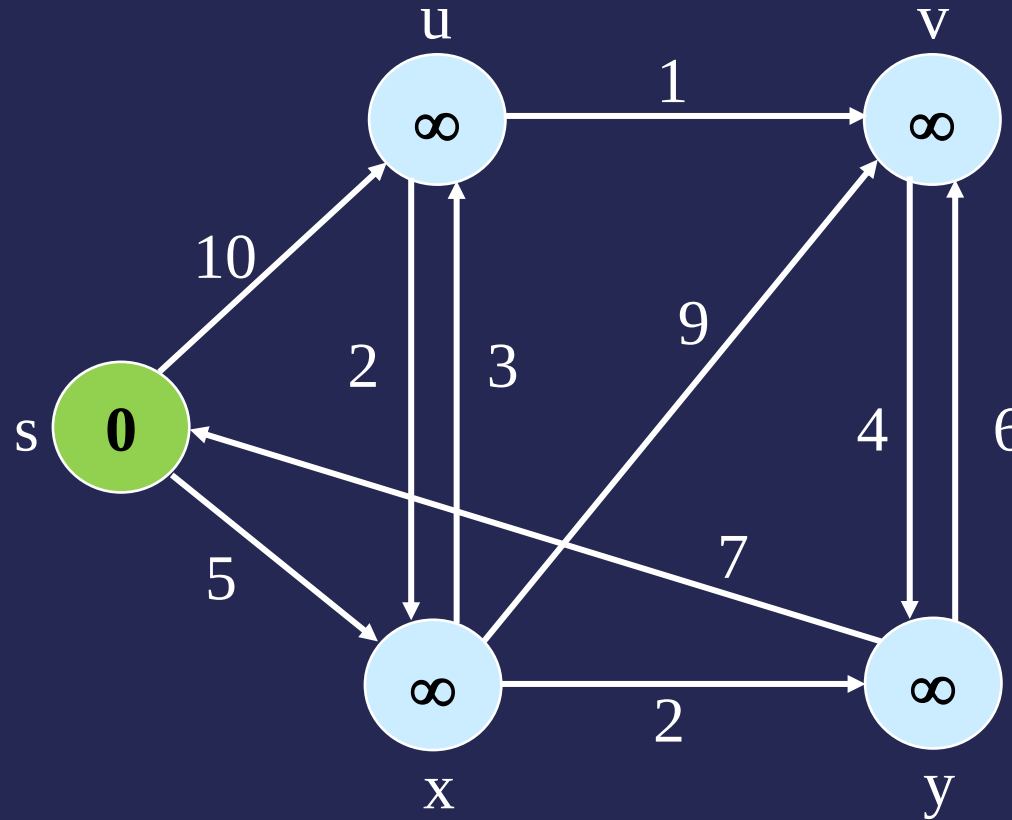
- Dijkstra algorithm used for find Single source shortest path.
- The problem of finding shortest paths from a source vertex  $s$  to all other vertices in the graph.
- Assumes **no negative-weight edges**.





# Dijkstra Algorithm

*Step-1: Node s consider as source node.*



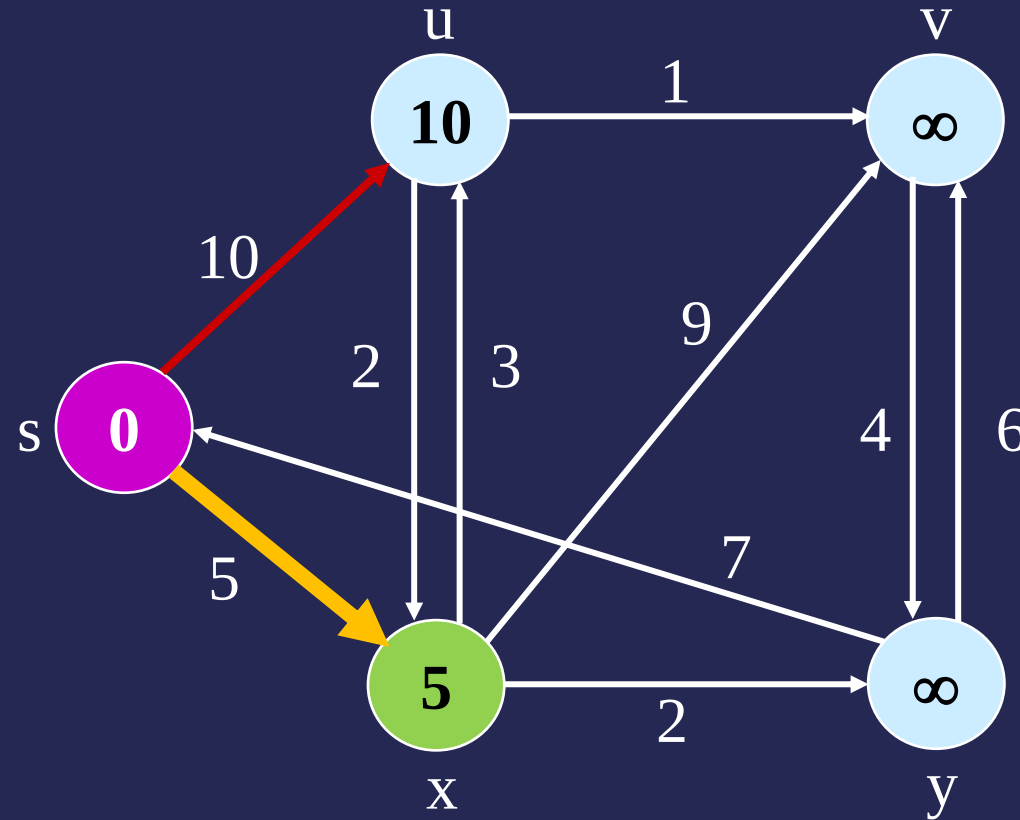
Distance(source) = 0

Distance (all vertices) =  $\infty$



# Dijkstra Algorithm

Step-2: Pick vertex with minimum distance.



Distance(u) = 10

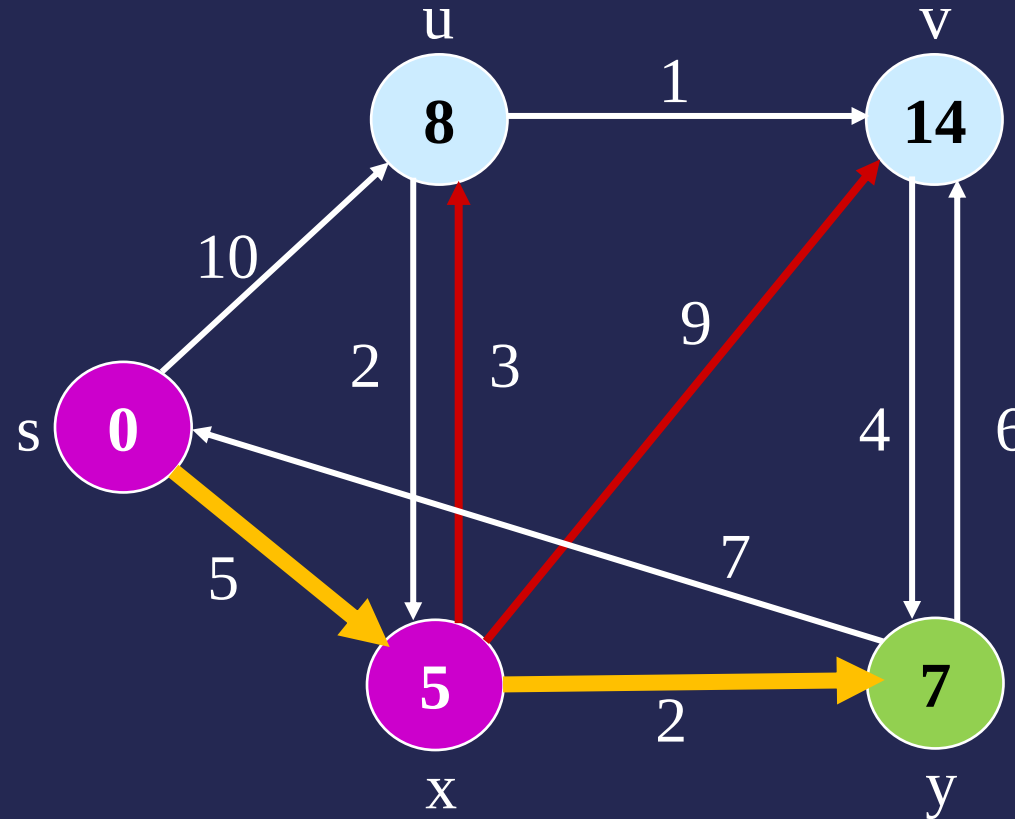
Distance (x) = 5

**Select node x**



# Dijkstra Algorithm

Step-3: Pick vertex with minimum distance.



Distance(u) = 8

Distance (v) = 14

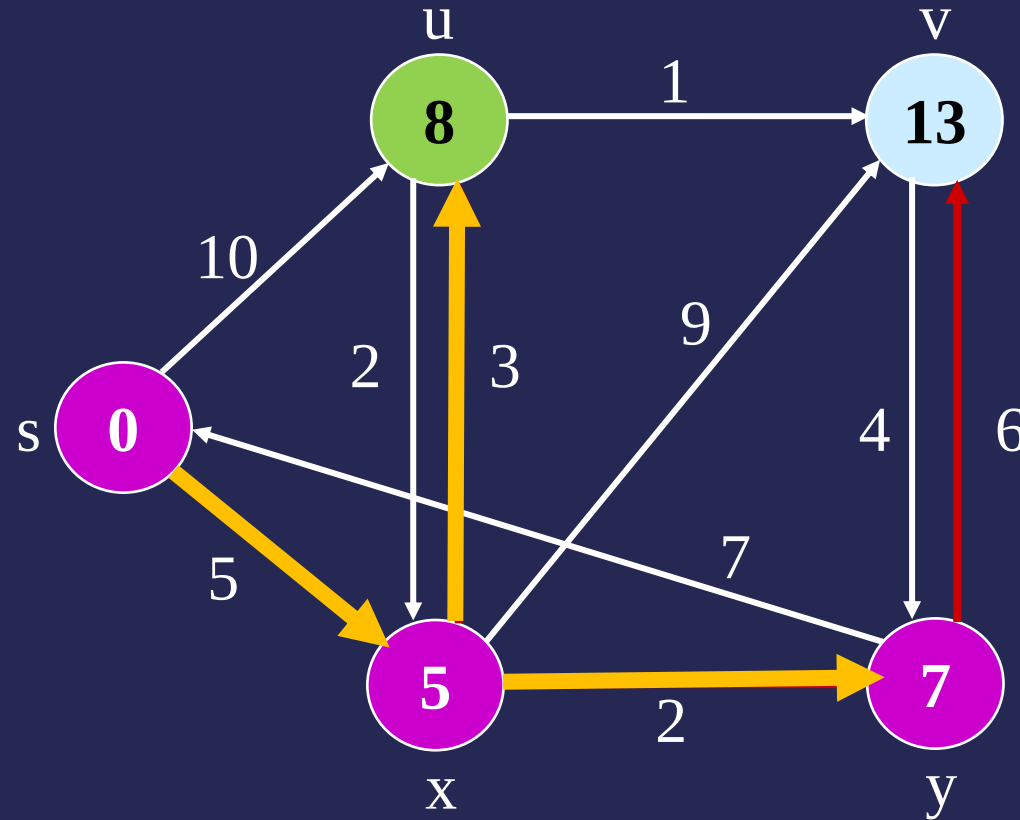
Distance (y) = 7

**Select node y**



# Dijkstra Algorithm

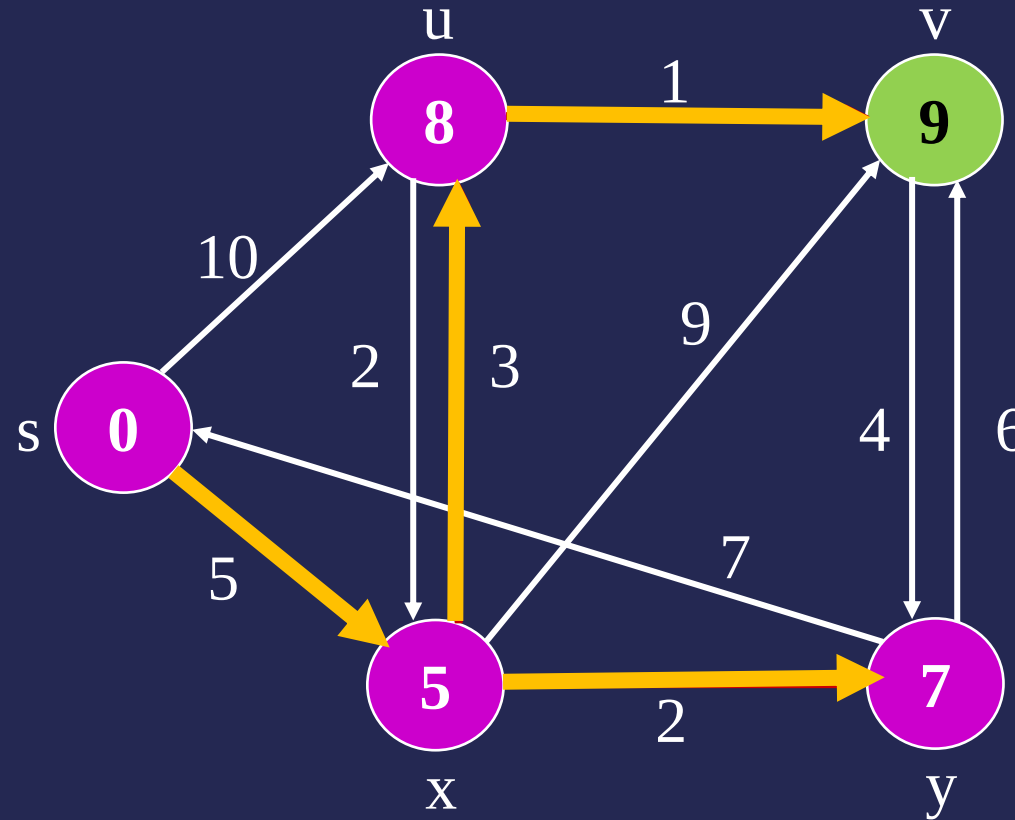
Step-4: Pick vertex with minimum distance.





# Dijkstra Algorithm

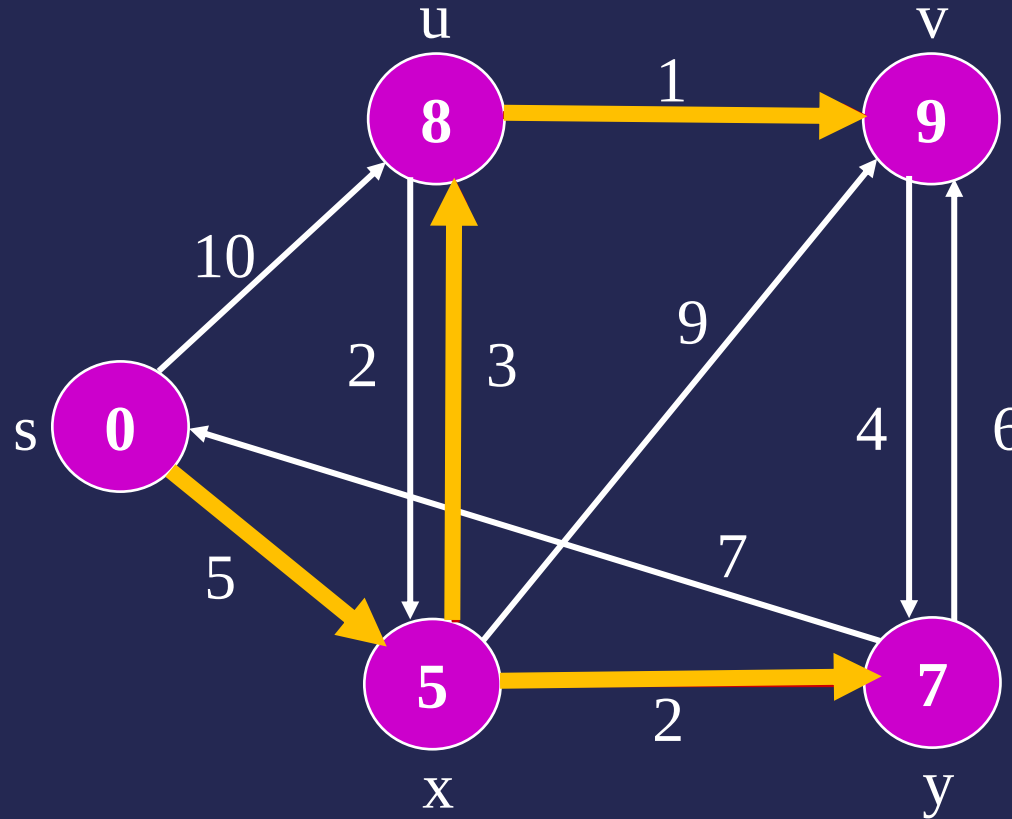
Step-5: Pick vertex with minimum distance.





# Dijkstra Algorithm

Shortest path from node s.



| Start | End | Distance |
|-------|-----|----------|
| s     | x   | 5        |
| s     | u   | 8        |
| s     | y   | 7        |
| s     | v   | 9        |



# Dijkstra Algorithm

## ■ Algorithm

- Maintains a set **S** of vertices whose SP from s (Starting node) has been determined.
- Repeatedly selects u in V-S with minimum SP (Shortest path).
- Store V – S in **priority queue Q**.

**Dijkstra(G, w, s)**

1. Initialize\_Single\_source(G, s);

2.  $S = \emptyset$ ;

3.  $Q = G.V$ ;

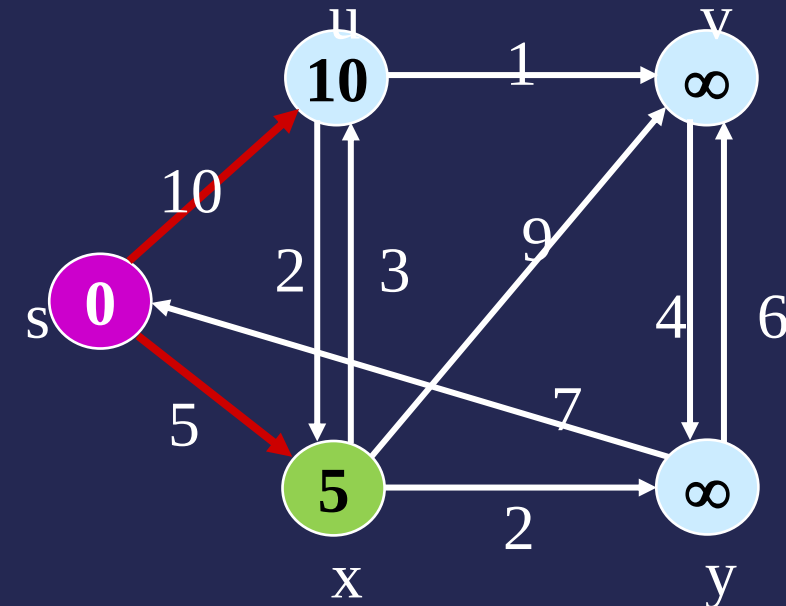
4. while  $Q \neq \emptyset$  do

5.      $u = \text{Extract-Min}(Q)$ ;

6.      $S = S \cup \{u\}$ ;

7.     for each vertex  $v \in G.\text{Adj}[u]$   
do

8.         Relax(u, v, w)

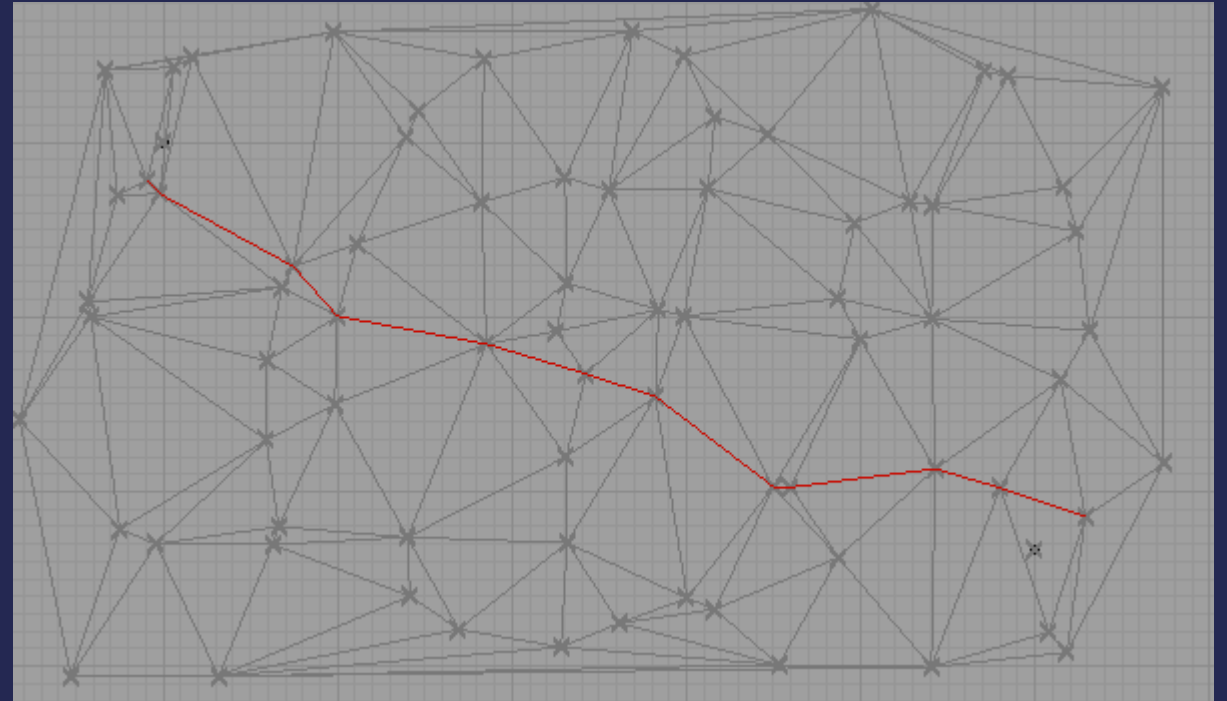


Update path & Distance from starting node s to v using u.





# *ALL PAIR SHORTEST PATH FLOYD WARSHALL ALGORITHM*



*Mr. Prayag Patel*

*Assistant Professor  
IT-ICT Department, LJIET  
[prayag.patel@ljinstitutes.edu.in](mailto:prayag.patel@ljinstitutes.edu.in)*



# *All Pair Shortest Path*

## *Types of algorithms for All Pair Shortest Path*

1) Matrix multiplication and Shortest path

2) Floyd warshall algorithm



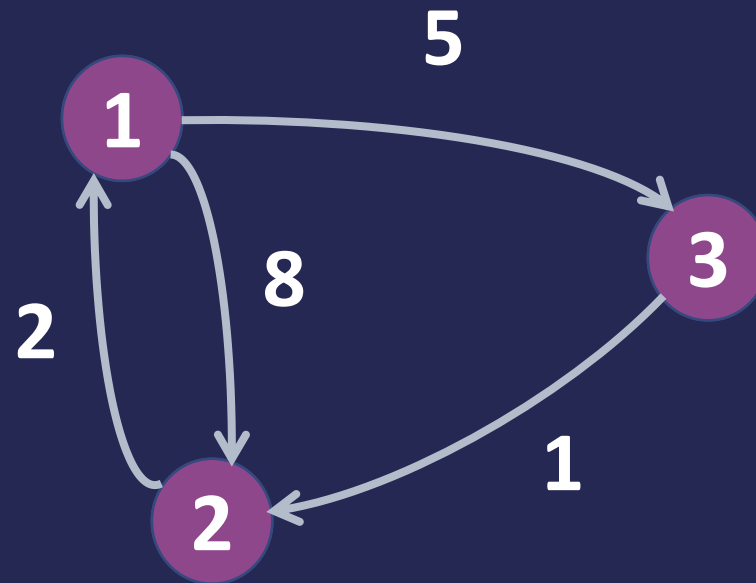
## 2) *Floyd warshall algorithm*

- Floyd's algorithm is for finding the shortest path between every pair of vertices of a graph.
- The Graph is represented by adjacency matrix where the number of rows and columns equals the number of vertices.
- If there is an edge between  $i$  to  $j$  then the  $M[i][j] = W_{ij}$  , where  $W_{ij}$  represents the weight of edge between  $i$  and  $j$ .



# *Floyd warshall algorithm*

*Example: Find the all pair shortest path for given graph using Floyd warshall algorithm.*

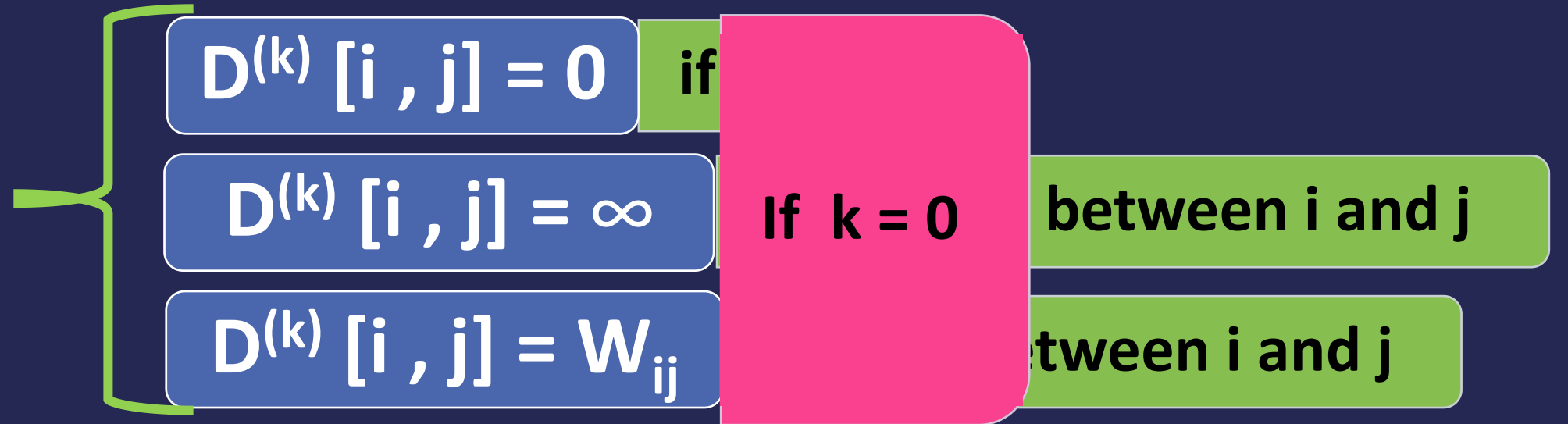




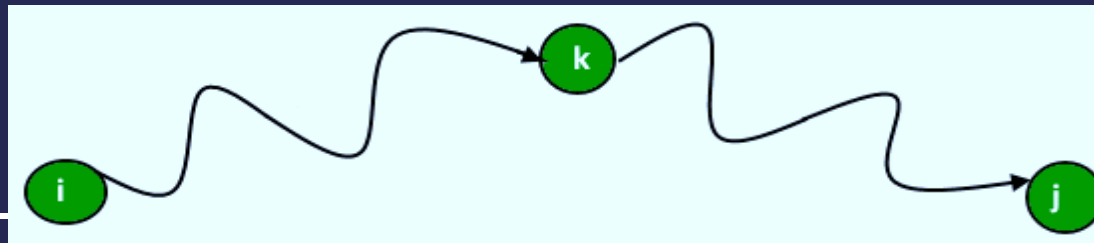
# Floyd warshall algorithm

*Step-1: Characterize the structure of an optimal solution*

*Formula for Floyd warshall algorithm*



$$D^{(k)} [i, j] = \min \{ D^{(k-1)} [i, j], D^{(k-1)} [i, k] + D^{(k-1)} [k, j] \}$$



if  $k > 0$

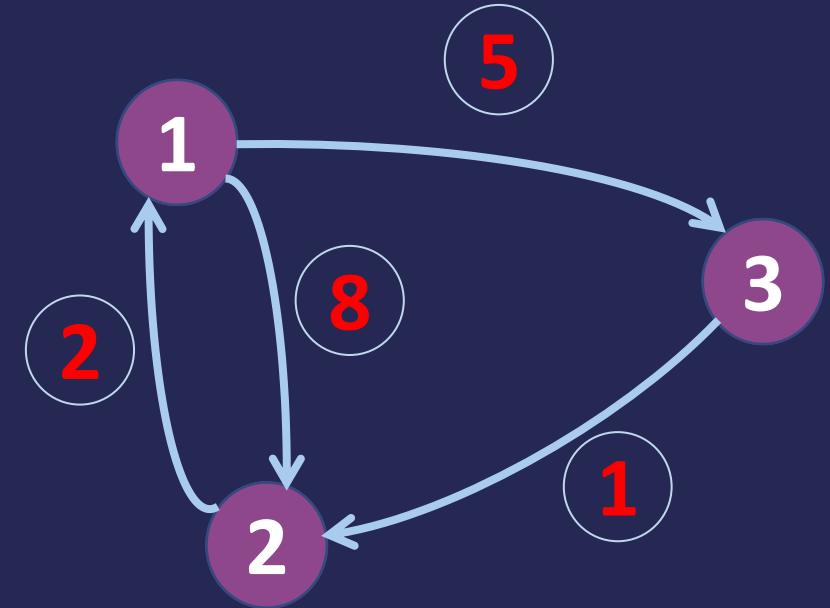


# Floyd warshall algorithm

Step-2: Consider Direct Distance

$D^{(0)}$  =

|   | 1        | 2        | 3        |
|---|----------|----------|----------|
| 1 | 0        | $\infty$ | $\infty$ |
| 2 | $\infty$ | 0        | $\infty$ |
| 3 | $\infty$ | $\infty$ | 0        |



$$D^{(k)}[i, j] = 0$$

if  $i = j$

$$D^{(k)}[i, j] = \infty$$

if no edge between  $i$  and  $j$

$$D^{(k)}[i, j] = W_{ij}$$

if edge between  $i$  and  $j$

If  $k = 0$

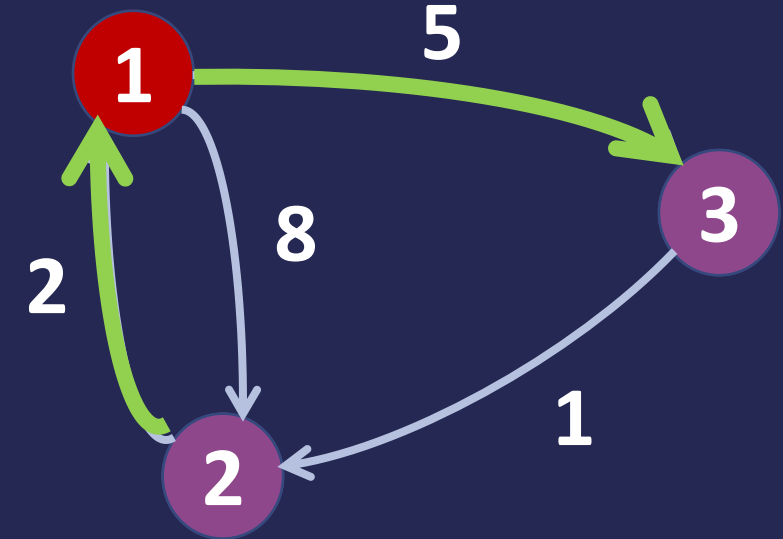


# Floyd warshall algorithm

Step-3: Consider Node-1 as an intermediate node

$D^{(1)} =$

|   | 1        | 2 | 3        |
|---|----------|---|----------|
| 1 | 0        | 8 | 5        |
| 2 | 2        | 0 | $\infty$ |
| 3 | $\infty$ | 1 | 0        |



$$\begin{aligned} D^{(1)} [2, 3] &= \min \{ D^{(0)} [2, 3], D^{(0)} [2, 1] + D^{(0)} [1, 3] \} \\ &= \min \{ \infty, 2 + 5 \} \\ &= \min \{ \infty, 7 \} \\ &= 7 \end{aligned}$$

$$D^{(k)} [i, j] = \min \{ D^{(k-1)} [i, j], D^{(k-1)} [i, k] + D^{(k-1)} [k, j] \}$$

if  $k > 0$

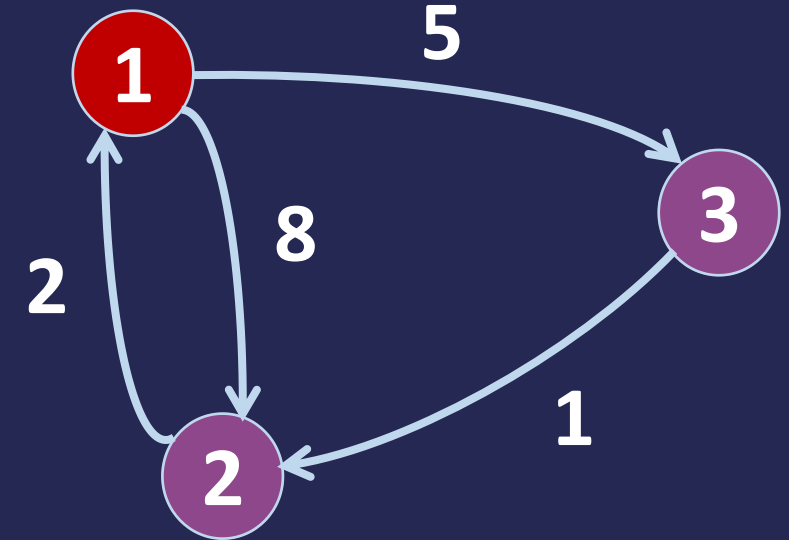


# Floyd warshall algorithm

Step-3: Consider Node-1 as an intermediate node

$D^{(1)} =$

|   | 1        | 2 | 3 |
|---|----------|---|---|
| 1 | 0        | 8 | 5 |
| 2 | 2        | 0 | 7 |
| 3 | $\infty$ | 1 | 0 |



$$D^{(k)} [i, j] = \min \{ D^{(k-1)} [i, j], D^{(k-1)} [i, k] + D^{(k-1)} [k, j] \} \quad \text{if } k > 0$$



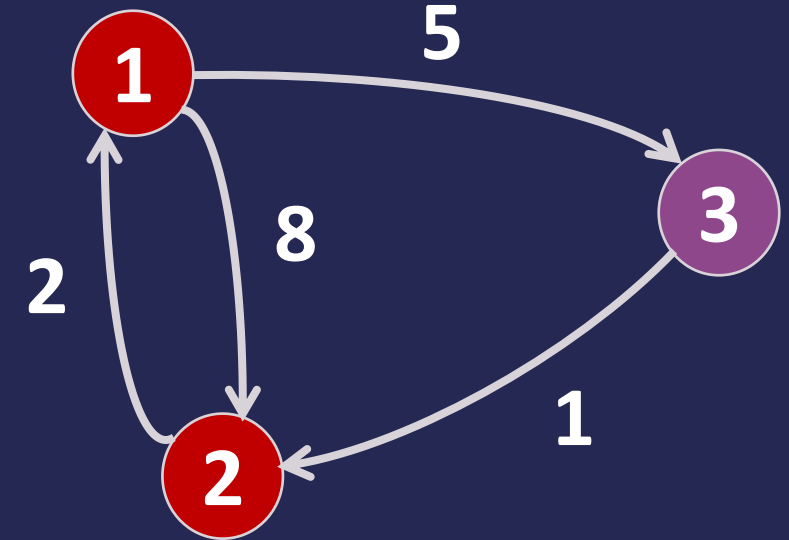


# Floyd warshall algorithm

Step-4: Consider Node-2 as an intermediate node

$D^{(2)} =$

|   | 1        | 2 | 3 |
|---|----------|---|---|
| 1 | 0        | 8 | 5 |
| 2 | 2        | 0 | 7 |
| 3 | $\infty$ | 1 | 0 |



$$D^{(k)} [i, j] = \min \{ D^{(k-1)} [i, j], D^{(k-1)} [i, k] + D^{(k-1)} [k, j] \} \quad \text{if } k > 0$$

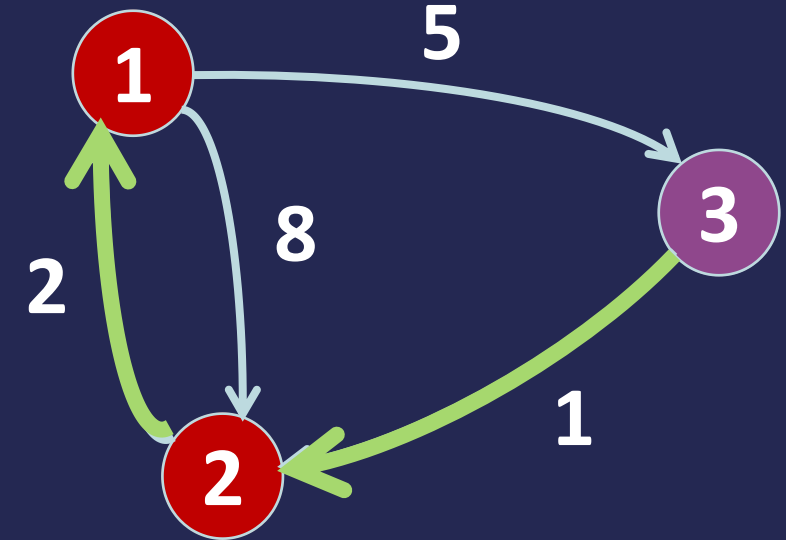


# Floyd warshall algorithm

Step-4: Consider Node-2 as an intermediate node

$D^{(2)} =$

|   | 1        | 2 | 3 |
|---|----------|---|---|
| 1 | 0        | 8 | 5 |
| 2 | 2        | 0 | 7 |
| 3 | $\infty$ | 1 | 0 |



$$\begin{aligned} D^{(2)} [3, 1] &= \min \{ D^{(1)} [3, 1], D^{(1)} [3, 2] + D^{(1)} [2, 1] \} \\ &= \min \{ \infty, 1 + 2 \} \\ &= \min \{ \infty, 3 \} \\ &= 3 \end{aligned}$$

$$D^{(k)} [i, j] = \min \{ D^{(k-1)} [i, j], D^{(k-1)} [i, k] + D^{(k-1)} [k, j] \} \quad \text{if } k > 0$$

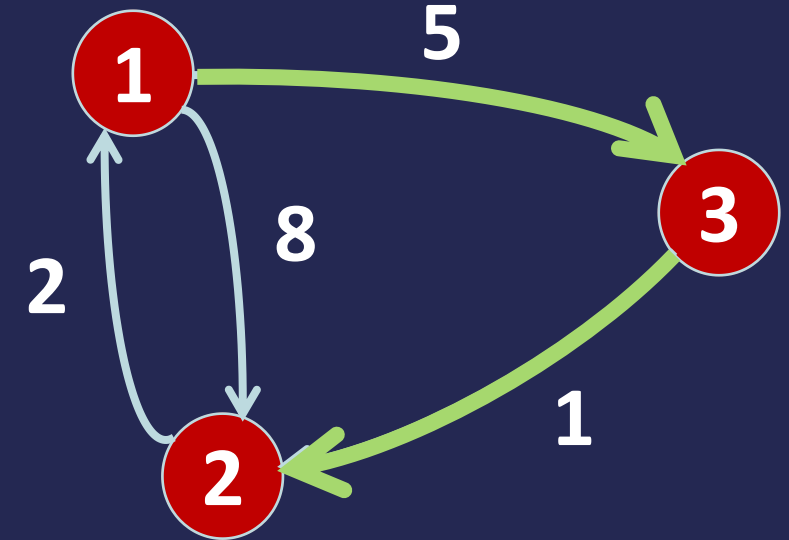


# Floyd warshall algorithm

Step-5: Consider Node-3 as an intermediate node

$D^{(3)} =$

|   | 1 | 2 | 3 |
|---|---|---|---|
| 1 | 0 | 8 | 5 |
| 2 | 2 | 0 | 7 |
| 3 | 3 | 1 | 0 |



$$\begin{aligned} D^{(3)} [1, 2] &= \min \{ D^{(2)} [1, 2], D^{(2)} [1, 3] + D^{(2)} [3, 2] \} \\ &= \min \{ 8, 5 + 1 \} \\ &= \min \{ 8, 6 \} \\ &= 6 \end{aligned}$$

$$D^{(k)} [i, j] = \min \{ D^{(k-1)} [i, j], D^{(k-1)} [i, k] + D^{(k-1)} [k, j] \} \quad \text{if } k > 0$$

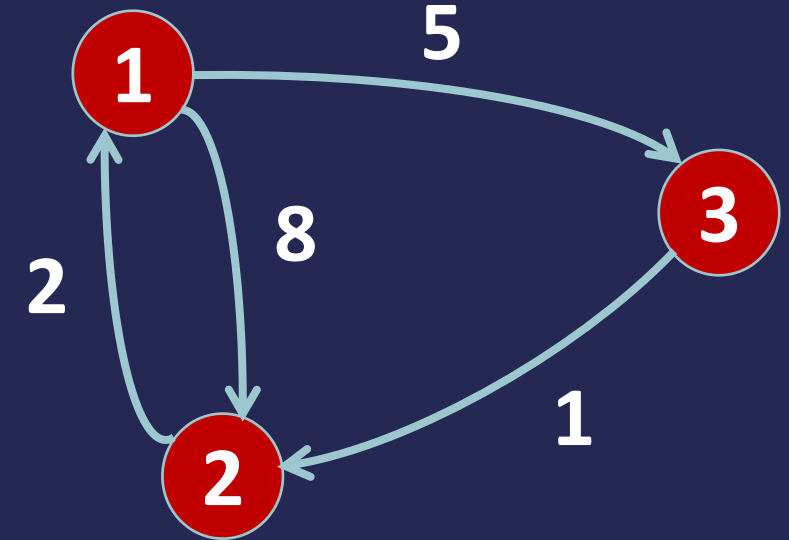


# Floyd warshall algorithm

Step-5: Consider Node-3 as an intermediate node

$D^{(3)} =$

|   | 1 | 2 | 3 |
|---|---|---|---|
| 1 | 0 | 6 | 5 |
| 2 | 2 | 0 | 7 |
| 3 | 3 | 1 | 0 |



$$D^{(k)} [i, j] = \min \{ D^{(k-1)} [i, j], D^{(k-1)} [i, k] + D^{(k-1)} [k, j] \} \quad \text{if } k > 0$$

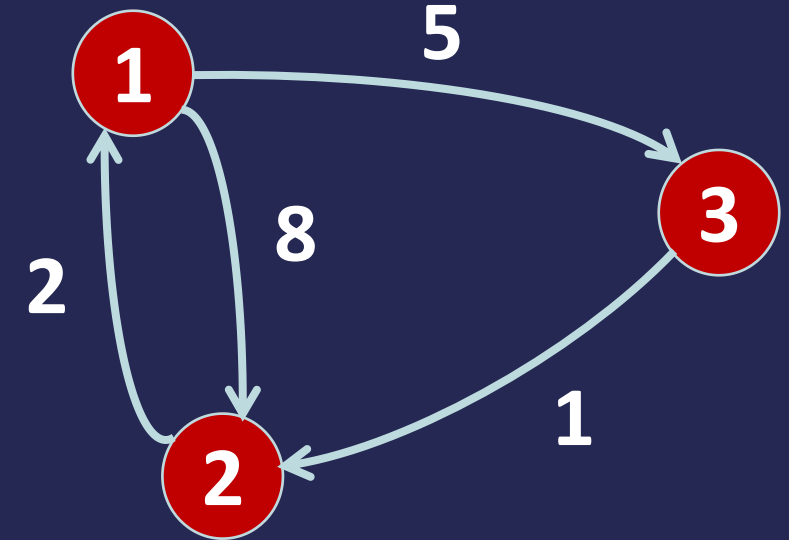


# Floyd warshall algorithm

Shortest path can be obtained as:

$D^{(3)} =$

|   | 1 | 2 | 3 |
|---|---|---|---|
| 1 | 0 | 8 | 5 |
| 2 | 2 | 0 | 7 |
| 3 | 3 | 1 | 0 |



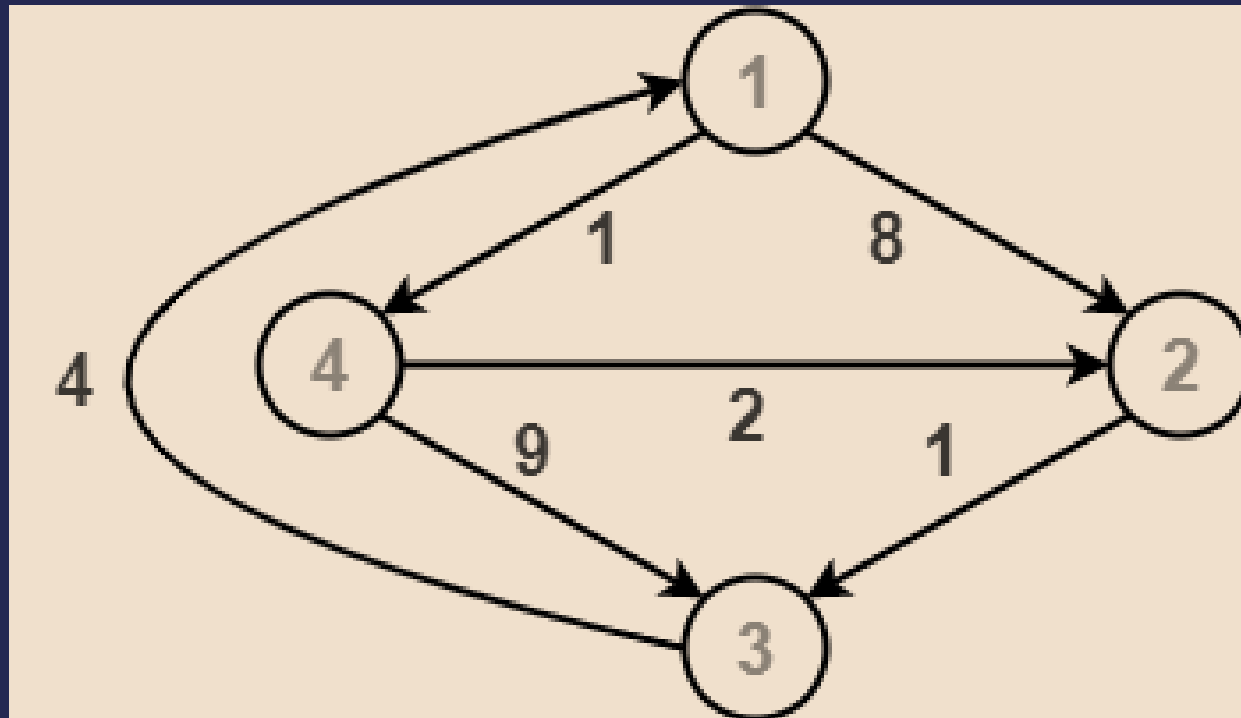
$$D^{(k)} [i, j] = \min \{ D^{(k-1)} [i, j], D^{(k-1)} [i, k] + D^{(k-1)} [k, j] \}$$

if  $k > 0$



# *Floyd warshall algorithm*

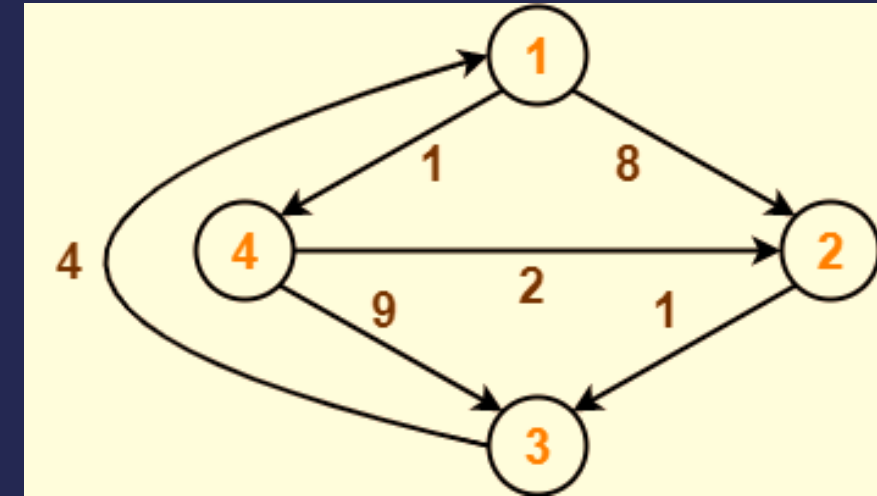
*Example: Find the all pair shortest path for given graph using Floyd warshall algorithm.*





# Floyd warshall algorithm

Shortest path can be obtained as:

$$D_1 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 8 & \infty & 1 \\ \infty & 0 & 1 & \infty \\ 4 & 12 & 0 & 5 \\ \infty & 2 & 9 & 0 \end{bmatrix} \end{matrix}$$


$$D_2 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 8 & 9 & 1 \\ \infty & 0 & 1 & \infty \\ 4 & 12 & 0 & 5 \\ \infty & 2 & 3 & 0 \end{bmatrix} \end{matrix}$$

$$D_3 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 8 & 9 & 1 \\ 5 & 0 & 1 & 6 \\ 4 & 12 & 0 & 5 \\ 7 & 2 & 3 & 0 \end{bmatrix} \end{matrix}$$

$$D_4 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 4 & 1 \\ 5 & 0 & 1 & 6 \\ 4 & 7 & 0 & 5 \\ 7 & 2 & 3 & 0 \end{bmatrix} \end{matrix}$$

*THANK YOU FOR  
WATCHING!*